

Sparse MoE 训练实战

从 top-k gating 到 EP + All-to-All 的完整指南

duo.an

2026-09-03

目录

1	前言	5
1.1	这本书是写给谁的	5
1.2	这本书讲什么	5
1.3	怎么读	5
1.4	记号约定	6
1.5	与相关材料的关系	6
2	MoE 是什么，为什么要有	7
2.1	一句话直觉	7
2.2	为什么 dense 撑不下去：scaling law 的视角	7
2.3	MoE 的核心思想：条件计算	7
2.4	代价：三个新问题	8
2.5	一张图看清 MoE 层的位置	8
2.6	里程碑速览	9
2.7	本书 optimization ladder 概览	9
2.8	面试考点	10
3	Router / Top-K Gating	11
3.1	数学形式	11
3.2	用一个具体例子拉一遍 tensor	11
3.3	Router 的三种变体	12
3.4	Backward：梯度怎么流？	13
3.5	为什么 softmax 要用 fp32	13
3.6	Router 的 kernel 视角	14
3.7	Router 的可视化 debug	14
3.8	面试考点	15
4	Dispatch 与 Combine 的三种实现范式	17
4.1	问题精确化	17
4.2	范式 A：Scatter / Gather（本书教学实现）	17
4.3	范式 B：Permute + Grouped GEMM（生产级）	19
4.4	范式 C：Block-Sparse GEMM（Megablocks）	22
4.5	三种范式对比	23
4.6	选择建议	24
4.7	面试考点	24
5	逐行 Code Walkthrough (test_moe.py)	26
5.1	全文件结构	26
5.2	Part 1: Expert 定义	26
5.3	Part 2: MoE 主体——init	27
5.4	Part 3: MoE forward——展开和 router	28
5.5	Part 4: MoE forward——dispatch 循环	29
5.6	Part 5: 收尾	30
5.7	两个 smoke test	31
5.8	完整 tensor 生命周期图	31
5.9	常见改错练习	32

5.10	面试考点	33
6	Load Balancing : Aux Loss、Z-Loss、Bias Tuning	35
6.1	问题重述 : Expert Collapse	35
6.2	方案 1: GShard/Switch Aux Loss	35
6.3	方案 2: Router Z-Loss (ST-MoE)	37
6.4	方案 3: DeepSeek-V3 的 Aux-Loss-Free 均衡	37
6.5	方案对比	39
6.6	Expert Choice : 反向路由	39
6.7	实操 : 监控 collapse	40
6.8	面试考点	41
7	单机训练性能优化	43
7.1	瓶颈画像 : 先看开销分布	43
7.2	优化 ladder	43
7.3	v1: Grouped GEMM	44
7.4	v2: Fused Router	45
7.5	v3: Fused Permute	47
7.6	v4: Fused SwiGLU	49
7.7	v5: Block-Sparse (Megablocks)	50
7.8	v6: torch.compile	50
7.9	显存优化	51
7.10	面试考点	51
8	分布式训练 : EP、All-to-All 与 Fine-Grained Overlap	53
8.1	MoE 训练的三堵墙 (Three Walls)	53
8.2	四种并行的分工	54
8.3	为什么需要 Expert Parallel	55
8.4	Wall 1 · Memory Wall : 显存怎么省	55
8.5	EP 数据流 : 从单机到 EP	57
8.6	通信量分析	61
8.7	Wall 2 · Communication Wall : a2a 怎么打	62
8.8	EP × TP × DP 组合	64
8.9	Overlap: 隐藏通信	65
8.10	Wall 3 · Compute Efficiency Wall : kernel 碎片化	68
8.11	NCCL 调优	69
8.12	主要框架	69
8.13	完整训练配置样例	72
8.14	常见坑	74
8.15	面试考点	74
9	模型层与低精度 : Fine-grained、Shared、MLA、FP8	76
9.1	细粒度专家 + Shared Expert : DeepSeekMoE 范式	76
9.2	LatentMoE : 把 hidden 压掉一部分	77
9.3	Multi-head Latent Attention (MLA)	78
9.4	FP8 训练 : DeepSeek-V3 recipe	78
9.5	生命周期 : Upcycling 与 Pruning / Merging	80
9.6	面试考点	81
10	数值稳定性、精度与常见陷阱	83

10.1 分类	83
10.2 数值精度类	83
10.3 边界情况类	84
10.4 路由 semantics 类	85
10.5 Autograd 类	86
10.6 分布式类	86
10.7 训练稳定性与调试	87
10.8 常见坑速查表	88
10.9 面试考点	88
11 附录 A：符号表 & 延伸阅读	90
11.1 符号表	90
11.2 延伸阅读	90
11.3 本仓库文件	91
12 附录 D：MoE 系统面试题 20 问	93
12.1 基础概念 (Q1-Q5)	93
12.2 Router 与负载均衡 (Q6-Q10)	93
12.3 实现与性能 (Q11-Q15)	94
12.4 分布式 (Q16-Q20)	95
12.5 加分题：动手实操	96

前言

这本书是写给谁的

这本书面向已经写过 Transformer、想把 **Sparse Mixture-of-Experts (MoE)** 从“看过论文”读到“能自己写、能上分布式、能在面试里被系统性追问”那种深度的读者。

在阅读本书之前，你应该：

- 会用 PyTorch，读得懂 `nn.Linear` / `nn.Module`。
- 熟悉标准 Transformer block：Attention + FFN + LayerNorm + residual。
- 对 `torch.softmax` / `torch.topk` / `torch.where` / `index_add_` 这些张量操作有基本认知。
- 对分布式训练的 DP / TP / PP 三种并行有一个概念——不熟也没关系，第 8 章会把 EP (Expert Parallel) 从零推一遍。

如果 Transformer FFN 层完全没接触过，建议先读姊妹卷《CUDA Kernel 优化实战》第 6 章 (MLP)，回来再读本书。

这本书讲什么

MoE 表面看是“把一个大 FFN 拆成 E 份，每 token 只走 K 份”，但真正的复杂度在工程：

- Router 怎么设计？softmax + topk 就够了吗？
- 每个 batch 里 M 个专家的 token 数是变量，怎么做 batched GEMM？
- 单机 8 卡时 experts 怎么分？跨机时 all-to-all 怎么 overlap？
- 训练不稳定怎么办？专家坍塌怎么办？

每一章围绕一个具体主题展开——从零基础的路由直觉，到 grouped GEMM 的实现，再到 DeepSeek-V3 那种规模的分布式训练。每章都会讲清楚：

- 这一步为什么需要（对比朴素方案的缺陷）。
- 具体实现里每个 **tensor** 的 **shape** 变化（配 tensor 可视化图）。
- 什么场景用不上（不是所有优化都总有用）。
- 面试里常见的追问。

怎么读

- 顺序读：章节间有依赖。第 5 章 code walkthrough 用到第 3、4 章的路由 & dispatch 概念；第 7 章单机性能优化建立在第 4 章的 dispatch 范式上；第 8 章分布式建立在第 7 章的 grouped GEMM 之上。
- 配代码读：所有代码来自本仓库 `python/pytorch/test_moe.py`，可以直接跑：

```
python python/pytorch/test_moe.py
```

第 5 章会逐行讲解这份代码。

- 把面试考点框读进去：每章末尾用紫色框标出常见面试追问，答案在正文里都能找到依据。附录 D 汇总了 20 道系统面试题（含参考答案）。

记号约定

- B, S, H : batch size / sequence length / hidden dim
- $N = B \times S$: flatten 之后的 token 总数
- E : 专家总数 (num_experts)
- K : top-k , 每个 token 激活的专家数
- M_e : 路由到专家 e 的 token 数 (负载不均 , $\sum_e M_e = NK$)
- I : FFN 内的 intermediate dim (通常 $I \approx 4H$)
- C : capacity , 每个专家在 batch 内允许接收的最大 token 数
- EP / TP / DP / PP : expert / tensor / data / pipeline parallel 的世界大小
- 张量 shape 用 (a, b, c) 标注 , 如 expert_indices: (N, K)

性能数字除非注明, 都在 A100 80GB SXM4 / H100 SXM5 上给出量级估算, 不承诺精确复现。生产系统请以 nsys / ncu 或框架自带 profiler 为准。

与相关材料的关系

- 本书假设的最小 PyTorch 实现在 python/pytorch/test_moe.py , 是理解 MoE 的“读物”, 不是生产实现。
- 生产实现细节参考 Megatron-LM MoE、DeepSpeed-MoE、Tutel、Megablocks、vLLM fused_moe。本书讲为什么这些框架要这样做, 源码级细节请直接读代码。
- Mixtral、DeepSeek-V3 等具体模型的 checkpoint 分析在附录 A。

好, 翻页开始。

MoE 是什么，为什么要有

在动手写代码前，先花一章建立直觉——为什么 dense Transformer 撑不下去、稀疏 MoE 到底解决什么问题、代价是什么。这一章不涉及具体实现，但每一段都对应后面章节要动手做的事。

一句话直觉

Dense FFN：每个 token 都走同一个大 FFN。参数量 = 计算量。

MoE FFN：把大 FFN 拆成 E 个“专家”（每个是小 FFN），每个 token 只走其中 K 个（ $K \ll E$ ）。

结果：总参数量 $\times \frac{E}{K}$ ，激活计算量几乎不变。这是 MoE 的核心卖点——“同样的 FLOPs，更大的模型”。

用图对比一下（ $E = 4, K = 1$ ）：



Dense: 每个 token 都过唯一的大 FFN



MoE ($K=1$): 每个 token 只激活 4 个专家中的 1 个

为什么 dense 撑不下去：scaling law 的视角

从 Kaplan 2020 / Chinchilla 2022 起，我们已经很清楚：**loss** 关于参数量 P 、数据量 D 、计算量 C 是幂律关系。想让 loss 更低，只有三条路：加参数、加数据、加计算。

Dense 架构的问题：三者是强绑定的。

$$C_{\text{forward}} \approx 2PN_{\text{tokens}}$$

要更强的模型，就得更多参数；参数一多，同数据下的 FLOPs 就同步增加。训练一个 dense 70B 用 15T token 需要约 $C = 2 \times 7 \times 10^{10} \times 1.5 \times 10^{13} = 2.1 \times 10^{24}$ FLOP——这大概是 6000× H100 训 3 个月。要 700B？直接 10 倍。

Key insight. Dense scaling 的“物理”局限：每个 **token** 都要读一遍所有参数。参数量线性 → 每 token 内存流量线性 → 训推成本线性。想打破这个绑定，必须让“读多少参数”和“总参数量”解耦——这就是 conditional computation 的思想。

MoE 的核心思想：条件计算

Conditional Computation (Bengio et al. 2013, Shazeer et al. 2017)：不是所有 token 都需要全部知识。给每个 token 一个“router”，只激活相关子网络。

用数学讲：把一个 FFN 变成 E 个 FFN 的加权和，但权重是稀疏的：

$$y = \sum_{e=1}^E g_e(x) \cdot \text{FFN}_e(x)$$

其中 $g_e(x) \in [0, 1]$ 且 $\sum_e g_e = 1$ ，且 g 只在 K 个专家上非零——其他专家不计算，等于 0 加进去。

于是：

$$P_{\text{total}} = E \cdot P_{\text{expert}} \quad (\text{模型容量})$$

$$P_{\text{active}} = K \cdot P_{\text{expert}} \quad (\text{每 token 计算})$$

$$\text{稀疏度} = \frac{K}{E}$$

Mixtral 8×7B : $E = 8, K = 2 \rightarrow$ 稀疏度 25% , 总 47B 参数、激活 13B。DeepSeek-V3 : $E = 256 + 1, K = 8 + 1 \rightarrow$ 稀疏度 3.5% , 671B/37B。

Note. Mixtral 名字里“8×7B”是营销术语——不是 8 份 7B 参数相加。每个专家共享 attention/embedding，专家只是 FFN 部分。真实参数约 47B，激活约 13B (含 attention)。

代价：三个新问题

天下没有免费的午餐。MoE 引入三个 dense 时代没有的问题：

1. 路由决策要学

Router 是可学习的，训练早期它可能给出“总是选专家 0”的解——其他专家收不到梯度，等价于浪费 $E - 1$ 份参数。解决方案是负载均衡损失（第 6 章）或 DeepSeek 那种 auxiliary-loss-free 的 bias tuning。

2. 负载天然不均

即使 router 收敛良好，某个 batch 里恰好大部分 token 都路由到 expert 0 是完全可能的。这意味着：

- 显存里 E 个专家权重都得存
- 但计算时只有 1-2 个专家真在忙，其他 SM 空闲
- 大 batch 下“每个专家的 batch size”是 skewed 分布——GEMM 的 M 维忽大忽小

第 7 章会讲怎么用 grouped GEMM / block-sparse 应对。

3. 分布式通信复杂

Dense 训练里，一个 FFN 的通信量就是权重梯度 all-reduce。MoE 里如果把 experts 分布在多张卡上 (expert parallel)，每一层都要做两次 **all-to-all**——一次把 token 送到对的专家、一次把结果送回来。第 8 章是这本书最长的章节，就在讲这件事。

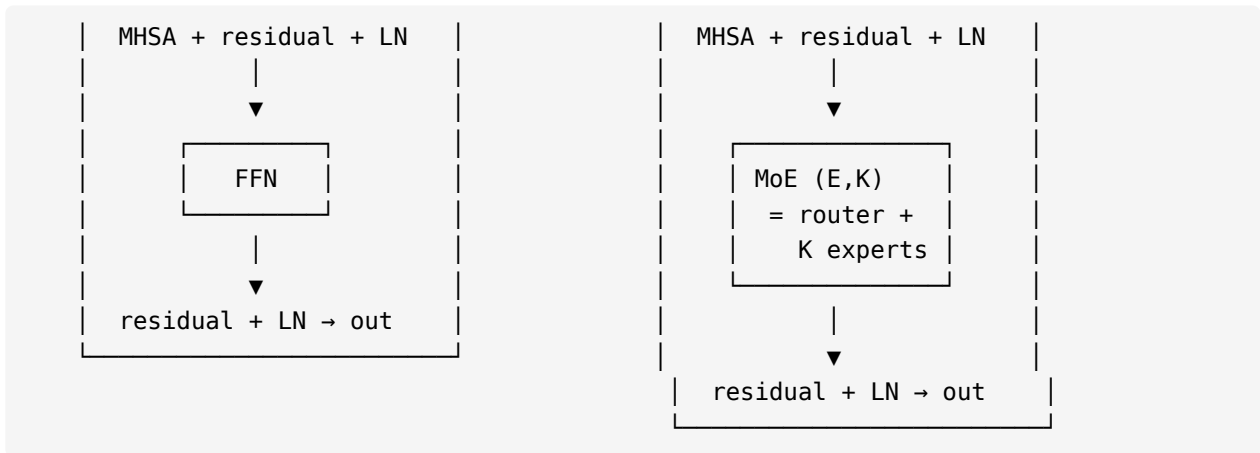
Warning. 常见误解：既然专家稀疏，MoE 训推都会更便宜。错。MoE 只是 FLOP 便宜；显存开销（要装下所有专家）和通信开销（all-to-all）都比 dense 更高。所以 MoE 是“更大模型，同 FLOPs”，不是“同能力，更少资源”。

一张图看清 MoE 层的位置

MoE 层是 **Transformer block** 内 **FFN** 子层的替换品，其他部分 (attention、LN、residual) 不变：

Transformer block (dense):

Transformer block (MoE):



替换粒度可以是“每层都换”（Switch Transformer、Mixtral）或“每隔 2 层换一层”（GShard、DeepSeek-MoE 部分变种）。

里程碑速览

模型	年份	E	K	特色
Sparsely-Gated MoE	2017	> 1000	2	开山，per-layer huge E
GShard	2020	2048	2	翻译，capacity + drop
Switch Transformer	2021	$> 10^4$	1	$K = 1$ 极简路由
GLaM	2021	64	2	1.2T 参数
Mixtral 8×7B	2023	8	2	开源 SOTA，SwiGLU expert
DBRX	2024	16	4	细粒度专家
DeepSeek-V2 / V3	2024	160 + / 256 +	6/8	细粒度 + shared expert
Qwen-1.5-MoE	2024	60+	4	上采样自 dense

表 1 MoE 里程碑。近年趋势：单层专家数从“上千”降到“几十 几百”（细粒度专家）， K 从 1 升到 6-8（表达力）。

本书 **optimization ladder** 概览

作为 metadata：本书讲的“MoE”优化 ladder 大致长这样（每个 rung 后面会详细讲）：

版本	核心思路	相对成本 / 收益
v0: naive scatter/gather	for-loop E 个专家 + `where` + `index_add`	教学； $E \leq 16$
v1: grouped GEMM	permute 到 packed 排序， 一次 kernel 跑完 E 个专家	生产单机；2-4× 加速
v2: fused router	Linear+softmax+topk+reduction 融成 1 个 kernel	轻量收益，减一次 HBM 读写
v3: block-sparse (Megablocks)	无 padding、无 drop token	vs v1 更好的 skew 容忍

v4: EP + async all-to-all	跨卡专家并行，通信/计算 overlap	8+ 卡训练必需
v5: fine-grained overlap (DSv3)	dispatch/attention/mlp/combine 四阶段 pipeline	1000+ 卡训练

后面每一章基本都在这条 ladder 上爬。第 5 章的教学代码对应 v0 ;第 7 章讲 v1-v3 ;第 8 章讲 v4-v5。

面试考点

面试考点. **Q1**: MoE 相比 dense 的核心权衡是什么？

A: 参数量和计算量解耦。用 E 倍参数换 $\frac{K}{E}$ 的 FLOPs (相对同参数量的 dense), 但代价是：显存要装完整参数 ($E \times P_{\text{expert}}$)，多一层路由计算、跨卡时 all-to-all 通信开销。

面试考点. **Q2**: “Mixtral 8×7B” 是不是 8 个独立的 7B 模型？

A: 不是。只有 FFN 部分是 8 份 experts；attention、embedding、LN 都共享。实际总参数量约 47B，激活约 13B。名字来源于“8 experts, each 7B params in FFN”的营销修辞。

面试考点. **Q3**: 稀疏 MoE 和 ensemble 有什么区别？

A: 关键区别是路由。Ensemble 让每个成员看所有输入、取平均或投票；MoE 让 router 学会把每个 token 送到相应专家，专家专注不同“技能”。而且 MoE 只激活 K 个专家，ensemble 全跑一遍。

面试考点. **Q4**: 为什么现在的 MoE 都是“few experts, larger each” (Mixtral 8 expert) 而不是 Shazeer 2017 的“many tiny experts”？

A: 早期 many-tiny 在小 batch 下每个专家几乎无 token，GEMM 内存 bound + launch overhead 主导。少而大的专家 (few large) 在同参数量下 GEMM 密度更高。但 DeepSeek 又转回“many fine-grained” + shared expert 的混合方向——工程 trade-off 还在演化，没有唯一正确答案。

Router / Top-K Gating

Router 是 MoE 的“大脑”——每一 forward 都要给每个 token 决定“去哪些专家”。这是一个每步都执行、每层都执行、结果直接影响后续所有计算的模块。这一章我们把 router 从数学公式一直讲到 kernel 实现。

数学形式

给定 hidden state $x \in \mathbb{R}^H$ (单个 token), router 是：

$$\ell = W_g x \quad (\text{logits}, \ell \in \mathbb{R}^E)$$

$$p = \text{softmax}(\ell) \quad (\text{probs}, p \in \mathbb{R}^E)$$

$$(w, i) = \text{top-k}(p, K) \quad (w \in \mathbb{R}^K, i \in \mathbb{Z}^K)$$

$$\tilde{w} = \frac{w}{\sum_j w_j} \quad (\text{renormalize})$$

其中 $W_g \in \mathbb{R}^{E \times H}$ 是唯一的可学习参数——每层 MoE 一个。相对于专家 FFN, router 的参数数量小到可以忽略 ($EH \approx 8 \times 4096 = 3 \times 10^4$ vs 单专家 $\approx 10^8$)。

Batch 版本：输入是 $X \in \mathbb{R}^{N \times H}$ ($N = B \times S$), 输出是：

- `expert_indices`: (N, K) — 每个 token 选的专家 id
- `expert_weights`: (N, K) — 每个 token 分给这些专家的权重 ($\sum = 1$)

对应 `test_moe.py`：

```
gate_logits = self.gate(hidden_states) # (N, E)
gate_probs = F.softmax(gate_logits, dim=-1, dtype=torch.float32).to(dtype)
expert_weights, expert_indices = torch.topk(gate_probs, k=self.top_k, dim=-1)
expert_weights = expert_weights / expert_weights.sum(dim=-1, keepdim=True)
```

用一个具体例子拉一遍 **tensor**

以 $N = 4, E = 4, K = 2$ 为例。设 gate 输出 (softmax 后) 是：

gate_probs (N=4, E=4), red border = top-2

	E0	E1	E2	E3
t0	0.1	0.6	0.05	0.25
t1	0.4	0.1	0.45	0.05
t2	0.05	0.2	0.15	0.6
t3	0.3	0.3	0.3	0.1

Top-2 选出：

token	top-2 experts	raw probs	after renorm	说明
t0	E1, E3	0.60, 0.25	0.706, 0.294	清晰主选
t1	E2, E0	0.45, 0.40	0.529, 0.471	两个专家分数接近
t2	E3, E1	0.60, 0.20	0.750, 0.250	清晰主选
t3	E0, E1	0.30, 0.30	0.500, 0.500	路由熵高（“没主见”）

Renormalization 前后 数学不同：

- Renorm 前的 w ：反映专家在全 E 个里的相对置信；t3 的 (0.30, 0.30) 意味着 router 对 t3 没信心。
- Renorm 后的 $\tilde{w}$ ：反映专家在选中的 K 个里的相对权重；t3 的 (0.5, 0.5) 意味着两个专家平分。

Key insight. Renorm 会丢失“router 对这个 token 有多少信心”这个信息。生产实现在这一步往往有两种取舍：Mixtral 用 `softmax(topk(logits))` 保留熵信息；GShard/本书 demo 用 renorm。DeepSeek-V3 用 `sigmoid + bias`（详见第 6 章“DeepSeek-V3 的 Aux-Loss-Free 均衡”），完全绕开了这个 trade-off。

从张量形状看，一次 router 前向：



Router 的三种变体

上面是最标准的一种。生产系统会看到几个变体：

(A) softmax → topk → renorm (本书 demo, GShard)

上文详解，保留所有 E 个 logits 的相对信息，然后砍掉尾巴。

(B) topk(logits) → softmax over K (Mixtral)

```

topk_logits, expert_indices = torch.topk(gate_logits, k=K, dim=-1)
expert_weights = F.softmax(topk_logits, dim=-1)

```

不做全 E 的 **softmax**，直接对 top-k logits 做局部 softmax。计算量小（只对 K 个 exp），保留熵信息。缺点：反向传播时未选中的 $E - K$ 个 logits 完全无梯度——router 早期训练收敛更依赖 aux loss。

(C) sigmoid gating (Switch 变种, DeepSeek-V3)

```
gate_scores = torch.sigmoid(gate_logits) # 每个专家独立打分，无归一
```

不做 softmax，各专家独立 sigmoid 打分。DeepSeek-V3 在这个基础上加了动态 bias（详见第 6 章），实现无 aux loss 的负载均衡。

三种变体的差异不体现在推理精度上（可以互相 fine-tune 转换），但训练动力学差异明显：

方案	所有 logits 有梯度？	保留熵信息？	aux loss 依赖度
A. softmax+topk+renorm	是 (通过 softmax)	否 (被 renorm 抹掉)	中
B. topk+softmax	否 (只有 top-K)	是	高
C. sigmoid + bias	是	是 (无归一)	无 (用 bias 替代)

Backward：梯度怎么流？

Router 的反向传播比 dense FFN 微妙——因为 topk 是不可微操作。

事实：torch.topk 的 backward 只把梯度传给“被选中的 K 个位置”，其他 $E - K$ 位置的 logit 梯度是 0。

以变体 (A) 为例， $\partial_{\theta} \ell_j$ 只在 j 是 top-k 之一时非零，梯度沿“prob-space renorm $\rightarrow$ softmax”链回传。

结果：如果 router 从未选过某个专家 e ，那么 ℓ_e 从未获得直接梯度信号——router 学不会去选它。这是 **expert collapse** 的直接来源。

Warning. “梯度只流 top-k”是 MoE 的基本事实，不是 bug 也不是数值噪声。补救靠三条：(1) aux loss（第 6 章）；(2) router z-loss；(3) 权重初始化 + warmup，让早期路由本身就分散。DeepSeek 用可学习 bias 是第四条路。

一个反直觉的问题：既然梯度只流 top-k，为什么 Mixtral 变体 (B) 训练还能收敛？

答：靠 aux loss 给 router 一个“往均匀分布拉”的梯度——这个梯度是对 p_i 求的（含所有 E 个），会流回所有 logit。详见第 6 章的 aux loss 一节。

为什么 softmax 要用 fp32

test_moe.py 里这一句：

```
gate_probs = F.softmax(gate_logits, dim=-1, dtype=torch.float32).to(dtype)
```

明确指定 dtype=torch.float32。这不是可选优化，是正确性要求。

原因：softmax 里的 $\exp(x)$ 在 fp16/bf16 下容易 overflow / underflow：

- fp16: 最大值 65504， $\exp(11) \approx 6 \times 10^4 \approx$ 溢出边界， $x > 11$ 就 saturate。
- bf16: 指数位比 fp16 多，但尾数只有 7 bit， $\sum \exp$ 的精度差。

后果如果不用 **fp32**：所有 p_i 变成 $\frac{1}{E}$ ，router 死掉——每个 token 均匀选专家，模型完全丧失路由能力。

Note. 实践中 gate 的 $\text{Linear}(H \rightarrow E)$ 输出本身可以是 bf16 (因为 E 很小, $W_g x$ 数值不易爆), 只有 softmax 内部升 fp32。Torch 的 `F.softmax(..., dtype=torch.float32)` 会做 upcast + softmax + 输出还是 fp32, 最后用户手动 `.to(bf16)`。

Router 的 kernel 视角

从性能角度看, router 一步做的事:

$$\ell = XW_g^T \quad (\text{GEMM})$$

$$p = \text{softmax}(\ell) \quad (\text{每行独立})$$

$$(w, i) = \text{topk}(p, K) \quad (\text{每行独立})$$

其中:

- GEMM: 形状 $(N, H) \times (H, E)$, N 大 ($\sim 10^4$) E 小 (~ 10) H 中等 ($\sim 10^3$)。这是一个 skinny GEMM——tile 数少、可能 memory-bound。
- Softmax: 形状 (N, E) , 每行 reduce, E 很小 (~ 10), 一个 warp 甚至几个 lane 就搞定。
- Topk: 形状 (N, E) , 每行选前 K 大。 E 小的时候直接 bitonic sort 或 $O(KE)$ 都行。

Naive 实现: 分别用 `cublasGemmEx` + `softmax kernel` + `topk kernel` 三次 launch。中间 logits (N, E) 和 probs (N, E) 各写读一次 HBM。

生产实现: 三步 fuse 成一个 kernel。中间态不落 HBM, 只在寄存器/shared memory:

```
fused_router_kernel:
  read X row (N-wise parallelism)
  compute W_g @ x → logits[E]          # in register (E small)
  softmax with fp32 accumulator         # in register
  bitonic topk-K                        # in register
  write expert_indices, expert_weights # 2 × (N, K) 输出
```

节省的 HBM 流量: 一次 (N, E) 读 + 一次 (N, E) 写。当 $N = 10^6, E = 8, \text{fp16}$, 约 $2 \times 10^6 \times 8 \times 2 = 32 \text{ MB}$ ——不多, 但每层每 step 都要做, 累加起来占端到端 forward 的 1-3%。

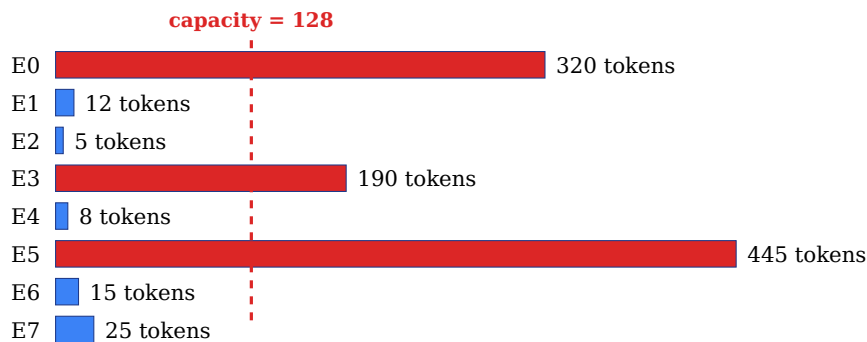
Key insight. Router 的 kernel 优化收益有限 (router 本身占端到端 5% 以内), 但所有生产框架都会 **fuse** 它——一是因为写起来相对容易 (问题结构简单), 二是因为 kernel launch 数下降对 CUDA graph capture 友好。

对应的开源实现: vLLM `fused_moe.py::fused_topk`, SGLang `fused_moe_router`, Megatron-LM `fused_router_kernel`。

Router 的可视化 debug

训练 MoE 有一个必备可视化: 每个 **batch** 里每个专家收到了多少 **token**。

例: $E = 8$ 时, 理想是每个专家收 $N \frac{K}{E}$ 个 token。实际训练早期可能是:



红色虚线是 capacity 上限（如设 $C = \text{ceil}(N \frac{K}{E} \times 1.25)$ ）。上图 E0/E3/E5 都会超出 **capacity** 触发丢 token，同时 E1/E2/E4/E6 严重欠载——这是典型的 expert collapse 早期表征，说明 aux loss 系数不够。

Note. 生产训练时每 100 step 打印 f_i (每专家 token 比例) 的直方图；如果 $\frac{\text{std}(f)}{\text{mean}(f)} > 0.5$ 持续超过 1000 step，通常需要：(a) 提升 aux loss 系数从 0.01 到 0.03；(b) 检查 router weight init (should be small, e.g. std=0.02)；(c) 考虑 warmup router 单独。

面试考点

面试考点. **Q1:** 为什么 gate 的 softmax 必须 fp32？

A: bf16/fp16 下 exp 容易饱和/下溢， E 个 logits 差距被压缩，router 输出退化为均匀分布，模型丧失路由能力。fp32 只在 softmax 内部升精度，最终 gate_probs 可 downcast 回来——计算量增加 < 1%，稳定性 huge。

面试考点. **Q2:** torch.topk 会返回相同专家 id 两次吗？

A: 不会。topk 返回的是 E 个 logit 的 top-K 索引，天然 unique。所以每个 token 的 expert_indices[n, :] 是 K 个不同 expert id，index_add_ 里同一个 token_ids 值最多出现一次（在单个 expert_idx 循环内）。

面试考点. **Q3:** Router 的反向传播中，未被选中的 $E - K$ 个 logits 怎么获得梯度？

A: 三条路径：(1) 通过 softmax 分母——变体 A 在 renorm 前，未选中 logits 仍参与 softmax 分母，有小梯度；(2) aux loss——直接对所有 p_i 惩罚（梯度流所有 logits）；(3) router z-loss——惩罚 $\text{logsumexp}(\text{logits})^2$ ，也流所有 logits。变体 B (Mixtral) 只有 (2)(3)。

面试考点. **Q4:** DeepSeek-V3 为什么弃用 aux loss？

A: aux loss 会让 router 收到“两种冲突梯度”——一份来自主 loss（学“正确路由”），一份来自 aux loss（学“均匀分布”）。冲突可能让 router 停在次优解。DeepSeek-V3 用可学习 bias 动态修正（溢出专家 bias↓ / 欠载 bias↑），只影响路由决策、不影响 gate 权重梯度，绕开冲突。见第 6 章。

面试考点. **Q5:** 如果 router 的 Linear(H, E) 权重初始化太大会怎样？

A: 初始 logits 方差大 $\rightarrow$ softmax 输出接近 one-hot $\rightarrow$ 早期路由固化到少数几个专家 $\rightarrow$ 其他专家永远收不到梯度 $\rightarrow$ 永久 collapse。Mixtral 官方 init 用 `std=0.02` 而不是 `std=1/sqrt(H)`，就是这个原因。

面试考点. **Q6**: 一个 token 的 K 个专家选择顺序有意义吗？

A: 数学上 —— 加权求和满足交换律，顺序无关。但 `k_ids` (topk 返回的位置索引) 有意义：它告诉你“此 token 把这个专家当作 top-1 / top-2 / ...”，用于从 `expert_weights (N, K)` 里取对应权重。顺序对调试和 **aux loss** 计算有用，但对最终输出没影响。

Dispatch 与 Combine 的三种实现范式

Router 只给出“每个 token 该去哪些专家”的决策；真正把数据搬过去、把结果搬回来的是 **dispatch** 和 **combine**。这一章是本书工程细节最密的一章——三种范式(scatter/gather、grouped GEMM、block-sparse) 之间的取舍决定了单机性能上限，也决定了分布式实现难度。

问题精确化

给定：

- `hidden_states`: (N, H)
- `expert_indices`: (N, K)，每行 K 个 unique expert id $\in [0, E)$
- `expert_weights`: (N, K)，每行 K 个 fp probability，和为 1
- E 个 `experts[e]`: (H,) $\rightarrow$ (H,) (本质是 FFN)

目标：算出

$$y_n = \sum_{k=0}^{K-1} \tilde{w}_{n,k} \cdot \text{FFN}_{i_{n,k}}(x_n)$$

产出 `output`: (N, H)。

核心难点：每个 token 走 K 个专家、每个专家的 batch size M_e 是运行时决定的变量。这不能直接翻译成静态形状的 GEMM。

范式 A：Scatter / Gather (本书教学实现)

最直观的实现——外层循环遍历专家，每次找出所有路由到它的 token，跑一次小 FFN：

```
out = torch.zeros_like(hidden_states)          # (N, H)

for e in range(E):
    # 1. 找出命中此专家的 (token, k) 位置
    token_ids, k_ids = torch.where(expert_indices == e)
    if token_ids.numel() == 0:
        continue

    # 2. gather: 从 hidden 里取出这些 token
    x_e = hidden_states[token_ids]              # (M_e, H)

    # 3. compute: 单专家 FFN
    y_e = experts[e](x_e)                      # (M_e, H)

    # 4. weight: 乘以对应 gate 权重
    w_e = expert_weights[token_ids, k_ids].unsqueeze(-1) # (M_e, 1)

    # 5. scatter-add: 累加回 output 对应位置
    out.index_add_(0, token_ids, y_e * w_e)
```

Shape 流：



图解一步 **dispatch**

以第 3 章 §“用一个具体例子拉一遍 tensor” 的例子 ($N = 4, E = 4, K = 2$) ,我们看 `expert_idx = 1` 的循环：

```

expert_indices = [[1, 3],      # t0 top-1=E1, top-2=E3
                  [2, 0],      # t1 top-1=E2, top-2=E0
                  [3, 1],      # t2 top-1=E3, top-2=E1
                  [0, 1]]      # t3 top-1=E0, top-2=E1

(expert_indices == 1) =
    [[T, F],
     [F, F],
     [F, T],
     [F, T]]

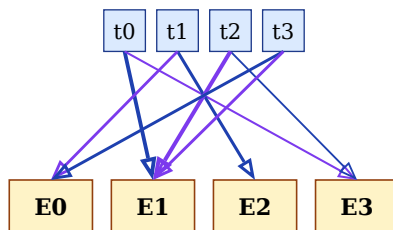
where(...) →
    token_ids = [0, 2, 3]      # 3 个 token 命中 E1
    k_ids      = [0, 1, 1]      # t0: E1 是 top-1; t2/t3: E1 是 top-2

gather →      hidden_states[[0,2,3]] : (3, H)
FFN →         experts[1](·)           : (3, H)
weight →      expert_weights[[0,2,3], [0,1,1]] : (3,)
               = [0.706, 0.250, 0.500]

scatter-add → out[[0,2,3]] += y * w
  
```

用 dispatch 图看更清楚：

Dispatch: $N=4$ tokens $\rightarrow E=4$ experts, $K=2$ (箭头粗细 = gate 权重)



复杂度与瓶颈

单次 forward 触发的 kernel launch :

- Router: 3 次 (Linear, softmax, topk) — 可 fuse 到 1 次
- Dispatch 循环: E 次 gather + E 次 FFN (2 GEMM + 1 activation) + E 次 `index_add_`
- 合计 $O(6E + 1)$ 次 kernel launch

对 $E = 8$: 约 50 次 launch $\times 5 \mu s = 250 \mu s$ 的固定开销——在小 batch 下比 FFN 本身还贵。

核心问题 : 每个专家的 GEMM 是 $(M_e, H) \times (H, I)$, M_e 可能很小 (数十)。TensorCore 需要 M 至少 128 才能吃满, $M_e = 30$ 的 GEMM 效率约 15%。 E 个这样的小 GEMM 加起来, SM 利用率极低。

Warning. 范式 A 只适合教学和调研原型。在 $E > 4$ 或 batch 大时, 一定要转到范式 B。本书 `test_moe.py` 用范式 A, 是为了让读者能读懂 —— 生产上没有一个 **MoE** 框架用这个实现。

一个隐藏的正确性坑

`torch.index_add_` 在 GPU 上默认非确定性——多个 index 相同的写入可能顺序不同。但在范式 A 中, 单个 **expert** 循环内 `token_ids` 保证 unique (因为 `expert_indices[n, :]` 每行 K 个专家 id 互不相同 $\rightarrow$ 一个 token 在同一个 `expert_idx == e` 循环内最多出现一次)。所以本书 demo 里 `index_add_` 退化为普通 scatter, 是确定的。

但是 : 跨 `expert_idx` 循环, 同一个 token 被 `index_add_` K 次——这是设计需要的 (top-k 加权求和)。这里 add 的顺序可能受 kernel launch 调度影响 :

```
out[t0] = w_(t0,0) * FFN_{i_(t0,0)}(x_(t0)) + w_(t0,1) * FFN_{i_(t0,1)}(x_(t0))
```

由于浮点加法非结合, 两次 **add** 的顺序影响低位 **bit**。要严格 reproducible 训练 :

```
torch.use_deterministic_algorithms(True)
```

代价约 5-15% 慢。

范式 B : Permute + Grouped GEMM (生产级)

范式 A 的根本痛点 : E 次独立 **GEMM**。如果能把 E 次 $(M_e, H) \times (H, I)$ 打包成一次 grouped GEMM, 就能大幅提升效率。

核心思路 : 把 **tokens** 按专家排序

给每个 (token, k) pair 分配一个 slot (共 NK 个 slot), 把 slot 按 `expert_indices` 排序打包 :

输入:

```
expert_indices = [[1, 3],
```

```

        [2, 0],
        [3, 1],
        [0, 1]]          # shape (N=4, K=2)

flatten & 排序:
    slots (before) = [(t0,k0)→E1, (t0,k1)→E3,
                      (t1,k0)→E2, (t1,k1)→E0,
                      (t2,k0)→E3, (t2,k1)→E1,
                      (t3,k0)→E0, (t3,k1)→E1]

按 expert 排序 (稳定排序):
    permute_order = [3,      6,              # E0: (t1,k1), (t3,k0)
                     0,      5,      7,      # E1: (t0,k0), (t2,k1), (t3,k1)
                     2,              # E2: (t1,k0)
                     1,      4]          # E3: (t0,k1), (t2,k0)

    group_sizes = [2, 3, 1, 2]          # 每个专家的 token 数

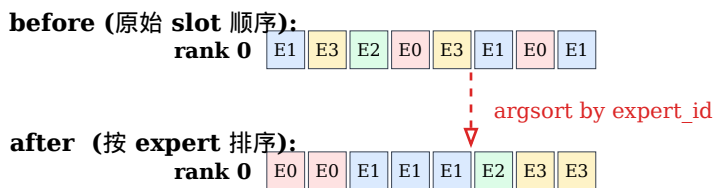
    total = 8 = N*K

```

Permute 后的 memory layout

用图看 (同一 GPU 上的内存布局, 无跨卡通信):

Permute (single-GPU): 8 个 (token, k) slot 按目标 expert 分组打包



打包后的 tensor `packed_input: (N*K, H)` 里, 第 0-1 行是 E0 的 token, 第 2-4 行是 E1 的 token, 以此类推。

Note. 这里不是分布式 all-to-all——只是本卡的内存 gather。真正的 all-to-all (跨 GPU 的 dispatch) 出现在第 8 章, 长得像但语义完全不同。

Grouped GEMM

Grouped GEMM 一次调用完成 E 个不同 shape 的 GEMM:

```

# 概念签名 (PyTorch 2.4+ 提供 torch._grouped_mm)
packed_out = torch._grouped_mm(
    packed_input,          # (N*K, H)
    W_stacked,             # (E, H, I) - 所有专家权重堆叠
    group_sizes,           # (E,) int - 每个专家的 token 数
)
# packed_out: (N*K, I)

```

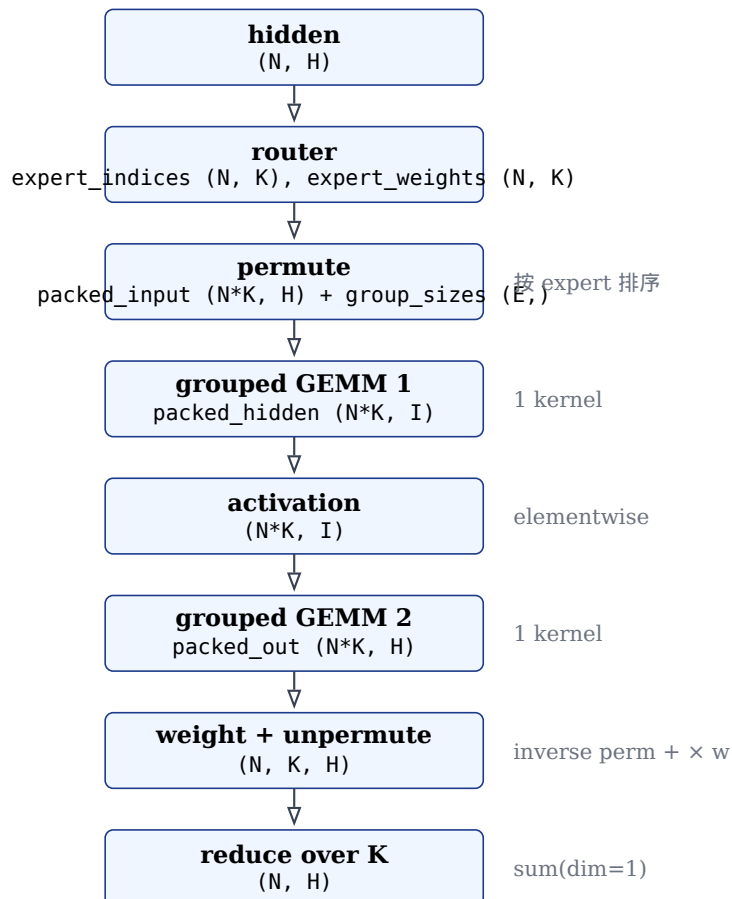
Kernel 内部: CUTLASS 的 `GroupedGemm` template 根据 `group_sizes` 在 CTA 层生成 E 个 sub-problem tile, 动态分配 SM 资源。相当于一次 launch 完成 E 个 GEMM。

收益：kernel launch 从 $E \rightarrow 1$ ；tile scheduler 全局最优（能把小 group 和大 group 混合调度）；SM 利用率从 15% $\rightarrow$ 60-80%。

完整范式 B 流程

1. router $\rightarrow$ expert_indices (N,K), expert_weights (N,K)
2. permute:
 - 展平 (N,K) $\rightarrow$ (N*K,) 得 flat_expert_ids
 - argsort 得 permute_idx
 - packed_input = hidden_states.repeat_interleave(K)[permute_idx]
 - 或用 fused kernel 直接 gather 到 packed
3. grouped_gemm_1: packed_input @ W_up $\rightarrow$ packed_hidden (N*K, I)
4. activation: (SwiGLU / ReLU / GELU) elementwise
5. grouped_gemm_2: packed_hidden @ W_down $\rightarrow$ packed_out (N*K, H)
6. weight & unpermute:
 - packed_out *= expert_weights[permute_idx] (broadcast)
 - out[permute_idx.argsort()] = packed_out (inverse permute)
7. sum over K:
 - out.view(N, K, H).sum(dim=1) $\rightarrow$ (N, H)

Shape 全链路：



范式 B 的代价

不是免费午餐：

(1) Permute / unpermute 是新增开销

Permute 本身要一次全内存 gather : NKH elements 的读写。当 $N = 10^5, K = 2, H = 4096, \text{bf16}$, 约 3 GB traffic。fusion 到 grouped-gemm 的 prologue 里可以摊平大半 , 但独立版本会显著。

(2) $M_e = 0$ 的空专家要正确处理

grouped GEMM kernel 必须允许 `group_sizes[e] = 0` ——传入的 offset 不变 , 跳过。CUTLASS GroupedGemm 支持 , 但早期版本有 bug。

(3) capacity 与 drop token

有些实现给每个 expert 设 capacity $C = \text{ceil}(N \frac{K}{E} \times c_{\text{factor}})$, 超出的 token 直接丢弃。丢弃 token 通过 residual 分支跳过 FFN。范式 B 里 drop 天然对齐 : packed 到 group e 的 slot 超过 C 就截断。

- 优点 : packed shape 完全静态 ((E, C, H))、GEMM tuning 更好。
- 缺点 : 训练早期 collapse 时大量 drop $\rightarrow$ 损失 & 信息丢失。

(4) 需要自定义 kernel

不是 vanilla PyTorch 一行搞定——需要 `torch._grouped_mm` 或框架自带的 `MoELayer`。这是 Tutel / Megatron-MoE / DeepSpeed-MoE 的核心 IP。

Key insight. 范式 B 的性能取决于 **permute+unpermute** 是否 **fuse** 进 **grouped GEMM**。裸调 `torch._grouped_mm` 只 fuse 了 GEMM , permute 仍是独立 kernel ; 生产实现 (Tutel `fast_dispatch`、Megatron `moe_permute`) 会把 permute 融进 GEMM 的 prologue , 把 3 次 $(N \times K, H)$ 的 HBM 往返压到 1 次。

范式 C : Block-Sparse GEMM (Megablocks)

Megablocks (Gale et al. 2022) 的洞察 : 所有 **expert** 权重可以拼成一个大 **block-diagonal** 矩阵 , 然后用一次块稀疏 GEMM 完成计算。

Block-sparse 视角

先按范式 B 那样把 tokens 排好——`packed_input`: $(N \times K, H)$ 。每一行对应一个 (token, k) slot , 属于某个 expert e 。Megablocks 的洞察是 : 把所有 **expert** 权重视为一个大的 **block-diagonal** 矩阵 , 然后一次 block-sparse GEMM 就完成全部计算 :

$$\underbrace{\mathbf{Y}}_{(NK, EI)} = \underbrace{\hat{\mathbf{X}}}_{(NK, EH)} \times \underbrace{\mathbf{W}_{\text{big}}}_{(EH, EI)}$$

其中 $\mathbf{W}_{\text{big}}$ 是把 E 个 $\mathbf{W}_{\text{up}} \in \mathbb{R}^{H \times I}$ 沿对角线拼起来 , 只有对角上的 E 个 block 非零 :

$$\mathbf{W}_{\text{big}} = \begin{bmatrix} \mathbf{W}_{\text{up}}^{\{E0\}} & & & \\ & \mathbf{W}_{\text{up}}^{\{E1\}} & & \\ & & \dots & \\ & & & \mathbf{W}_{\text{up}}^{\{E_{E-1}\}} \end{bmatrix} \quad (E-1)/E \text{ 稀疏}$$

$\hat{\mathbf{X}}$ 是 `packed_input` 的“扩展” : 第 i 行只在 slot 所属 expert 的列区间放原来的 hidden vector、其余全零。这样 $\hat{\mathbf{X}} \times \mathbf{W}_{\text{big}}$ 每行只在对应 expert 的输出区间产生非零结果 , 行为等价于范式 B 的 grouped GEMM——但 kernel 上真正跑的是 block-sparse SpMM (CUTLASS block-sparse、Sputnik、Megablocks 自带的 `stk`)。

生产实现里 $\hat{X}$ 不会真的 materialize——它是逻辑视图，SpMM kernel 直接从 packed_input + slot→expert 的映射 gather。整个过程等价于对 packed layout 加一次 block-sparse 元数据，不需要 padding、不需要 drop。

关键优势

(1) 完全无 padding、无 drop

不需要 capacity， M_e 可以是任意值——block-sparse GEMM 天然处理变长。

(2) 与 dense GEMM 性能可比

只要 block 大小 ≥ 128 ，CUTLASS 的 block-sparse kernel 效率接近 dense。Megablocks 论文报告 in-block density = 1、跨 block sparse pattern，dense-equivalent throughput 达到 90%+。

(3) 训练早期 collapse 也不 drop

对训练动力学友好——router 可以慢慢学，不会因为 drop 导致信号丢失。

关键代价

(1) Kernel 复杂

需要 block-sparse metadata (block row/col indices)，permute 逻辑更复杂。这是 Megablocks 的核心 IP。

(2) 分布式集成难度高

Block-sparse metadata 在 all-to-all 之后需要重新计算——这也是为什么 Megablocks 早期只在单机上。后来 (2024) 有 distributed megablocks 集成到 Databricks DBRX。

三种范式对比

维度	A. scatter	B. grouped GEMM	C. block-sparse	说明
Kernel launch 数	$O(E)$	$O(1)$	$O(1)$	B/C 大幅减少
SM 利用率	low (< 30%)	high (60% +)	very high (80% +)	取决于 tile 大小
支持 drop token	手动实现	是 (常用)	不需要	C 是最大优势
Permute overhead	无 (循环内 gather)	显式 permute	显式 permute	B/C 需要 permute fuse
空专家 $M_e = 0$	continue	需支持	自然处理	B 早期 kernel 有 bug
代码复杂度	50 行	500 行 + kernel	2000 行 + kernel	C 最难
分布式集成	不适用	成熟 (Megatron, Tutel)	新 (DBRX, 2024)	B 是主流

选择建议

不同场景下选哪个范式，一个 rough guide：

- 学习/调研：范式 A。写起来快、跑起来对、能读懂。
- 单机 $E \leq 8$ ：范式 A 或 B 均可；B 收益 $2-3\times$ ，但对 batch 小的场景 (< 512 tokens) 收益递减。
- 单机 $E > 8$ 或大 **batch**：范式 B 是标准。
- 需要 **drop-free** 训练：范式 C。
- 多机 **EP**：范式 B + all-to-all；范式 C 集成难度高，选型看框架支持。

Note. 个人建议：即使目标是生产系统，先写一遍范式 A 的正确版、跑通 unit test，然后把 FFN 部分替换成范式 B。范式 A 的输出是“reference”，用来对拍 B/C 的正确性——特别是 drop token、capacity、边界条件的处理。

面试考点

面试考点. **Q1**: 范式 A (scatter) 主要的性能问题是什么？

A: (1) E 次 kernel launch overhead ;(2) 每个专家 GEMM 的 M 维小 (M_e 数十), TensorCore 无法吃满 ;(3) M_e 随 batch 抖动, 无法预测性能 ;(4) `index_add_` 的 memory access pattern 不连续。总体表现：SM 利用率 $< 30\%$ 。

面试考点. **Q2**: Grouped GEMM 为什么比 for-loop 快？

A: 一次 kernel 内部做全局 tile 调度：把 E 个不同大小的 sub-problem 打散成 tile，动态负载均衡；SM 一直有 work，不需要等 kernel launch。等价于把 E 次串行 launch 换成一次 launch + 内部并行。

面试考点. **Q3**: `permute_idx` 和 `permute_idx.argsort()` 的关系？

A: 逆排列。若 `permute_idx = [3, 6, 0, 5, 7, 2, 1, 4]`，则 `argsort(permute_idx) = [2, 6, 5, 0, 7, 3, 1, 4]`——这是 unpermute 用的 index。生产实现里 permute + unpermute 的 index 只算一次，缓存到 workspace 复用。

面试考点. **Q4**: Capacity factor 应该设多少？

A: 训练 $c_{\text{factor}} = 1.25$ 是 GShard/Switch 经验值；推理常用 $c_{\text{factor}} \geq 2$ 或直接 no-drop (Megablocks 路径)。理论下界：如果 router 输出接近均匀， $M_e \approx N \frac{K}{E}$ ， $c_{\text{factor}} = 1$ 就够；训练早期 collapse 时 M_e 可以是 $10N \frac{K}{E}$ ， $c_{\text{factor}} = 2$ 都不够——所以 drop 是必然事件，只是频率问题。

面试考点. **Q5**: Megablocks vs grouped GEMM 的核心差异？

A: Grouped GEMM 需要 permute 到 packed layout，capacity + drop 才能保证 shape 静态；Megablocks 用 block-sparse GEMM 天然处理变长，不 **drop**，代价是 kernel 复杂度和分布式集成难。Megablocks 训练动力学更好（无 drop），但工程负担重。

面试考点. **Q6**: 如果不用 `torch.where`，还能怎么实现 range B 的 permute？

A: (1) `expert_ids.flatten().argsort()` 直接给 permute index ; (2) counting sort 因为 E 小 (`bincount + cumsum` 得 group offsets) ; (3) fused kernel 直接从 `expert_indices` 生成 sorted flat index + offsets 一次搞定。生产实现用 (3) , 因为 `argsort` 有隐式排序开销。

逐行 Code Walkthrough (test_moe.py)

这一章配合本仓库 `python/pytorch/test_moe.py` 逐段拆解——把前两章的抽象概念落到具体的 PyTorch 代码上。所有 tensor shape 都带具体数字，方便对着代码 debug。

跑法：

```
python python/pytorch/test_moe.py
# ok
```

代码 100 行左右，运行样例 $B = 2, S = 4, H = 8, I = 16, E = 4, K = 2$ 。

全文件结构

```
import torch
import torch.nn.functional as F
from jaxtyping import Float
from torch import Tensor, nn

class Expert(nn.Module): ...      # 单个专家 FFN
class MoE(nn.Module): ...        # MoE 层

def test_moe_forward_shape(): ... # smoke test 1
def test_moe_backward(): ...      # smoke test 2

if __name__ == "__main__":
    test_moe_forward_shape()
    test_moe_backward()
    print("ok")
```

三部分：Expert (单专家 FFN)、MoE 主体、两个 smoke test。

Part 1: Expert 定义

```
class Expert(nn.Module):
    """一个普通的 2 层 FFN，用作单个专家。
    结构：Linear -> ReLU -> Linear，无 bias（与 Mixtral 对齐）
    """
    def __init__(self, hidden_size: int, intermediate_size: int) -> None:
        super().__init__()
        self.fc1 = nn.Linear(hidden_size, intermediate_size, bias=False)
        self.fc2 = nn.Linear(intermediate_size, hidden_size, bias=False)

    def forward(
        self, hidden_states: Float[Tensor, "N H"]
    ) -> Float[Tensor, "N H"]:
        return self.fc2(F.relu(self.fc1(hidden_states)))
```

说明：一个专家就是一个 $(H \rightarrow I \rightarrow H)$ 的普通 FFN。

样例参数量 : $H = 8, I = 16$,每个专家 $H \times I + I \times H = 256$ params , $E = 4$ 时总 expert 参数 = 1024。

生产模型的差异 : Mixtral/Llama 用 SwiGLU :

```
class MixtralExpert(nn.Module):
    def __init__(self, H, I):
        self.w_gate = nn.Linear(H, I, bias=False)
        self.w_up = nn.Linear(H, I, bias=False)
        self.w_down = nn.Linear(I, H, bias=False)
    def forward(self, x):
        return self.w_down(F.silu(self.w_gate(x)) * self.w_up(x))
```

SwiGLU 比标准 MLP 多 50% 参数量 (3 个 Linear 而不是 2 个), 但训练/推理都实测更好。教学实现里为了简洁, 本书用 ReLU 版。

Part 2: MoE 主体——init

```
class MoE(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        intermediate_size: int,
        num_experts: int,
        top_k: int,
    ) -> None:
        super().__init__()
        assert 1 <= top_k <= num_experts

        self.hidden_size = hidden_size
        self.num_experts = num_experts
        self.top_k = top_k

        self.gate = nn.Linear(hidden_size, num_experts, bias=False)
        self.experts = nn.ModuleList(
            [Expert(hidden_size, intermediate_size)
             for _ in range(num_experts)]
        )
```

关键点 :

- `self.gate` : router 的唯一可学习参数 , $(H \rightarrow E)$ 。
- `self.experts` : 用 `nn.ModuleList` 是为了让 `.parameters()` / `.to(device)` 递归看到子模块。用普通 `list` 会导致参数不注册。

样例配置 $H = 8, I = 16, E = 4, K = 2$:

- Router: $8 \times 4 = 32$ params
- Experts: $4 \times 256 = 1024$ params
- Total: 1056 params

真实 Mixtral-8×7B :

- Router: $4096 \times 8 = 32,768$ params $\times 32$ layers = 1M params (可忽略)
- Experts per layer: $8 \times (4096 \times 14336 \times 3) \approx 1.4\text{B}$; 32 layers = 45B (主要参数)

- Attention + LN + embed: 剩下的 2B

Part 3: MoE forward——展开和 router

```
def forward(
    self, hidden_states: Float[Tensor, "B S H"]
) -> tuple[Float[Tensor, "B S H"], Float[Tensor, "BS E"]]:
    batch_size, seq_len, hidden_size = hidden_states.shape
    dtype = hidden_states.dtype

    # (B, S, H) -> (N, H), N = B * S
    hidden_states = hidden_states.view(-1, hidden_size)
    num_tokens = hidden_states.shape[0]
```

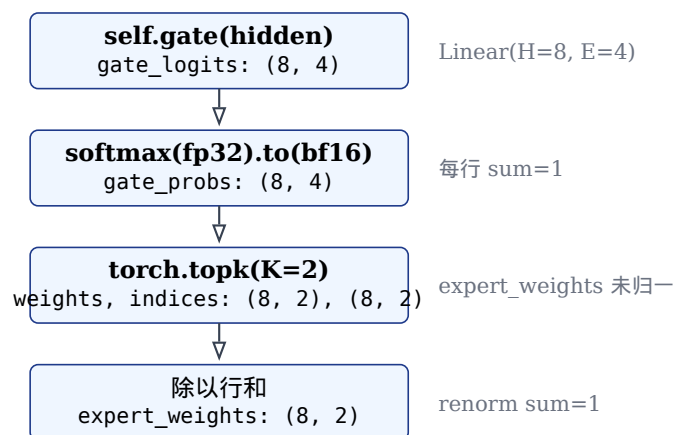
样例： $(2, 4, 8) \rightarrow (8, 8)$ 。这一步等价于把 **batch** 和 **seq** 视为独立 **token** 序列——因为 MoE 的路由是逐 token 独立的，两个维度合并不影响语义。

易错点：调用 `.view(-1, H)` 要求 tensor 是 contiguous。如果上游有 transpose，需要 `.reshape(-1, H)` 或先 `.contiguous()`。生产实现里 hidden_states 从 attention 出来通常是 contiguous 的，不用担心。

```
# ----- 1. Router -----
gate_logits = self.gate(hidden_states) # (N, E)
gate_probs = F.softmax(gate_logits, dim=-1, dtype=torch.float32).to(dtype)

expert_weights, expert_indices = torch.topk(
    gate_probs, k=self.top_k, dim=-1
)
expert_weights = expert_weights / expert_weights.sum(dim=-1, keepdim=True)
expert_weights = expert_weights.to(dtype)
```

Shape 追踪 ($N = 8, E = 4, K = 2$) :



三个 subtle 点：

(1) dtype=torch.float32 参数

`F.softmax(..., dtype=torch.float32)` 会先 upcast `gate_logits` 到 fp32、再算 softmax、返回 fp32。然后 `.to(dtype)` 转回原 dtype (通常 bf16)。第 3 章“为什么 softmax 要用 fp32”解释过为什么必要。

(2) torch.topk 的返回顺序

返回的 K 个专家是按 **gate_probs** 值降序排的。所以 `expert_indices[n, 0]` 是 top-1, `expert_indices[n, 1]` 是 top-2。后续 dispatch 里 `k_ids = 0` 对应 top-1、`k_ids = 1` 对应 top-2。

(3) renormalize 前后的语义

在第 3 章解释过：renorm 前保留“router 置信度”信息，renorm 后是 top-K 内的相对权重。本书 demo 用 renorm，Mixtral 用 `softmax(topk(logits))`，两者数学等价性我们已经推过。

Part 4: MoE forward——dispatch 循环

这是全文件最关键的 20 行：

```
# ----- 2. Dispatch -----
out = torch.zeros_like(hidden_states) # (N, H)

for expert_idx in range(self.num_experts):
    # 找到所有把 expert_idx 选为 top-k 之一的 (token, k) 位置
    token_ids, k_ids = torch.where(expert_indices == expert_idx)
    if token_ids.numel() == 0:
        continue

    expert_input = hidden_states[token_ids] # (M, H)
    expert_output = self.experts[expert_idx](expert_input) # (M, H)

    weights = expert_weights[token_ids, k_ids].unsqueeze(-1) # (M, 1)
    out.index_add_(0, token_ids, (expert_output * weights).to(dtype))
```

用 `expert_idx = 1` 一步拆解 (延续第 4 章的例子)：

Line 1: `torch.where(expert_indices == 1)`

- 输入: `expert_indices` shape (8,2)
- 中间: `(expert_indices == 1)` 是 (8,2) 的 bool tensor
- 输出: 两个 $(M,)$ 的 int64 tensor, M 是命中的 (token, k) 位置数

对样例数据 (随机, 假设有 5 个 hit)：

```
token_ids = tensor([0, 2, 3, 5, 6])
k_ids     = tensor([0, 1, 0, 0, 1])
```

Line 2: `if token_ids.numel() == 0: continue`

处理 $M_e = 0$ 的边界情况——某专家这个 batch 一个 token 都没收到。生产实现要小心：如果这里不 continue, `hidden_states[[]]` 是 empty tensor, `Linear` 对 empty 输入是合法的, 但 `.to(cuda)` 上有时会出错。范式 A 就地 continue 最安全。

Line 3: `hidden_states[token_ids]`

Fancy indexing, $(N, H) \rightarrow (M, H)$, 这是一次 gather kernel。生成的 tensor 是新分配的、connected 到 autograd 图。

Line 4: `self.experts[expert_idx](expert_input)`

单专家 FFN, $(M, H) \rightarrow (M, H)$ 。内部两次 Linear + 一次 ReLU, 共 3 个 kernel launch (本书教学实现, 未 fuse)。

Line 5: `expert_weights[token_ids, k_ids].unsqueeze(-1)`

这是最容易看错的一句。

- `expert_weights` shape $(N, K) = (8, 2)$
- `token_ids` shape $(M,) = (5,)$
- `k_ids` shape $(M,) = (5,)$

`expert_weights[token_ids, k_ids]` 是 **2D fancy indexing**——每个 (t, k) pair 取一个元素, 返回 shape $(M,)$:

```
weights[i] = expert_weights[token_ids[i], k_ids[i]]

= expert_weights[[0,2,3,5,6], [0,1,0,0,1]] # 逐元素配对
= [expert_weights[0, 0], # t0 把 E1 选为 top-1 的权重
    expert_weights[2, 1], # t2 把 E1 选为 top-2 的权重
    expert_weights[3, 0], # ...
    expert_weights[5, 0],
    expert_weights[6, 1]]

→ shape (5,)
```

然后 `.unsqueeze(-1)` 变 $(5, 1)$, 方便和 $(5, H)$ broadcast 相乘。

Key insight. `expert_weights[token_ids, k_ids]` 用了 numpy/pytorch 的同轴 **broadcasting fancy indexing**——两个 index tensor shape 必须一致, 结果 shape 也一致。这不是 `expert_weights[token_ids][:, k_ids]` (后者是先取行, 再对每行的所有列取 `k_ids`, 结果是 (M, M))。生产代码 review 常见 bug。

Line 6: `out.index_add_(0, token_ids, ...)`

Scatter-add :

```
for i in range(M):
    out[token_ids[i]] += (expert_output[i] * weights[i]).to(dtype)
```

0 是 dim (沿第 0 维即 token 维累加)。`token_ids` 在单次 `expert_idx` 循环内 unique (第 4 章“一个隐藏的正确性坑”解释过), 所以这里等价于 gather-write, 没有 race。但跨 `expert_idx` 循环, `out[t]` 会被 K 次累加——这正是 top-K 加权求和。

Line 7: `.to(dtype)`

确保输出 dtype 一致。中间 `expert_output * weights` 可能因为 `weights` 是 fp32 (renorm 阶段) 而被 upcast——这里明确 downcast 回 bf16。

Part 5: 收尾

```
out = out.view(batch_size, seq_len, hidden_size)
return out, gate_logits
```

Reshape 回 (B, S, H) 。返回 `gate_logits` 是为了外部可以用它算 aux loss (第 6 章)。生产实现通常返回 dict, 包含 `router_probs`, `expert_indices`, `f_i` (per-expert token count) 等 debug 信息。

两个 smoke test

```
def test_moe_forward_shape():
    torch.manual_seed(0)
    B, S, H, I, E, K = 2, 4, 8, 16, 4, 2
    moe = MoE(hidden_size=H, intermediate_size=I, num_experts=E, top_k=K)
    x = torch.randn(B, S, H)

    y, gate_logits = moe(x)

    assert y.shape == (B, S, H)
    assert gate_logits.shape == (B * S, E)
    assert torch.isfinite(y).all()
```

用 `torch.manual_seed(0)` 固定 router 初始化和输入, 保证 test 可重复。检查 (1) 输出 shape 保持; (2) `gate_logits` 是 flatten 后的 (N, E) ; (3) 输出全有限 (没有 NaN/Inf)。

```
def test_moe_backward():
    torch.manual_seed(0)
    B, S, H, I, E, K = 4, 16, 8, 16, 4, 2 # 更大 batch, 每个 expert 概率被 hit
    moe = MoE(hidden_size=H, intermediate_size=I, num_experts=E, top_k=K)
    x = torch.randn(B, S, H, requires_grad=True)

    y, _ = moe(x)
    y.sum().backward()

    assert x.grad is not None and torch.isfinite(x.grad).all()
    assert moe.gate.weight.grad is not None
    got_grad = [e.fc1.weight.grad is not None for e in moe.experts]
    assert any(got_grad)
```

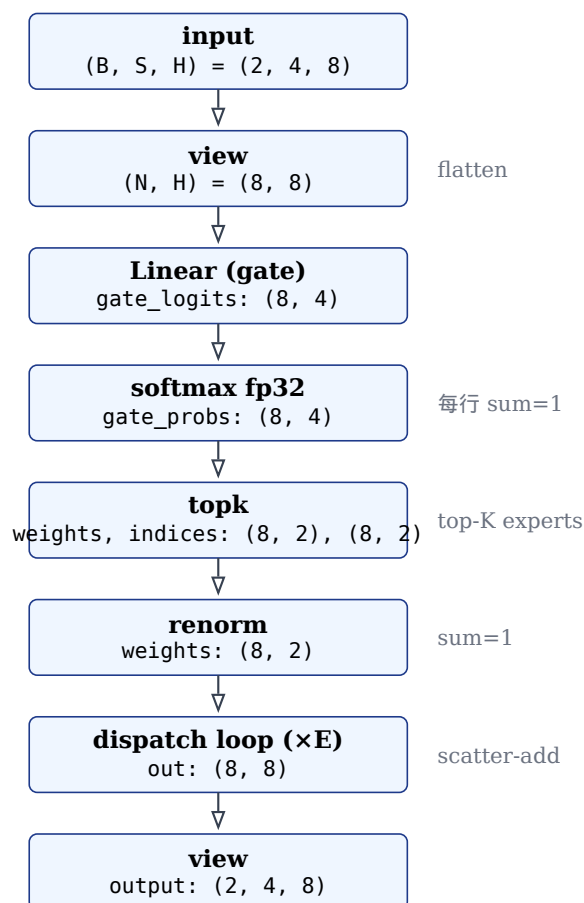
关键设计：

(1) 更大的 batch : $BS = 64$, 避免每个专家概率 = 0。 $K = 2, E = 4$, 理想每专家 $M_e = 32$ ——大概率所有 expert 都被至少一个 token 选中。

(2) `assert any(got_grad)` : 只要求至少一个 expert 有梯度——第 3 章讲过, topk 反向只流 K 个专家, 某个专家在 batch 里未被选中就没梯度。这个 test 不能 `assert all`。

完整 tensor 生命周期图

把整个 forward 汇成一张图：



常见改错练习

写完 MoE 你会想改的几个方向，各自的坑：

(1) 想加 **SwiGLU expert**

改 `Expert.forward`：

```
def forward(self, x):
    return self.w_down(F.silu(self.w_gate(x)) * self.w_up(x))
```

`init` 里加 `w_gate`, `w_up`, `w_down` 三个 Linear。注意参数量增加 **50%**。

(2) 想改成 **Mixtral 风格 router**

替换：

```
# 原本
gate_probs = F.softmax(gate_logits, dim=-1, dtype=torch.float32).to(dtype)
expert_weights, expert_indices = torch.topk(gate_probs, k=K, dim=-1)
expert_weights = expert_weights / expert_weights.sum(dim=-1, keepdim=True)

# Mixtral 风格
top_logits, expert_indices = torch.topk(gate_logits, k=K, dim=-1)
expert_weights = F.softmax(top_logits, dim=-1, dtype=torch.float32).to(dtype)
```

结果几乎等价，但反向传播动力学不同（第 3 章“Backward”一节）。

(3) 想加 **aux loss**

返回 `gate_probs` 而不是 `gate_logits` , 然后在训练脚本里 :

```
def aux_loss(gate_probs, expert_indices, num_experts):
    N, E = gate_probs.shape
    # f_i: 每个 expert 被 top-1 选中的比例
    f = torch.zeros(E, device=gate_probs.device)
    top1 = expert_indices[:, 0]
    f.scatter_add_(0, top1, torch.ones_like(top1, dtype=torch.float))
    f = f / N
    # P_i: gate_probs 的平均
    P = gate_probs.mean(dim=0)
    return E * (f * P).sum()
```

详情见第 6 章。

(4) 想加 **capacity + drop**

需要在 dispatch 前 truncate :

```
capacity = int(num_tokens * K / E * capacity_factor)
# 每个 expert 保留前 capacity 个 (按位置或按 gate prob)
```

范式 A 里做 drop 反直觉——先在 `expert_indices` 里 mark drop 位置 (e.g. 设为 `-1`) , `torch.where` 前跳过。范式 B 里 drop 天然 (超 capacity 就截断 packed)。

面试考点

面试考点. **Q1:** 为什么用 `view(-1, H)` 而不是 `reshape(-1, H)` ?

A: `view` 要求 contiguous 但不做拷贝——快; `reshape` 允许非 contiguous , 会隐式拷贝。上游 attention 输出通常 contiguous , 用 `view` 更快。如果无法确定, 用 `reshape` 或先 `.contiguous().view(-1, H)` 。

面试考点. **Q2:** `expert_weights[token_ids, k_ids]` 和 `expert_weights[token_ids][:, k_ids]` 有什么区别?

A: 前者是 **2D fancy indexing** , 两个 index tensor 同步 broadcast , 输出 shape $(M,)$; 后者是先行取再取列, 输出 shape (M, M) 。工程 bug 高发点。

面试考点. **Q3:** `index_add_` 里如果 `token_ids` 有重复, PyTorch 保证顺序吗?

A: GPU 上默认不保证——多个原子加的顺序取决于硬件调度。要 reproducible 训练用 `torch.use_deterministic_algorithms(True)` , 代价约 5-15% 慢。本书 demo 里单次 **expert** 循环内 `token_ids` 保证 unique , 所以不涉及; 跨循环的 K 次累加受此影响。

面试考点. **Q4:** 为什么 `test_moe_backward` 只 assert any 而不是 all ?

A: topk 反向只把梯度流给被选中的 K 个专家；小 batch 下某个专家可能一个 **token** 都没被选中，参数梯度就是 None。all 会假阳性 fail；any 只验证 backward 至少走通一条路径。生产 unit test 里通常用更大的 N 或 mock router 强制均匀。

面试考点. Q5: 如果 expert_indices 里同一行有两个相同的 expert id (比如 [3, 3]), 会怎样？

A: torch.topk 保证返回值不重复——它是取索引，不同索引对应不同位置。要出现重复必须 gate_probs 完全相等 (tie)——tie 时 torch 的 tie-breaking 是 stable (按位置序返回)，结果 index 不同，仍然 **unique**。因此 test_moe.py 假设成立。

面试考点. Q6: 有没有办法让 dispatch 循环用 torch.compile 加速？

A: 部分能——router 部分 (Linear + softmax + topk) 编译收益明显。但循环内 torch.where + 动态形状 gather 会导致 graph break。要真正吃到编译红利，需要换成范式 **B** (静态 packed shape)。所以生产实现的正确姿势是先换范式，再上编译。

Load Balancing :Aux Loss、Z-Loss、Bias Tuning

MoE 训练面临一个根本冲突：**Router** 要学最优路由，但如果放任不管，router 学到的最优路由往往是“永远选那几个专家”——其他专家永远收不到梯度、变成 dead weights。

这一章讲三种（历史演进的）解决方案：GShard 时代的 aux loss、ST-MoE 加的 z-loss、DeepSeek-V3 的 aux-loss-free bias tuning。

问题重述：Expert Collapse

第 3 章“Backward”一节讲过 topk 反向传播只流 K 个专家。这意味着：

如果 **router** 从未选过 **expert** e ，那么 ℓ_e （对应的 gate logit）从未有过直接梯度信号。它只能通过 softmax 的分母被间接影响——但这个梯度非常弱，尤其在 renorm 或 sigmoid gating 下几乎为零。

具体化：假设初始化时 router 有轻微偏好 expert 0，第一步后 expert 0 收到的 token 稍多、参数更新、下 step 更受青睐——正反馈循环 → expert 0 独占 → 其他 expert 死掉。

现象：训练时 $\frac{\text{std}(f_e)}{\text{mean}(f_e)}$ （per-expert token 比例的变异系数）持续 > 0.5 且不下降。

Warning. Expert collapse 不是罕见事件——没有 load balancing 机制的 MoE 训练几乎 100% collapse。所有主流 MoE 都必须有某种均衡机制。选哪种是 trade-off，不选是 bug。

方案 1: GShard/Switch Aux Loss

Lepikhin et al. 2020 (GShard) 提出的 auxiliary loss，也是最经典的一种。

数学定义

对一个 batch（总 N 个 token），定义两个 E 维向量：

$$f_e = \frac{1}{N} \sum_{n=1}^N \mathbb{1}[e \text{ 是 top-1 for token } n]$$

即“expert e 被 token 作为 top-1 选中的比例”。 $f \in [0, 1]^E$ ， $\sum f = 1$ 。

$$P_e = \frac{1}{N} \sum_{n=1}^N \text{gate_probs}[n, e]$$

即“gate 输出的 expert e 概率的 batch 平均”。 $P \in [0, 1]^E$ ， $\sum P = 1$ 。

Aux loss:

$$L_{\text{aux}} = E \cdot \sum_{e=1}^E f_e \cdot P_e = E \cdot (f \cdot P)$$

为什么这样定义？

理想情况： $f_e = P_e = \frac{1}{E}$ （完全均匀），此时 $L_{\text{aux}} = E \cdot E \cdot \left(\frac{1}{E}\right)^2 = 1$ ——达到最小值。

不平衡情况：某个 expert 特别受宠， $f_e = P_e \approx 1$ ，其他 = 0， $L_{\text{aux}} = E \cdot 1 \cdot 1 = E$ ——远大于 1，惩罚被放大 E 倍。

关键设计：

- f_e 不可微 (indicator function)，但不 **backprop** 也没关系——它只作为“这个 batch 实际发生了什么”的 scalar。
- P_e 可微，通过 $P_e \cdot f_e$ 的乘法收到梯度。梯度大小与实际负载 f_e 成正比：expert 越过载， P_e 收到的抑制越强。

直觉： L_{aux} 是“预期负载 × 实际负载”的相关系数放大版；当分布均匀时最小。

完整训练 **loss**

$$L_{\text{total}} = L_{\text{CE}} + \alpha \sum_{\text{layers}} L_{\text{aux}}^{(\ell)}$$

系数 α 通常 0.01。太大会让 router 过分追求均匀、伤害精度；太小 collapse。

实现

```
def switch_aux_loss(gate_probs, expert_indices, num_experts):
    """
    gate_probs: (N, E) - router 输出概率
    expert_indices: (N, K) - topk 后的 expert id (int)
    """
    N, E = gate_probs.shape
    # f_e: top-1 命中率
    top1 = expert_indices[:, 0] # (N,)
    f = torch.bincount(top1, minlength=E).float() / N # (E,)

    # P_e: gate_probs 均值
    P = gate_probs.mean(dim=0) # (E,)

    return E * (f * P).sum()
```

Warning: `bincount` 在 GPU 上是 sync 操作 (要跨 block 归并)，在 forward 里频繁调用会拖速度。生产实现用 `torch.scatter_add_` 或 pre-allocated buffer。

单纯 **aux loss** 的问题

(1) 梯度冲突

Router 收到两种梯度：主 loss 说“这个 token 应该去 expert 3”，aux loss 说“你太偏爱 expert 3 了，去别的”。两个梯度可能同号可能异号——异号时 router 停在次优解。

(2) 对 **top-K** 的偏差

f_e 只统计 **top-1**，忽略 top-2, top-3。如果 aux loss 只监督 top-1，router 可能学到“top-1 均匀，但 top-2 全部去 expert 0”——绕开监督。修正：

$$f_e = \frac{1}{NK} \sum_{n,k} \mathbb{1}[i_{n,k} = e]$$

统计所有 top-K。这是 Switch/ST-MoE 用的版本。

(3) L_{aux} 的 **scale** 依赖 **batch size**

f_e 是 batch 内比例, batch 越大波动越小、 L_{aux} 越接近理论最小值——即使 router 没变好, L_{aux} 也自然下降。评估 collapse 不能只看 L_{aux} 。

方案 2: Router Z-Loss (ST-MoE)

Zoph et al. 2022 (ST-MoE) 在 aux loss 基础上加了第二项——**router z-loss**。

目的

约束 `logsumexp(gate_logits)` 不要过大, 防止:

- Router 在训练早期把 logit 推到 10^3 量级 $\rightarrow$ softmax 极度锐化 $\rightarrow$ 熵坍塌
- fp16 溢出 (`exp(gate_logits)` overflow)

数学

$$L_z = \frac{1}{N} \sum_{n=1}^N \left(\log \sum_{e=1}^E \exp(\ell_{n,e}) \right)^2$$

即 log-sum-exp 平方的 batch 平均。系数 $\beta \approx 0.001$ 。

$$L_{\text{total}} = L_{\text{CE}} + \alpha \sum_{\ell} L_{\text{aux}}^{(\ell)} + \beta \sum_{\ell} L_z^{(\ell)}$$

为什么“平方”

L_z 就是要惩罚 $|\text{lse}|$ 大——对称。平方比绝对值有更好的梯度 (0 附近梯度小, 惩罚集中在极端值)。

Key insight. z-loss 本质是 **regularizer**, 不是均衡机制。它是“aux loss + z-loss”组合的一部分——单独用 z-loss 不能防止 collapse, 但配上 aux loss 能显著改善训练稳定性和推理时的 fp16 兼容性。ST-MoE、GLaM、Palm-2 都用这套。

方案 3: DeepSeek-V3 的 Aux-Loss-Free 均衡

DeepSeek-V3 (2024) 提出一种不用 **aux loss** 的方法, 用 可学习偏置:

核心思路

给每个 expert 一个 scalar bias b_e , 只在路由决策时生效:

$$\hat{\ell}_{n,e} = \ell_{n,e} + b_e$$

$$(w, i) = \text{topk}(\sigma(\hat{\ell}), K) \quad (\text{注意用 sigmoid 而非 softmax})$$

但计算 **gate** 权重时仍用 ℓ (不含 bias), 保证 b 只影响选谁、不污染 gate 权重梯度:

$$\tilde{w}_{n,k} = \sigma(\ell_{n,i_{n,k}})$$

(DeepSeek-V3 的具体公式里还有一步 K 内的归一化 + normalization。核心是 bias 只影响 top-k selection。)

动态调整 b_e

b_e 不是通过梯度下降学——它是训练循环里手动调整的参数 (放在 `buffer` 而不是 `parameter` 里):

```

import torch, torch.nn as nn, torch.nn.functional as F

class DeepSeekV3Router(nn.Module):
    """
    Aux-loss-free router (DeepSeek-V3, 2024).
    - gate logits + learnable bias 决定路由;
    - gate 权重 (发给下游 combine) 只用 gate logits (不含 bias);
    - bias 通过 rule-based 反馈调整, 不参与梯度.
    """
    def __init__(self, hidden_size: int, num_experts: int, top_k: int,
                  bias_lr: float = 1e-3):
        super().__init__()
        self.E, self.K = num_experts, top_k
        self.gate = nn.Linear(hidden_size, num_experts, bias=False)
        # bias 不参与 autograd - 用 buffer, 手动更新
        self.register_buffer("expert_bias", torch.zeros(num_experts))
        self.bias_lr = bias_lr

    def forward(self, x):
        # x: (N, H)
        logits = self.gate(x) # (N, E), 参与 autograd
        # 路由用 logits + bias, sigmoid 打分 (DeepSeek 用 sigmoid, 不是 softmax)
        scores = torch.sigmoid(logits + self.expert_bias) # (N, E)
        # top-K 选择
        _, indices = torch.topk(scores, self.K, dim=-1) # (N, K)

        # 给下游用的 combine 权重: 只用 logits (不含 bias!), 保持梯度纯净
        gate_scores = torch.sigmoid(logits) # (N, E)
        weights = torch.gather(gate_scores, -1, indices) # (N, K)
        weights = weights / weights.sum(-1, keepdim=True) # renorm
        return weights, indices, logits

    @torch.no_grad()
    def update_bias(self, indices: torch.Tensor):
        """
        每个 step 结束调用 (或每 N step 攒一次).
        indices: 本次 forward 的 top-K expert ids, shape (N, K).
        """
        # 统计每个 expert 被选中的次数
        load = torch.bincount(indices.flatten(), minlength=self.E).float()
        mean = load.mean()
        # 过载 → bias↓; 欠载 → bias↑
        # sign(load - mean) ∈ {-1, 0, +1}, 更新方向反过来
        self.expert_bias -= self.bias_lr * torch.sign(load - mean)

```

`bias_lr` 是小学习率 (DeepSeek-V3 论文里用 0.001)。等价于反馈控制：过载专家的 bias 减少 → 下次更难被选 → 负载下降；欠载相反。

分布式注意：`load` 必须先 AllReduce 到全局 (跨 DP + EP 组) 再更新 bias，否则每 rank 各自往不同方向调，训练发散。生产实现：

```
if dist.is_initialized():
    dist.all_reduce(load, op=dist.ReduceOp.SUM)
```

为什么不用梯度？

因为 b 只影响 topk 的索引选择（离散决策）——从 b 到最终 loss 的映射经过 topk 这个不可导算子，梯度是 0。手动 rule-based 调整绕开了这个不可导性。

Key insight. Aux-loss-free 的关键优势：**router** 的梯度信号纯净——只从主任务学“哪个专家对这个 token 好”，不需要同时学“我要负载均衡”。DeepSeek-V3 报告在 671B 规模上比 aux-loss 版本 loss 更低。

代价

- 需要 sigmoid gating（不是 softmax），部分改变数值动力学。
- 需要额外的动态更新逻辑，不能纯 forward。
- 效果对 γ 敏感。

方案对比

方案	Router 梯度纯度	实现复杂度	代表	备注
Aux loss	中 (有干扰)	低	GShard, Mixtral	alpha=0.01
Aux + z-loss	中	中	ST-MoE, GLaM	beta=0.001
Aux-loss-free bias	高 (无干扰)	中	DeepSeek-V3	gamma=0.001
Expert Choice routing	高	高	Zhou 2022	反向选 token
Capacity + drop	(orthogonal)	低	Switch	限制上界

Expert Choice：反向路由

Zhou et al. 2022 提出一种反直觉方案：不是 **token** 选专家，而是专家选 **token**。

每个专家从所有 N 个 token 中，按打分选前 $C = \text{top-}C$ 个：

```
# 常规: for each token, pick top-K experts
# expert choice: for each expert, pick top-C tokens

scores = gate_logits # (N, E)
# 按 E 维 topk (每 expert 选 top-C 个 token)
top_scores, top_token_ids = torch.topk(scores.T, k=C, dim=-1)
# top_token_ids: (E, C)
```

结果：每个专家恰好收 C 个 **token**，天然均衡；但每个 **token** 可能被 $0/1/2/\dots$ 个专家选中，不是恰好 K 。

$C = N \frac{K}{E}$ 时总计算量与 top-K 版本相同，但同一个 **token** 可能被多个专家选（如果它对多个专家都得分高），也可能被 0 个专家选（罕见但要处理，一般走 residual 跳过）。

优缺点：

- ✓ 完美负载均衡，无需 aux loss
- ✓ Router 梯度纯净
- ✗ 破坏了“每 token K 个专家”的对称性——同一 token 收到的 expert 数量随机
- ✗ 训练与推理不一致（推理时通常仍用 top-K）
- ✗ 未成为主流（Google 内部用，开源社区少见）

实操：监控 collapse

训练时除了 loss 曲线，还必须监控以下指标（wandb 每 100 step log 一次）：

1. f_e 直方图：per-expert token 比例
2. $\frac{\text{std}(f)}{\text{mean}(f)}$ ：均衡度指标（< 0.1 良好，> 0.5 需要介入）
3. Router 熵 $H(p) = -\sum_e p_e \log p_e$ ：随训练先增后稳才对
4. Drop rate：如果用 capacity，每 step 记录被 drop 的 token 数
5. L_{aux} 值：应缓慢单调下降到 ≈ 1 ；稳定 > 1.5 说明 collapse

一个够生产用的监控 hook（挂在 MoE 层上即可）：

```
import torch, torch.distributed as dist

class MoEMonitor:
    """挂在每个 MoE 层，每 log_every 个 step 汇总一次."""
    def __init__(self, num_experts: int, log_every: int = 100):
        self.E = num_experts
        self.log_every = log_every
        self.reset()

    def reset(self):
        self.load_sum = torch.zeros(self.E, device="cuda") # 累加每 step 的 load
        self.entropy_sum = torch.tensor(0.0, device="cuda")
        self.drop_sum = torch.tensor(0.0, device="cuda")
        self.n_steps = 0

    @torch.no_grad()
    def record(self, gate_probs: torch.Tensor, indices: torch.Tensor,
               n_dropped: int = 0):
        # gate_probs: (N, E) softmax 后
        # indices: (N, K) 被选专家
        load = torch.bincount(indices.flatten(), minlength=self.E).float()
        self.load_sum += load
        # 熵：平均在 batch 上
        p = gate_probs.clamp_min(1e-9)
        self.entropy_sum += -(p * p.log()).sum(-1).mean()
        self.drop_sum += n_dropped
        self.n_steps += 1

    @torch.no_grad()
    def flush(self, step: int, writer=None):
        """返回一个 dict，可以直接 log 到 wandb/tensorboard."""
        if dist.is_initialized():
            dist.all_reduce(self.load_sum)
            dist.all_reduce(self.entropy_sum)
```



```

        dist.all_reduce(self.drop_sum)

    f = self.load_sum / self.load_sum.sum()           # 归一化的 per-expert 比
    stats = {
        "moe/load_std_over_mean": (f.std() / f.mean()).item(),
        "moe/router_entropy":      (self.entropy_sum / self.n_steps).item(),
        "moe/drop_rate":            (self.drop_sum / self.n_steps).item(),
        "moe/min_load_frac":        f.min().item(),
        "moe/max_load_frac":        f.max().item(),
    }
    if writer is not None:
        for k, v in stats.items():
            writer.add_scalar(k, v, step)
        writer.add_histogram("moe/per_expert_load", f, step)
    self.reset()
    return stats

# 用法
monitor = MoEMonitor(num_experts=8, log_every=100)
for step, batch in enumerate(data_loader):
    weights, indices, gate_probs = router(batch)
    ...
    monitor.record(gate_probs, indices, n_dropped=drop_count)
    if step % monitor.log_every == 0:
        print(monitor.flush(step))

```

Note. 见到 collapse 的三个补救顺序：(a) 检查 router weight init ($\text{std} \leq 0.02$)；(b) 提升 aux loss 系数 ($0.01 \rightarrow 0.03 \rightarrow 0.1$)；(c) 换 aux-loss-free 或加 z-loss。如果都不行，可能是数据分布问题——不同专家对不同数据类型的偏好本身就有物理 skew（比如代码 vs 自然语言），一味均衡反而伤害精度。

面试考点

面试考点. Q1: 为什么 aux loss 的定义里 f 不可微没关系？

A: f 只做为“已发生事件的 scalar”，与 P 相乘后， P 一侧可微、梯度通过 softmax 回传到 router。 f 相当于给 P 加权重的“目标分布”——大 f （过载）的 expert 在其 P 分量上有大梯度惩罚。

面试考点. Q2: 为什么 aux loss 系数是 0.01 而不是 1.0？

A: aux loss 只是辅助约束、不是主任务。系数太大 (> 0.1) 会让 router 完全牺牲精度追求均匀，模型 loss 显著变差；太小 collapse。0.01 是经验值。DeepSeek 报告 aux-loss-free 在他们的规模下比 aux loss 好，可能因为 aux loss 系数在千亿模型上难以精调。

面试考点. Q3: Z-loss 和 aux loss 的作用差异？

A: aux loss 管分布均衡 (防 collapse); z-loss 管数值稳定 (防 logit 爆炸)。前者约束 softmax 输出的形状, 后者约束 softmax 输入的 magnitude。二者正交, 可以 (也应当) 同时用。

面试考点. **Q4**: DeepSeek 的 aux-loss-free 里 b 不参与梯度, 那 b 是怎么初始化的? 初始 collapse 怎么办?

A: b_e 初始化为 0——所有专家等价。第一个 batch 后统计负载、调整 b , 几十个 step 内负载趋于均衡。文章里没细说 γ warmup 策略, 但社区复现建议前 1000 step 用大一点的 $\gamma = 0.01$, 之后降到 0.001。

面试考点. **Q5**: Expert Choice 里如果某 token 被 0 个专家选中怎么办?

A: 通过 residual 分支跳过 MoE 层 ($y = x$)。理论上极少发生 (除非某 token 对所有专家都打分极低); 实操中往往加一个 shared expert (所有 token 都走) 兜底, 见 DeepSeek-MoE 的设计。

面试考点. **Q6**: 如果 capacity + drop + aux loss 同时用, 会不会冲突?

A: 不会——它们互补。Aux loss 让 router 输出趋于均衡 (减少 drop 概率); capacity 是硬上限 (防最坏情况)。生产上通常都开: aux loss $\alpha = 0.01$ + capacity factor 1.25。DeepSeek-V3 更极端: 既有 no-drop 目标又有 bias tuning, 实测 drop 率 $< 0.1\%$ 。

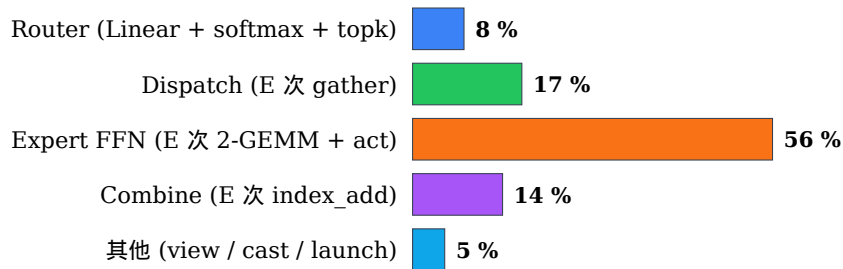
单机训练性能优化

前面五章讲了 MoE 能算对，这一章讲怎么算得快。目标是单机场景下（无跨机通信），把 MoE 层从“naive 慢 3-5×”打到“接近 dense FFN 的效率”。

分布式（EP、all-to-all）留到第 8 章——那需要先理解本章的 grouped GEMM 基础。

瓶颈画像：先看开销分布

在 A100/H100 上跑一个中等 MoE 层（ $BS = 8192, H = 4096, I = 4H, E = 8, K = 2$ ），naive PyTorch 实现（范式 A，for-loop）的 forward wall-clock 时间大致按下面比例分布（数量级估算，实测值随 batch size 与硬件浮动）：



关键观察：

(1) Expert FFN 占大头（约 56%），但效率极差——每个 sub-GEMM 的 M 维只有 $\sim B \frac{S}{E} = \frac{8192}{8} = 1024$ （假设完美均衡）或更小（实际 skewed）。TensorCore 需要 $M \geq 128$ 才吃满， $M_e = 30$ 量级时 GEMM 效率跌到 15-30%。

(2) Dispatch + Combine 加起来约 31%——这在 dense FFN 里完全不存在，是 MoE 的额外税。

(3) Router 只占 8%，优化收益有限。

一个 **dense FFN** 的对比：同规模 dense FFN 的 GEMM 效率 80-90%——MoE 有巨大的头顶空间。

优化 ladder

版本	核心思路	相对成本 / 收益
v0: naive for-loop	第 5 章 test_moe.py	基线
v1: grouped GEMM	permute + torch._grouped_mm	1.5-2×
v2: v1 + fused router	router 融合成 1 kernel	+3-5%
v3: v2 + fused permute	permute 融进 grouped GEMM prologue	+10-15%
v4: v3 + fused activation	SwiGLU 融成 1 kernel	+5%
v5: v4 + block-sparse	无 drop / 无 padding	+5-10% (drop 时更多)
v6: v5 + torch.compile	auto tuning + kernel fuse	+5-15% (case-dependent)

从 v0 到 v6，端到端 MoE 层加速 3-4×。生产系统 (Megatron-MoE, Tutel) 一般在 v3/v4 附近。本章逐个讲每一层的原理与代价。

v1: Grouped GEMM

第 4 章“范式 B”已经讲过 permute + grouped GEMM 的原理。这里聚焦为什么快。

单次 **grouped GEMM** 的收益公式

假设 E 个专家，每个 M_e 个 token，做 $(M_e, H) \times (H, I)$ ：

Naive (v0): E 次独立 GEMM

- 每次 kernel launch: $\sim 5\mu s$
- Tile scheduling: 每 GEMM 独立
- SM 利用率: 若 $M_e < 256$ ，仅约 30%

Grouped (v1): 1 次 grouped GEMM

- 1 次 launch
- Kernel 内部把 E 个 sub-problem 拆成 tile 全局调度
- SM 利用率: 60 – 80%

总节省: launch overhead $(E - 1) \times 5\mu s = 35\mu s$ (for $E=8$) + GEMM 加速约 $1.5 \times$ 。

对 $E = 8$ ， $M_e \approx 250$ ，端到端 MoE forward 从 $\sim 600\mu s$ 降到 $\sim 300\mu s$ 。

代码骨架

```
def moe_grouped_gemm(hidden, W_up, W_down, expert_indices, expert_weights):
    N, H = hidden.shape
    E, _, I = W_up.shape # W_up: (E, H, I), W_down: (E, I, H)
    K = expert_indices.shape[1]

    # 1. 展平 (N, K) → (N*K, ), 记录每个 slot 的目标 expert
    flat_expert = expert_indices.view(-1) # (N*K, )
    flat_weight = expert_weights.view(-1) # (N*K, )
    # 每个 slot 对应的 token id
    flat_token = torch.arange(N, device=hidden.device).repeat_interleave(K)

    # 2. 按 expert 排序 → permute
    permute_idx = flat_expert.argsort(stable=True) # (N*K, )
    perm_tokens = flat_token[permute_idx] # (N*K, ) 排序后的 token
    id
    packed_input = hidden[perm_tokens] # (N*K, H)
    packed_weight = flat_weight[permute_idx] # (N*K, )

    # 3. 每个 expert 的 token 数
    group_sizes = torch.bincount(flat_expert, minlength=E) # (E, )

    # 4. Grouped GEMM (fc1)
    packed_hidden = torch._grouped_mm(packed_input, W_up, group_sizes)
    # (N*K, I)

    # 5. Activation
```

```

packed_hidden = F.relu(packed_hidden) # 或 SwiGLU

# 6. Grouped GEMM (fc2)
packed_out = torch._grouped_mm(packed_hidden, W_down, group_sizes)
# (N*K, H)

# 7. Weight
packed_out = packed_out * packed_weight.unsqueeze(-1)

# 8. Unpermute: 逆排列 + reduce over K
inverse_idx = permute_idx.argsort() # (N*K,)
unpacked = packed_out[inverse_idx] # (N*K, H) - 恢复到 (token, k) 排列
out = unpacked.view(N, K, H).sum(dim=1) # (N, H)

return out

```

需要注意的点

(1) `stable=True` 排序

必需——保证同一 expert 内部 token 的相对顺序稳定,方便 unpermute。unstable 排序会让 debug 变噩梦。

(2) `torch._grouped_mm` 的 API 变化

PyTorch 2.4+ 提供,但 API 在演进。生产实现用 CUTLASS/Triton 的 grouped GEMM。开源可参考 `grouped_gemm` (NVIDIA) 或 SGLang/vLLM 里的实现。

(3) M_e 可以为 0

`group_sizes[e] = 0` 时 kernel 应该跳过——早期 CUTLASS 版本有 bug 会 nan。测试时先跑 $E = 2, M_1 = 0, M_2 = NK$ 的极端 case 验证。

(4) W_{stacked} 的 layout

`W_up: (E, H, I)` 意味着 E 份权重连续存储。加载时对每个 group, kernel 会从 `W_up[e]` 起始位置取。这个 layout 也影响后续的 EP 权重分片——不同 EP rank 持有不同 slice `W_up[e_start:e_end]`。

v2: Fused Router

Router 的 Linear + softmax + topk + renorm 是 4 次 kernel launch。融成一个 kernel 的关键 trick: E 很小 (≤ 32 常见), 一整行 logits 可以塞进 register/shared memory, 从头到尾不落 HBM。

Triton 参考实现

只写“logits $\rightarrow$ softmax(fp32) $\rightarrow$ top-K $\rightarrow$ renorm”这四步的 fusion (假设 GEMM 之后接这个 kernel; 或者更激进的实现把 gate GEMM 也 fuse 进来, 见下文)

```

import torch, triton
import triton.language as tl

@triton.jit
def fused_topk_softmax_kernel(

```

```

logits_ptr,          # (N, E), 输入
weights_ptr,         # (N, K), 输出 renormed weights
indices_ptr,         # (N, K), 输出 expert ids
N, E: tl.constexpr, # N runtime, E compile-time (小, ≤32)
K: tl.constexpr,
BLOCK_N: tl.constexpr,
):
    pid = tl.program_id(0)
    row_ids = pid * BLOCK_N + tl.arange(0, BLOCK_N) # (BLOCK_N,)
    row_mask = row_ids < N

    # -- 1. load logits (BLOCK_N, E) --
    col = tl.arange(0, E) # (E,)
    ptrs = logits_ptr + row_ids[:, None] * E + col[None, :] # (BLOCK_N, E)
    x = tl.load(ptrs, mask=row_mask[:, None], other=-float("inf"))
    x = x.to(tl.float32) # fp32
accumulator

    # -- 2. softmax (fp32, numerically stable) --
    x = x - tl.max(x, axis=1, keep_dims=True)
    e = tl.exp(x)
    p = e / tl.sum(e, axis=1, keep_dims=True) # (BLOCK_N, E)

    # -- 3. top-K by repeated masked argmax (K 小, unrolled) --
    for k in tl.static_range(K):
        v = tl.max(p, axis=1, keep_dims=True) # (BLOCK_N, 1)
        idx = tl.argmax(p, axis=1) # (BLOCK_N,)
        # 写回本轮 top-1
        out_w = weights_ptr + row_ids * K + k
        out_i = indices_ptr + row_ids * K + k
        tl.store(out_w, tl.reshape(v, (BLOCK_N,)), mask=row_mask)
        tl.store(out_i, idx.to(tl.int32), mask=row_mask)
        # mask 掉本轮 winner, 进入下一轮
        p = tl.where(col[None, :] == idx[:, None], -float("inf"), p)

    # -- 4. renormalize (读回 K 个权重, 除以 sum) --
    # 如果 K 很小可以在上面循环里就累一个 sum, 然后最后一次写; 这里为清晰分开。
    w_ptrs = weights_ptr + row_ids[:, None] * K + tl.arange(0, K)[None, :]
    w = tl.load(w_ptrs, mask=row_mask[:, None])
    w = w / tl.sum(w, axis=1, keep_dims=True)
    tl.store(w_ptrs, w, mask=row_mask[:, None])

def fused_topk(logits: torch.Tensor, K: int, BLOCK_N: int = 64):
    N, E = logits.shape
    weights = torch.empty((N, K), dtype=torch.float32, device=logits.device)
    indices = torch.empty((N, K), dtype=torch.int32, device=logits.device)
    grid = (triton.cdiv(N, BLOCK_N),)
    fused_topk_softmax_kernel[grid](
        logits, weights, indices,
        N=N, E=E, K=K, BLOCK_N=BLOCK_N,

```

```
)
return weights, indices
```

用法：

```
gate_logits = hidden @ W_gate.T          # (N, E), bf16 GEMM 依然走 cuBLAS
w, idx = fused_topk(gate_logits, K=2)    # (N, K), (N, K)
```

关键点

(1) fp32 accumulator 是刚需

`x.to(tl.float32)` 是这个 kernel 的正确性核心——第 3 章讲过 bf16 下 softmax 会崩。这也是为什么 fused router 一定要自己写：`F.softmax(..., dtype=torch.float32)` 内部会 materialize 一个 fp32 中间 tensor，而这里全程在 register。

(2) top-K 的选择

上面用“K 次 masked argmax”是 $O(KE)$ ，对 $K \leq 8, E \leq 32$ 完全够用。想更快可以用 bitonic sort 的 static kernel (vLLM `fused_moe.py` 的做法)，但代码复杂 3 倍、收益 $< 1\%$ 。

(3) 一步到位的 gate + router fusion

上面只 fuse 了 softmax/topk/renorm。更激进的实现把 gate GEMM 也 fuse 进来： $X (bf16, (N, H)) \rightarrow W_g (bf16, (H, E)) \rightarrow$ 一个 kernel 直接输出 (weights, indices)。E 很小时 GEMM 是 memory-bound skinny GEMM，用 Triton 手写反而比 cuBLAS 快 20-30%。SGLang 的 `moe_align_block_size` 和 vLLM `fused_moe_router` 都走这条路。

收益: 3-5% 端到端。相对小但免费——因为 CUDA graph 需要静态图，fuse 后减少 launch 有额外的 scheduling 收益。

代表实现：`vllm/model_executor/layers/fused_moe/fused_moe.py::fused_topk` (同样的 Triton 结构，做了更多 tuning)；SGLang `moe_align_block_size`。

v3: Fused Permute

`Permute(hidden[perm_tokens])` 本质是一个 gather，读 (NK, H) elements。有两条 fusion 思路：

1. 独立 **permute kernel** (简单，也能省 10-15%) —— 把 gather 写成一个 Triton kernel，比 `hidden[perm_tokens]` 少一次中间 tensor 的 HBM 往返。
2. 融进 **grouped GEMM prologue** (激进) —— 不 materialize `packed_input`，直接从 hidden gather 到 GEMM 的 shared memory。生产实现基本都走这条。

方案 1: 独立 permute 的 Triton 实现

```
@triton.jit
def permute_kernel(
    src_ptr,          # (N, H) hidden_states
    dst_ptr,          # (M, H) packed_input, M = N*K
    perm_tokens_ptr,  # (M,) 每个 slot 的源 token id
    M, H: tl.constexpr,
    BLOCK_H: tl.constexpr,
):
```

```

# 每个 program 负责一行 packed_input
m = tl.program_id(0)
src_row = tl.load(perm_tokens_ptr + m)
h_off = tl.arange(0, BLOCK_H)
# 循环整行 (H 可能 > BLOCK_H)
for h0 in range(0, H, BLOCK_H):
    offs = h0 + h_off
    mask = offs < H
    v = tl.load(src_ptr + src_row * H + offs, mask=mask)
    tl.store(dst_ptr + m * H + offs, v, mask=mask)

def permute(hidden, perm_tokens, BLOCK_H=128):
    N, H = hidden.shape
    M = perm_tokens.numel()
    dst = torch.empty((M, H), dtype=hidden.dtype, device=hidden.device)
    permute_kernel[(M,)](hidden, dst, perm_tokens, M=M, H=H, BLOCK_H=BLOCK_H)
    return dst

```

反向就是 `index_add_` (对同一 `src_row` 累加 K 次 `dst` 梯度), 也可以写成一个 atomic add Triton kernel。

方案 2: 融进 **grouped GEMM** 的 **prologue**

伪代码：

```

grouped_gemm_with_permute_kernel(hidden, W_up, perm_tokens, group_offsets, ...):
    // 每个 CTA 处理一个 GEMM tile: BLOCK_M × BLOCK_N
    m0, group_id = tile_to_group(blockIdx, group_offsets)

    // 加载 BLOCK_M 行——原本从 packed_input 读, 现在从 hidden gather
    for i in range(BLOCK_M):
        token_id = perm_tokens[m0 + i]
        As[i, :] = hidden[token_id, :]           // gather 直接到 shared memory

    // 加载 W_up[group_id] 的 BLOCK_N 列到 Bs
    // Normal tiled GEMM ...

```

Triton 里可以直接把 gather 写在 `tl.load` 的 pointer 表达式里：

```

# grouped GEMM inner loop 里, 读 A tile 的部分
row_ids = m0 + tl.arange(0, BLOCK_M)           # (BLOCK_M,)
token_ids = tl.load(perm_tokens_ptr + row_ids)   # gather idx
a_ptrs = hidden_ptr + token_ids[:, None] * H + k_offs[None, :]
a = tl.load(a_ptrs, mask=...)
# 后续 tl.dot(a, b) 走 tensor core

```

收益: 10-15%。省了一次 (NK, H) 的 HBM 读写。当 NKH 大 (千 MB 级) 时收益显著。

代价: kernel 复杂——permute 的 gather pattern 会破坏内存 coalescing, 需要 shared memory reshuffle 优化。NVIDIA 的 `grouped_gemm` 库、Megatron 的 `moe_permute` op、Tutel 的

`fast_dispatch` 都做了这个 fusion。裸调 `torch._grouped_mm` 不包含 fused permute，需要额外 wrap。

Key insight. 从 v1 到 v3 是“MoE 层从教学到生产”的核心转变。v1 用现成 API 就能实现；v3 需要 CUDA/CUTLASS 级别的自定义 kernel。生产实现的 90% 复杂度在 v3 这一步。

v4: Fused SwiGLU

SwiGLU 里 $\text{SiLU}(x) \cdot y$ 是 elementwise 但不可交换——需要 gate 和 up 两路都在 register 才能算。生产的做法是让一次 grouped GEMM 同时产出 gate 和 up，然后在 epilogue 里直接 SiLU + 乘。

Weight concat 布局

关键工程 trick：把 `W_gate` 和 `W_up` 沿输出维 concat：

```
# 训练开始时构造一次
W_gate_up = torch.cat([W_gate, W_up], dim=-1)      # (E, H, 2*I)
```

于是一次 grouped GEMM 直接出 $(NK, 2I)$ ：

```
def swiglu_grouped_gemm(packed_input, W_gate_up, W_down, group_sizes):
    # packed_input: (M, H), W_gate_up: (E, H, 2*I), W_down: (E, I, H)
    gate_up = torch._grouped_mm(packed_input, W_gate_up, group_sizes) # (M, 2I)
    hidden   = fused_silu_glu(gate_up)                                # (M, I)
    out      = torch._grouped_mm(hidden, W_down, group_sizes)         # (M, H)
    return out
```

fused_silu_glu 的 Triton 实现

$\text{SwiGLU}(x, y) = \text{SiLU}(x) \cdot y = \left(\frac{x}{1+e^{-x}}\right) \cdot y$ 。写成一个 kernel（含反向，因为训练要）：

```
@triton.jit
def silu_glu_fwd_kernel(
    inp_ptr,          # (M, 2*I) - 前 I 列是 gate, 后 I 列是 up
    out_ptr,          # (M, I)
    M, I: tl.constexpr,
    BLOCK: tl.constexpr,
):
    pid = tl.program_id(0)                                # 一个 program 负责一行的一段
    row = pid // tl.cdiv(I, BLOCK)
    col0 = (pid % tl.cdiv(I, BLOCK)) * BLOCK
    cols = col0 + tl.arange(0, BLOCK)
    mask = (row < M) & (cols < I)

    gate = tl.load(inp_ptr + row * 2 * I + cols, mask=mask).to(tl.float32)
    up    = tl.load(inp_ptr + row * 2 * I + I + cols, mask=mask).to(tl.float32)

    silu = gate * tl.sigmoid(gate)
    out   = silu * up
```

```

tl.store(out_ptr + row * I + cols, out.to(inp_ptr.dtype.element_ty), mask=mask)

def fused_silu_glu(x: torch.Tensor) -> torch.Tensor:
    M, two_I = x.shape
    assert two_I % 2 == 0
    I = two_I // 2
    y = torch.empty((M, I), dtype=x.dtype, device=x.device)
    BLOCK = 128
    grid = (M * triton.cdiv(I, BLOCK),)
    silu_glu_fwd_kernel[grid](x, y, M=M, I=I, BLOCK=BLOCK)
    return y

```

对应的 backward 也一样简单： $\partial_{\frac{L}{\partial}} x_{\text{gate}} = \partial_{\frac{L}{\partial}} \text{out} \cdot y \cdot \text{SiLU}'(x)$ ， $\partial_{\frac{L}{\partial}} x_{\text{up}} = \partial_{\frac{L}{\partial}} \text{out} \cdot \text{SiLU}(x)$ ，写一个双输出 kernel 就行。

更激进：Option B — 完整 fused MoE mega-kernel

vLLM 的 `fused_moe.py::fused_experts` 把 permute + grouped GEMM 1 + SwiGLU + grouped GEMM 2 + weight + unpermute 全部融成一个 kernel (推理侧，无 backward)。训练用得少——灵活性差、backward 难写。

收益: SwiGLU epilogue fusion 约 5%。完整 mega-kernel 在小 batch 下额外 +10%。

v5: Block-Sparse (Megablocks)

第 4 章“范式 C”已讲过原理。这里补充训练场景的对比：

Grouped GEMM (v1-v4):

- 需要 capacity + drop，或者手动 padding
- Drop 造成信息丢失，尤其训练早期 collapse 时严重
- Kernel 简单，主流 (Megatron, Tutel, DeepSpeed)

Block-Sparse (Megablocks):

- 无 drop，训练动力学更好
- Kernel 复杂 (block-sparse metadata)
- 单机成熟，分布式仍在演化

一个具体数字：Megablocks 论文报告在 1.3B MoE 上 vs 相同 capacity 的 grouped GEMM 版本，同 **wall-clock** 训练下 loss 低 0.02——不多，但免费。

工程选择：如果 team 有 CUDA 人力，Megablocks 值得投入；如果依赖开源框架，grouped GEMM 是稳妥选择。

v6: torch.compile

`torch.compile` 对 MoE 的作用取决于代码结构：

(1) 范式 A (for-loop) 上编译

有 graph break (`torch.where` + 动态 shape gather)。收益有限 (< 5%)。

(2) 范式 B (grouped GEMM) 上编译

Router 和 permute 部分能编译, grouped GEMM 本身是 pre-compiled kernel(编译器不动它), 整体收益 10-15%。

(3) 完整 fused MoE 上编译

如果已经 v5 级别 fused, `torch.compile` 主要吃 Python overhead 层(每 forward 的调度)——CUDA graph capture 有额外收益。

生产实现里 `torch.compile` 更多用在 non-MoE 部分 (attention、norm); MoE 部分依赖手写 kernel。

显存优化

MoE 显存开销与 dense 的差别:

(1) 权重: E 倍。 $E = 8$ 就是 8 倍 FFN 权重。这是 MoE 的固有开销。

(2) 激活: 与 dense 相当。中间 packed_hidden (NK, I) 只比 dense 大 K 倍。

(3) **Optimizer state**: 与权重同比例增大。AdamW 每参数 12 bytes (bf16 训练下 fp32 momentum + fp32 variance + fp32 master + bf16 param + bf16 grad $\approx 4+4+4 = 12$ for state, param/grad separate), 是模型权重的 6-8 $\times$ 。

显存优化手段:

(a) **ZeRO-1/2/3** 分片: optimizer state / grad / param 沿 DP 维度切。DeepSpeed 集成。

(b) **Selective activation ckpt**: packed_hidden (NK, I) 是最大激活, checkpoint 掉最省内存。代价 30% 显存 vs 30% 额外算力。

(c) **Expert offload**: 冷专家 (低命中率) 参数 offload 到 CPU/NVMe, 需要时 prefetch。稀疏专家 (DeepSeek 256 experts) 这个策略特别有效——推理侧 vLLM 已经实现。

(d) 量化权重: bf16 $\rightarrow$ fp8 (Hopper) 或 int4 (推理)。训练侧 fp8 主参数 + fp32 master 是当前趋势 (TransformerEngine)。

面试考点

面试考点. Q1: Grouped GEMM 相比 for-loop GEMM 的核心收益来源?

A: 三点: (1) 减少 $E - 1$ 次 kernel launch overhead; (2) 全局 tile scheduling —— 小 group 和大 group 混合调度、SM 常驻满载; (3) L2 缓存复用 —— weight 数据在 GEMM 内跨 group 尝试保留。SM 利用率从 30% 提升到 60-80%。

面试考点. Q2: 为什么 permute 融合到 GEMM prologue 收益大?

A: Permute 独立做时, (NK, H) 的 tensor 要写一次 HBM 再读一次; 融合后从原始 hidden 直接 gather 到 GEMM shared memory, 不产生中间 tensor 的 HBM 往返。当 NKH 是几百 MB 量级时, 节省的 HBM 带宽换算成 latency 就是 10-15% 端到端。

面试考点. Q3: 训练时能不能不 drop token?

A: 三种方式: (1) 极大 capacity factor (≥ 2), 牺牲显存 + kernel 效率; (2) 用 Megablocks (block-sparse), 从算法上避免; (3) 用 Expert Choice routing, 天然均衡。Mixtral 训练用 (1) 兜底, DBRX 用 (2)。

面试考点. **Q4**: 单机 MoE 里 activation checkpointing 该 checkpoint 什么?

A: packed_hidden: ($N \times K$, I) 是最大 activation ($4K \times$ dense FFN), first choice。次选是 gate_probs (N , E) 但小。不要 checkpoint expert_indices — 是 int, 重算成本高 (要重跑 softmax + topk)。

面试考点. **Q5**: MoE 的 SM 利用率天生比 dense 低吗?

A: 不一定。dense FFN 是一个大 GEMM, SM 利用率 80-90%; MoE grouped GEMM 因为 group size 可能小、L2 复用差、tile 调度不完美, 天花板约 70-80%。差距 10-15% —— 用大 batch (让 M_e 更大) 可以缩小。

面试考点. **Q6**: 如果我在 forward 里发现某个 expert 显存溢出, 能动态 offload 吗?

A: 训练侧不建议——forward/backward 都需要该 expert 的权重, 来回搬 CPU 会显著拖慢。正确做法: (1) EP 分布式, 让 expert 分布到不同 GPU; (2) 上 fp8 减半权重内存; (3) offload 只对冷 **expert** 有意义 (推理侧 vLLM 用得更多)。

分布式训练 :EP、All-to-All 与 Fine-Grained Overlap

单机 MoE 的性能上限是单张 GPU 的算力和显存。要训 100B+ 的 MoE 模型，必须跨机分布式。这是本书最长、也最容易出错的一章。

我们的组织方式：先建立 3 堵墙的诊断框架(Memory / Communication / Compute Efficiency)，再依次讲每堵墙 Megatron-Core、DeepSeek-V3、MegaScale-MoE 这些生产系统各自怎么打。这三堵墙强耦合——解决一堵往往会暴露另一堵，工业界的所谓“最佳实践”其实就是在三者之间反复权衡的结果。

MoE 训练的三堵墙 (Three Walls)

Megatron-Core MoE tech report¹ 把 MoE 分布式训练遇到的问题归纳成三堵互相制约的墙：

Wall	根因	度量指标	典型症状
Memory Wall	E 个 expert 的 weight/grad/ opt state 全部常驻，但每 token 只激活 K 个	GB/GPU	DeepSeek-V3 BF16、PP4×VPP4×EP64、256 GPU： 199.5 GB/GPU ——H100 80GB 装不下
Communication Wall	EP 需要 dispatch + combine 两次 all-to-all；跨节点带宽骤降 (IB 50 GB/s vs NVLink 160 GB/s)	step time 中通信占比	未优化时 EP a2a 占 forward 20-60% ；DeepSeek-V3 类跨节点场景可达 60%
Compute Efficiency Wall	细粒度 expert → 小 GEMM + 大量 kernel launch + host sync	SM 利用率 / MFU	细粒度 MoE 每 expert 只有 128 tokens，远低于 TensorCore 饱和阈值 (M ≥ 128)；dropless routing 引入 D2H sync

表 6 MoE 训练的三堵墙。三者强耦合：解决内存墙 → 引入 EP → 暴露通信墙 → overlap → 需要更大 batch → FP8 → kernel 碎片化 → CUDA Graph。

a2a 通信量公式（每卡每次 dispatch）：

¹Cui et al., Scalable Training of Mixture-of-Experts Models with Megatron-Core, 2026.

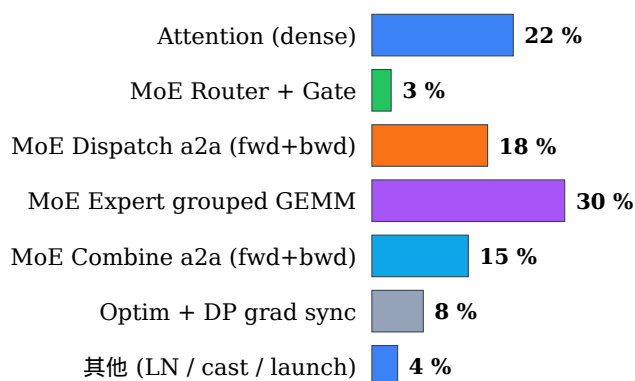
$$\text{vol}_{a2a} \approx T \cdot K \cdot h \cdot \frac{EP - 1}{EP}$$

一次 MoE 层 forward + backward 有 4 次 a2a (2 dispatch + 2 combine), 32 层模型每步 total **16-64 GB** per GPU。跨节点未 overlap 时这直接决定 step time。

Key insight. 3 walls 的意义是给出诊断顺序：先量出你当前 bottleneck 在哪堵墙，再选对应武器。Megatron-Core 的 `--profile` 与 DeepSeek profile-data² 都提供了区分这三类开销的 trace 工具。

参考基线：一个 **8 × 8 × 7B MoE** 的开销分布

以 Mixtral 8×7B (32 层、 $H = 4096, I = 14336, E = 8, K = 2$) 在 64 × H100 上、TP=1, PP=4, EP=8, DP=2, MBS=1, seq=4096 的 baseline 数字 (Megatron-Core v0.9 官方 benchmark 附近)：



读法：Attention + Expert 加起来 52%——这是“有效计算”，剩下 48% 里 33% 是 MoE 特有的 a2a。这就是 3 walls 的具体体现——你能把 33% 的通信 overlap 到接近 0，就能拿到理论上限的 1.7× 加速；这正是 DeepSeek DualPipe 与 Megatron-Core `--overlap-moe-expert-parallel-comm` 的战场。

数字都是数量级估算，实际 profile 请以 nsys / DeepSeek profile-data 为准。

四种并行的分工

MoE 分布式训练涉及四个正交的并行维度：

并行	切什么	通信原语	对 MoE 的作用
DP (Data Parallel)	batch 沿样本切	AllReduce (grad)	每 replica 有完整模型
TP (Tensor Parallel)	单个 GEMM 内切张量	AllReduce / AllGather	切 attention/expert 内的 GEMM
PP (Pipeline Parallel)	沿 layer 切	P2P send/recv	切 Transformer stack
EP (Expert Parallel)	专家沿 expert 维切	All-to-All	MoE 独有

四种并行可以任意组合：

$$\text{world_size} = \text{DP} \times \text{TP} \times \text{PP} \times \text{EP}$$

²<https://github.com/deepseek-ai/profile-data>

例如 Mixtral 8×7B 训练常见配置 :world = 64 卡 ,TP = 2, EP = 8, DP = 4, PP = 1 , $2 \times 8 \times 4 = 64$ 。

为什么需要 Expert Parallel

单卡 A100 80GB 能装多大 MoE ?

- Mixtral 8×7B : 47B params $\times$ 2 bytes (bf16) = 94 GB — 装不下
- 加上 optimizer state (12 bytes $\times$ 47B = 564 GB) — 需要 8+ 卡

必须切模型。四种切法在 MoE 里的适用性 :

(1) 纯 DP : 每卡都装全 MoE。不适用——单卡装不下。

(2) TP 切 expert GEMM : 把 $W_{\text{up}} \in \mathbb{R}^{H,I}$ 沿 I 维切。可行 , 但 E 个 expert 都被切 TP 份——每卡显存 = $\frac{E}{\text{TP}}$ expert 权重。等价于每卡还是有 $\frac{E}{\text{TP}}$ 个“半专家”, 扩展性有限。

(3) EP 切 expert : E 个专家分布在 EP 张卡上 , 每卡持有 $\frac{E}{\text{EP}}$ 个专家。这是最 natural 的切法——模型稀疏性 = 通信稀疏性。

(4) PP 切 layer : 一样切分 , 与 MoE 内部结构无关。

EP 是 MoE 训练的核心 , 其他三种是配套。这一章的 80% 内容是 EP。

Wall 1 · Memory Wall : 显存怎么省

单纯用 EP 切 expert 权重就能省 , 但还不够。DeepSeek-V3 tech report Table 3 给出的显存分解 (BF16、256 GPU、PP4×VPP4×EP64) :

组成	GB/GPU	占比
Weight + Grad	36.4	18%
Optimizer state	32.1	16%
Activation	131.0	66%
Total	199.5	100%

表 8 DeepSeek-V3 671B 显存分解。三大头里 activation 占 2/3——这决定了 memory wall 的主要武器是 激活优化 (recomp + offload + FP8) , 而不是 weight 分片。

Megatron-Core 和 DeepSeek 各自的 memory-wall 优化菜单 :

为节约横向空间 , 我们把“Megatron flag”单独列在下方一段 , 表里只保留机制与收益 :

优化	机制	DS-V3 / 参考收益
Memory-Efficient Permutation	Router 权重 $\tilde{w}$ 乘到 activation 上再进 FC2，数学等价（无 bias 时），省掉为 router backward 保存的 per-expert 输出	激活 26.3 GB → 换来更多 recompute 空间
Fine-grained Recomputation	模块级 output-discarding checkpoint，只重算 cheap 子模块（MLA up-proj、SwiGLU、LN），不整层 recompute（否则重触发 a2a）	MLA up-proj 30.4 + SwiGLU 3.8 + LN 8.2 = 42.4 GB ，额外算力 < 5%
Fine-grained Activation Offloading	模块级 D2H/H2D 加独立 CUDA stream，与 1F1B 的 a2a 兼容 overlap	mem -10.7% ，吞吐 -1.6% ；换 mapping 后 Qwen3-235B 反而 +15% 吞吐
Precision-aware Optimizer	Adam moment 存 BF16/FP8，update 时 fused kernel 里 upcast 到 FP32	Optimizer state 50%
Optimizer CPU Offload	forward/backward 时 optim state 在 CPU，step 前拉回	15-20 GB ，iter +0.1-0.2 s
FSDP + EP (Dual DeviceMesh)	Dense 层 FSDP over DP；Expert 层 FSDP over EDP；AllGather/RS 限定在小 EDP 组内，非-uniform sharding + NCCL UBR 零拷贝	Llama3 405B FSDP comm 10%↓
FP8/FP4 Activation	线性层输入按 FP8 存 checkpoint，激活占用 50%↓	激活 16 GB （DS-V3 activation 12%）
Distributed Optimizer (ZeRO-1)	Adam state 沿 DP 分片，通常与 FSDP 二选一	—
Capacity-based Drop	超 capacity 的 token 走 residual 跳过 FFN，activation shape 静态化便于 CUDA graph	训练早期防 OOM

表 9 Memory Wall 常用武器一览。DS-V3 收益数字来自 Megatron-Core Tech Report Table 3-5。

对应的 Megatron-Core CLI flag：

优化	Megatron flag
Memory-Efficient Permutation	默认 (v0.13+) + <code>--moe-permute-fusion</code>
Fine-grained Recomputation	<code>--recompute-granularity selective</code> <code>--recompute-modules mla_up_proj layernorm moe_act moe mlp core_attn</code>
Fine-grained Activation Offloading	<code>--fine-grained-activation-offloading</code> <code>--offload-modules expert_fcl moe_act ...</code>
Precision-aware Optimizer	<code>--use-precision-aware-optimizer</code> <code>--exp-avg-dtype bf16 --exp-avg-sq-dtype bf16</code>
Optimizer CPU Offload	<code>--optimizer-cpu-offload</code>
FSDP + EP (Dual DeviceMesh)	Megatron-FSDP 集成 (v0.15+), 无独立 flag, 配合 <code>--use-distributed-optimizer</code>
FP8/FP4 Activation	<code>--fp8-format e4m3 --fp8-recipe blockwise</code>
Distributed Optimizer (ZeRO-1)	<code>--use-distributed-optimizer</code>
Capacity-based Drop	<code>--moe-expert-capacity-factor 1.0 --moe-pad-expert-input-to-capacity</code>

表 10 Memory Wall 优化对应的 Megatron-Core 命令行 flag。

Key insight. 切 memory 的顺序有讲究：先切 **optimizer state** (ZeRO-1 / precision-aware) → 再切 **activation** (recompute + offload + FP8) → 最后切 **weight** (EP + TP)。因为 optimizer state 与 weight 无关但占用最大，切它不影响计算通信；activation 决定 batch size 上限；weight 切碎会引入通信。反过来做通常会陷入“切了一堆通信却省不了多少内存”。

EP 数据流：从单机到 EP

单机 (EP=1) 回顾

第 5 章的 MoE forward：

```

X: (N, H) → router → indices, weights
                ↓
            dispatch → each expert 本地计算 → combine
                ↓
            out: (N, H)

```

所有 expert 都在本卡，dispatch 是本卡 memory scatter。

EP=8 时会发生什么

假设 world_size=8 (全 EP, 无 DP/TP/PP), $E=8$, 每卡持有 1 个专家。每卡有本地 tokens X_{local} : (N_{local}, H) 。

Router 计算：每卡独立算自己 tokens 的路由。因为 gate weight W_g 是完整的 (W_g 很小, 全 rank 都存一份), 每卡都能算完整 topk。

但 **dispatch** 遇到问题：本卡上的 token 可能被路由到 其他卡上的 **expert**。这就需要跨卡把 **token** 送到目标 **expert** 所在的卡。

这就是 all-to-all。

All-to-All 通信模式

All-to-all = 每个 rank 向每个其他 rank 各发一份数据。语义：

```
before:                after all-to-all:
rank 0: [a0, b0, c0, d0]    rank 0: [a0, a1, a2, a3]
rank 1: [a1, b1, c1, d1]    rank 1: [b0, b1, b2, b3]
rank 2: [a2, b2, c2, d2]    rank 2: [c0, c1, c2, c3]
rank 3: [a3, b3, c3, d3]    rank 3: [d0, d1, d2, d3]
```

即 rank i 发送第 j 段给 rank j ，接收所有 rank 发给自己的段。

MoE 的用法：**rank i** 的第 j 段 = 应该由 **expert j** (在 **rank j**) 处理的 **tokens**。

EP forward 完整流程

```
每卡有 X_local: (N_local, H)
↓
router 本地 → expert_indices (N_local, K), expert_weights (N_local, K)
↓
按 expert 归属 rank 排序 → packed_input (N_local × K, H)
    同时构造 send_counts (EP,) – 发给每个 rank 多少 token
↓
[all_to_all_1] packed_input, send_counts
↓
本卡收到 recv_input (M_local, H)
    其中 M_local = 应由本卡上的 expert 处理的所有 token 总数
↓
本地 grouped GEMM: recv_input → local experts → recv_output (M_local, H)
↓
[all_to_all_2] recv_output, recv_counts (是发送时的 send_counts 的转置)
↓
本卡收回 (N_local × K, H)
↓
乘以 expert_weights + unpermute + reduce → out (N_local, H)
```

完整 PyTorch 实现 (教学版)

用 `torch.distributed.all_to_all_single` 写出来的极简 EP MoE，端到端能跑。跳过了 fused kernel、只保证语义正确：

```
import torch, torch.nn as nn, torch.nn.functional as F
import torch.distributed as dist

class EPMoE(nn.Module):
    """
    Expert-Parallel MoE (教学版).
    - world_size = EP, 无 DP/TP;
    - 每 rank 持有 num_experts // EP 个 expert 的 W_up / W_down.
```

```

"""
def __init__(self, hidden: int, inter: int, num_experts: int,
              top_k: int, ep_group=None):
    super().__init__()
    self.ep_group = ep_group or dist.group.WORLD
    self.ep_size = dist.get_world_size(self.ep_group)
    self.ep_rank = dist.get_rank(self.ep_group)
    assert num_experts % self.ep_size == 0
    self.E, self.K = num_experts, top_k
    self.E_local = num_experts // self.ep_size          # 本卡专家数

    # Router 每卡 replicate
    self.gate = nn.Linear(hidden, num_experts, bias=False)
    # 本地 expert 权重
    self.W_up = nn.Parameter(torch.empty(self.E_local, hidden, inter))
    self.W_down = nn.Parameter(torch.empty(self.E_local, inter, hidden))
    nn.init.kaiming_uniform_(self.W_up)
    nn.init.kaiming_uniform_(self.W_down)

def forward(self, x):
    N, H = x.shape
    # -- 1. router (每卡独立) --
    logits = self.gate(x)                                # (N, E)
    probs = F.softmax(logits, dim=-1, dtype=torch.float32).to(x.dtype)
    weights, indices = torch.topk(probs, self.K, dim=-1)  # (N, K), (N, K)
    weights = weights / weights.sum(-1, keepdim=True)

    # -- 2. 展平 + 按目标 rank 排序 --
    flat_expert = indices.flatten()                       # (N*K,)
    flat_weight = weights.flatten()                      # (N*K,)
    flat_token = torch.arange(N, device=x.device).repeat_interleave(self.K)

    target_rank = flat_expert // self.E_local             # 每 slot 该发往哪个
rank
    perm = target_rank.argsort(stable=True)
    send_tokens = x[flat_token[perm]]                   # (N*K, H) 打包好待
发
    send_weights = flat_weight[perm]                    # (N*K,)
    send_expert = flat_expert[perm]                     # (N*K,) 目标
expert (全局 id)

    send_counts = torch.bincount(target_rank, minlength=self.ep_size)
    # 3.a 先 all-to-all 交换 counts, 拿到 recv_counts
    recv_counts = torch.empty_like(send_counts)
    dist.all_to_all_single(recv_counts, send_counts, group=self.ep_group)
    M_local = int(recv_counts.sum().item())

    # -- 3.b all-to-all 数据: (dispatch) --
    recv_tokens = torch.empty((M_local, H),
                              dtype=x.dtype, device=x.device)
    dist.all_to_all_single(
        recv_tokens, send_tokens,

```

```

        output_split_sizes=recv_counts.tolist(),
        input_split_sizes =send_counts.tolist(),
        group=self.ep_group,
    )
    # 相同 pattern 也发 expert id 和 weight, 用于后面的 grouped GEMM
    recv_expert = torch.empty(M_local, dtype=send_expert.dtype,
device=x.device)
    recv_weight = torch.empty(M_local, dtype=send_weights.dtype,
device=x.device)
    dist.all_to_all_single(recv_expert, send_expert,
                           output_split_sizes=recv_counts.tolist(),
                           input_split_sizes =send_counts.tolist(),
                           group=self.ep_group)
    dist.all_to_all_single(recv_weight, send_weights,
                           output_split_sizes=recv_counts.tolist(),
                           input_split_sizes =send_counts.tolist(),
                           group=self.ep_group)

    # -- 4. 本地 expert 计算 (grouped GEMM) --
    # 把全局 expert id 转本地
    local_expert = recv_expert - self.ep_rank * self.E_local #  $\in [0, E_{\text{local}})$ 
    # 按 local expert 再排一次序 (方便 grouped GEMM)
    perm2 = local_expert.argsort(stable=True)
    packed_in = recv_tokens[perm2]
    group_sizes = torch.bincount(local_expert, minlength=self.E_local)
    # torch._grouped_mm 需要 PyTorch 2.4+; 无则退化到 for-loop
    h = torch._grouped_mm(packed_in, self.W_up, group_sizes)
    h = F.silu(h) # 演示用 SiLU
    y = torch._grouped_mm(h, self.W_down, group_sizes)
    # unpermute 回 recv 顺序
    expert_out = torch.empty_like(y)
    expert_out[perm2] = y
    # 乘 gate weight
    expert_out = expert_out * recv_weight.unsqueeze(-1)

    # -- 5. all-to-all 送回 (combine) --
    # send_counts / recv_counts 的角色互换
    back_tokens = torch.empty_like(send_tokens)
    dist.all_to_all_single(
        back_tokens, expert_out,
        output_split_sizes=send_counts.tolist(),
        input_split_sizes =recv_counts.tolist(),
        group=self.ep_group,
    )

    # -- 6. inverse permute + reduce over K --
    # back_tokens 里的顺序仍是 perm 后的; 恢复到 flat (N*K,) 原序
    flat_out = torch.empty_like(back_tokens)
    flat_out[perm] = back_tokens
    out = flat_out.view(N, self.K, H).sum(dim=1) # (N, H)
    return out, logits

```

跑起来 (单机 4 卡模拟 EP=4):

```
torchrun --nproc_per_node=4 test_ep_moe.py
```

生产级实现的差别

上面的教学版有几个明显的性能问题，生产实现都会修：

1. **4 次独立 all-to-all** (1 次 counts + 1 次 tokens + 1 次 expert ids + 1 次 weights) —— 生产实现把 expert id / weight 编码进 token 首几个字节，只做 1 次 counts + 1 次数据 all-to-all
2. **permute / unpermute** 是独立 **kernel** —— fuse 进 grouped GEMM prologue (第 7 章 § “v3 Fused Permute”)
3. **all-to-all** 是同步的 —— 生产用 `dist.all_to_all_single(..., async_op=True)` 拿 handle，让下游 attention / next-layer forward overlap
4. **bincount** 是 **GPU-CPU sync** —— 用固定 shape + shape hints 避免 (Tutel 的做法)
5. **torch autograd** 会为 **all_to_all** 自动生成反向，但需要用 Megatron 的 `_AllToAll.apply` 或 DeepEP 的封装才能拿到 async backward

参考实现：Megatron-LM `megatron/core/transformer/moe/token_dispatcher.py`，DeepEP (DeepSeek 开源) `deep_ep/buffer.py`。

用图表达：

All-to-all (dispatch): tokens 按目标 expert 分发



通信量分析

设：

- N_{local} : 每卡 token 数
- K : top-K
- H : hidden dim
- EP: EP world size

Forward 的 all-to-all：每卡向其他 rank 送 $\sim N_{\text{local}} \frac{K}{\text{EP}} \times H$ 数据 (假设路由均衡)，总 send/recv volume：

$$\text{vol}_{\text{a2a}} \approx N_{\text{local}} K H \times \text{bytes}_{\text{dtype}}$$

对 $N_{\text{local}} = 4096, K = 2, H = 4096, \text{bf16}$ ：

$$\text{vol}_{\text{a2a}} = 4096 \times 2 \times 4096 \times 2 = 128 \text{ MB}$$

单层 MoE forward 有 **2 次 all-to-all** (dispatch + combine)，backward 有 **2 次** (反向送 grad + 反向送 activation grad)，总 **4 次 $\times 128 \text{ MB} = 512 \text{ MB}$ per layer per step**。

一个 32 层的 MoE 训练 : $32 \times 512 \text{ MB} = 16 \text{ GB}$ per step 的 all-to-all 通信量。在 3.2 Tbps NVLink (400 GB/s bidi) 下需要 $\sim 40 \text{ ms}$ 通信 ; 在 400 Gbps IB (50 GB/s) 下需要 $\sim 320 \text{ ms}$ 。这就是为什么大规模 MoE 训练通信/计算 **overlap** 极其重要。

Key insight. 一个反直觉 : MoE 的通信量不依赖于 E (专家总数), 只依赖于 NKH 。加多专家不增加通信量——每 token 还是走 K 个专家。这也是 DeepSeek-V3 敢用 256 experts 的原因 : 通信开销与 8 experts 相同。

Wall 2 · Communication Wall : a2a 怎么打

MoE 通信优化在过去两年出了三代方案, 一代比一代激进。理解这条演进对定位自己的问题很关键 :

1. **NCCL alltoall** (baseline): `torch.distributed.all_to_all_single` ; 一对 rank 一个 IB pair, 每对都占带宽, 跨节点极慢。
2. **Hierarchical a2a** (DeepSpeed-MoE、Tutel): intra-node NVLink 聚合 $\rightarrow$ 单次 inter-node IB $\rightarrow$ intra-node 散发。
3. **IB + NVLink 并行调度 + warp specialization** (DeepEP、HybridEP): 一次 dispatch 让 IB 和 NVLink 同时工作, 且 combine 里融合 reduction。

方案 1 : Hierarchical a2a (DeepSpeed-MoE 风格)

三步 : intra-node $\rightarrow$ inter-node $\rightarrow$ intra-node。

```
def hierarchical_a2a(x, send_counts, ep_group):
    node_size = 8 # GPUs per node
    n_nodes = ep_group.size() // node_size
    x_by_node = rearrange_by_target_node(x, send_counts)
    x1 = intra_a2a(x_by_node, node_size) # NVLink 400 GB/s
    x2 = inter_a2a(x1, n_nodes) # IB 50 GB/s
    return intra_a2a(x2, node_size) # NVLink 再散发
```

IB 带宽消耗从 $O(N^2KH)$ 降到 $O(NKH)$ 。100+ 节点上 wall-clock 差 10 $\times$ 。缺点 : 三段串行, IB 与 NVLink 各自独占带宽的时候另一方闲着。

方案 2 : DeepEP — IB + NVLink 并行 + warp specialization

DeepSeek 开源³ 的 a2a 库, 是 DeepSeek-V3 训练用的原型。两个关键改进 :

(a) IB / NVLink 并行调度 : 目标节点内同 **in-node index** 的 GPU 作为“落地点”——token 先 IB 送到 target node 的对应 index GPU, 那边同时用 NVLink 二次分发 ; IB 和 NVLink 在时间上完全重叠而不是串行。

(b) Warp specialization + auto-tuned chunk size : 20 SM 划分为 10 个 comm channel, dispatch 三阶段 (IB send / IB $\rightarrow$ NVLink forward / NVLink recv) 由不同 warp 负责 ; combine 对称, NVLink $\rightarrow$ IB 阶段在 **kernel** 内完成 **reduction**。

配合 **Node-Limited Routing** (每 token 最多发往 $M = 4$ 个节点), 一次 dispatch 的 IB fan-out 有上界, 通信量可预测。

API 长这样 (V2):

³<https://github.com/deepseek-ai/DeepEP>

```
import deep_ep
buf = deep_ep.Buffer(group, num_nvl_bytes=..., num_rdma_bytes=...)

# forward dispatch
recv_x, handle = buf.dispatch(
    x, topk_idx=..., topk_weights=...,
    num_experts=E, num_sms=6,          # V2 只要 4-6 SM
    async_finish=True,                 # 拿到 handle 之后可以做别的
)
# backward dispatch 就是 combine
out = buf.combine(y, handle=handle, topk_weights=..., num_sms=6)
```

性能数字 (H800 + CX7 , 8K tokens/batch, $H = 7168$, top-8, FP8 dispatch / BF16 combine):

模式	EP	Dispatch BW	Combine BW	SM 数
Intranode (NVLink)	8	153 GB/s	158 GB/s	24 (V1)
Internode (RDMA V1)	64	51 GB/s	50 GB/s	24
Internode (RDMA V2)	32	90 GB/s	81 GB/s	12
Internode (RDMA V2)	64	61 GB/s	61 GB/s	6

表 11 DeepEP dispatch/combine 有效带宽。V2 相比 V1 用 **4×** 更少的 **SM** 达到同等带宽——省下来的 SM 归还给 grouped GEMM。

代价：DeepEP 占 **20 SM/GPU** (V1) 或 4-6 SM (V2), 带来大约 **20% GEMM** 效率损失 (V1), V2 才让 fine-grained MoE 真正可行。

方案 3：HybridEP (NVIDIA GB200/MNVL) — TMA + IBGDA

GB200 NVL72 场景下节点内 NVLink 域扩大到 72 卡, NVIDIA 提出 HybridEP : **TMA (Tensor Memory Accelerator) + IBGDA**, intra-node NVLink 与 inter-node RDMA fusion ; combine 在 kernel 内完成 reduction。

Megatron

flag: `--moe-token-dispatcher-type flex --moe-flex-dispatcher-backend hybridep`

EP	Dispatch (HybridEP / alltoall)	ratio	GB200 / H100
8	391 / 735 μ s	1.88×	GB200
64	675 / 930 μ s	1.38×	GB200
8	661 / 1265 μ s	1.91×	H100
64	4626 / 9164 μ s	1.98×	H100

表 12 HybridEP vs NCCL naive alltoall , 通信 kernel 时间 (Megatron-Core Tech Report Table 7)。H100 上 EP=64 时接近 2× 加速 ; GB200 上因 NVLink 域扩大 , naive 就已经很好 , 收益变小。

对 256 experts、top-8 的 DeepSeek-V3 类模型 , HybridEP 相比 DeepEP 端到端 **+14%** (NVIDIA blog 数据)

Megatron-Core 的 dispatcher 类型选择

Megatron-Core 把上述方案封装成 `--moe-token-dispatcher-type` 的 4 个选项：

Dispatcher	适用
allgather	<code>--moe-token-dispatcher-type allgather</code> (默认) TP、 小 EP、 大 top- K
alltoall	<code>--moe-token-dispatcher-type alltoall</code> 准 EP > 1
flex + DeepEP	<code>--moe-token-dispatcher-type flex --moe-flex-dispatcher-backend deep</code> 跨 节 点、 fine- grained MoE
flex + HybridEP	<code>--moe-token-dispatcher-type flex --moe-flex-dispatcher-backend hybridep</code> MNNVL

表 13 Megatron-Core MoE token dispatcher 类型。旧的 `alltoall_seq` 从 v0.13 起 deprecated，请忽略。

关键 tricks 小结

1. **Node-Limited Routing** : 每 token 最多送 M 个 node。DeepSeek $M = 4$ ，配合 8 卡/节点，等效扩展到 13 个可选 **expert** (不是全 256)，通信量恒定。
2. **FP8 dispatch** : activation 在 dispatch 前量化到 FP8，通信量 **-50%**，combine 仍用 BF16 保精度。
3. **Warp specialization** : 一个 kernel 里不同 warp 各司其职 (send / forward / recv)，避免各自 launch。
4. **SM carve-out** : 早期方案 (DeepEP V1) 占用 20 SM，DeepEP V2 降到 4-6 SM 才让 GEMM 有喘息空间。

EP × TP × DP 组合

生产训练里 EP 单独用是奢侈。真实配置往往 EP × TP × DP：

配置示例: Mixtral 8×7B on 64 × H100

```
world_size = 64
TP = 2      # 切 attention 和 expert 内 GEMM
```



```

EP = 8      # 8 个 expert 分布 8 卡
DP = 4      # 4 份数据并行 replica
PP = 1

placement (illustrative):
  TP-group: [rank i, i+1]          for i in 0, 2, 4, ..., 62
  EP-group: 每 EP 组 8 卡各持 1 expert
  DP-group: 跨 EP 组的相同 slot 位置

```

非 **expert** 参数 (attention, gate, LN) : TP 内切 + DP 全同步。梯度用标准 AllReduce。

Expert 参数 : EP 组间不同 **expert** (无需 sync); DP 组间同一 **expert** 的多份 **replica** (AllReduce)。

Forward 通信序列 (一个 MoE 层):

```

1. Attention:
  - QKV Linear (TP col-parallel)
  - Attention compute (local)
  - Output Linear (TP row-parallel) + AllReduce(TP)

2. Gate:
  - Linear(H, E) (replicated, small)
  - softmax + topk (local)

3. Dispatch:
  - all-to-all across EP-group
  - (packed input arrives)

4. Expert:
  - grouped_gemm(x, W_up) - W_up 沿 I 维 TP-partitioned
  - activation
  - grouped_gemm(h, W_down) - W_down 沿 I 维 TP-partitioned
  - AllReduce(TP) 因 W_down 是 row-parallel

5. Combine:
  - all-to-all across EP-group
  - weight + unpermute

```

每 **MoE** 层通信次数 : 2 次 AllReduce(TP) + 2 次 all-to-all(EP)。加上 backward 就是 4+4 = 8 次集合通信/层。

Overlap: 隐藏通信

MoE 训练的核心工程优化 : 让通信和计算重叠。有几个层次 :

Level 1: All-to-all 与 expert compute overlap

单层内可做的 :

- `all_to_all_1` 开始 → 立刻开始处理已到达的 tokens (chunked all-to-all)
- 或 : 本层 `all_to_all_2` (combine 送回) 与下层 attention overlap , 两条 CUDA stream 并行。

```

# 一层 MoE 内, 2 stream overlap
comm_stream = torch.cuda.Stream() # 只做通信
comp_stream = torch.cuda.current_stream()

# 1) dispatch a2a 起在 comm_stream
with torch.cuda.stream(comm_stream):
    recv_tokens = torch.empty(...)
    a2a_handle = dist.all_to_all_single(
        recv_tokens, send_tokens,
        output_split_sizes=recv_counts.tolist(),
        input_split_sizes=send_counts.tolist(),
        async_op=True,
    )
# 2) 等 dispatch 完成 (但让 comp_stream 通过 event 等待, 不阻塞 host)
a2a_handle.wait()
event_dispatch = torch.cuda.Event()
event_dispatch.record(comm_stream)
comp_stream.wait_event(event_dispatch)

# 3) expert compute 在 comp_stream 上跑
with torch.cuda.stream(comp_stream):
    y = grouped_gemm(recv_tokens, W_up)
    y = F.silu(y)
    y = grouped_gemm(y, W_down)

# 4) combine a2a 起在 comm_stream, 与下一层的 attention 并行
event_compute = torch.cuda.Event()
event_compute.record(comp_stream)
with torch.cuda.stream(comm_stream):
    comm_stream.wait_event(event_compute)
    back_handle = dist.all_to_all_single(
        back_tokens, y,
        output_split_sizes=send_counts.tolist(),
        input_split_sizes=recv_counts.tolist(),
        async_op=True,
    )
# 5) 下一层的 attention 可以在 comp_stream 上*立刻*开始
# 只有需要 back_tokens 的地方才 back_handle.wait()

```

关键点是 CUDA event 而不是 stream.wait_stream —— event 只等特定操作完成, 让 stream 之间的依赖尽可能松。

Level 2: 跨层 pipeline

$layer_l$ 的 `all_to_all_2` 完成的同时, $layer_{l+1}$ 的 attention 已经开始 (因为 attention 与 MoE 独立)。这需要 async grad accumulation + micro-batch pipelining。

Level 3: DeepSeek-V3 DualPipe

DeepSeek-V3 (2024) 展示了最激进的 overlap 策略——把每 PP chunk 拆成 4 个细粒度阶段：

1. Attention
2. All-to-all dispatch

3. MLP (MoE expert 计算)

4. All-to-all combine

再借鉴 ZeroBubble 把 backward 拆成 **B (input grad)** 和 **W (weight grad)** , 然后 双向 注入 micro-batch (pipeline 两端同时进)。每个 forward chunk 与配对的 backward chunk 在同一时刻用不同 SM 跑——通信 kernel 分走 20 SM 做 IB/NVLink , 剩下 112 SM 做 GEMM , 通信被计算完全隐藏。

Bubble 对比 (Tech Report Table 2 ; F、B、W 分别是 forward / input-grad / weight-grad 时间 , F&B 是 forward+backward 重叠后的时间):

Method	Bubble	每设备参数副本
1F1B	$(PP-1)(F+B)$	1×
ZB1P	$(PP-1)(F+B-2W)$	1×
DualPipe	$(PP/2-1)(F\&B+B-3W)$	2×

表 14 pipeline bubble 系数。 $PP=16$ 时 1F1B 是 15 个 stage , DualPipe 是 7 个——但代价是 **2×** 参数副本 (bidirectional 每端各存一份) , activation 也从 PP 增到 $PP+1$ 。

Wall-clock 收益 :DeepSeek 声称“a2a 与 PP 通信近乎零开销” , 但没给单一 %。参考 profile-data 里的 Chrome trace , $\text{comp} : \text{comm} \approx 1 : 1$ 的 setup 下 DualPipe 让 comm 完全隐藏。

工程代价 : 需要 2× 权重、需要自己实现 pipeline schedule、需要与 DeepEP 深度耦合。开源实现在 <https://github.com/deepseek-ai/DualPipe>。

Level 4: Megatron-Core 1F1B FWD-BWD Merged

DualPipe 的 2× 参数副本对内存墙不友好。Megatron-Core 提出一个折中 : 不改 **pipeline schedule** (仍是 **1F1B**) , 但在同一个 **iteration** 里把相邻 **micro-batch** 的 **FWD** 和 **BWD** 合并 , 两条 CUDA stream 交错——一条 compute、一条 comm。

关键 flag :

```
--overlap-moe-expert-parallel-comm # 打开 FWD/BWD merged 调度
--delay-wgrad-compute               # 把 backward 拆成 B (data grad) + W (weight grad)
                                     # 打破 B/dispatch → B/mlp 的依赖
```

需要环境变量 `CUDA_DEVICE_MAX_CONNECTIONS>1` 允许多 stream。

收益 : EP 通信占比 30-40% → < **5%** , overlap ratio 达 **93%** , 不需要 2× 权重。

Overlap 策略选择

策略	额外内存	a2a 隐藏率	代码复杂度	何时用
naive	1×	0%	低	< 32 卡 debug
Level 1 (2-stream)	1×	50%	低	64-256 卡快速上手
Level 2 (跨层 pipeline)	1×	70%	中	512+ 卡、attention 有余量
DualPipe	2×	95%	高	1000+ 卡且能吃下 2× 权重
1F1B FWD-BWD merged	1×	93%	中	Megatron-Core 用户 , 推荐

表 15 四代 overlap 策略选择。绝大多数场景 Level 4 已经够——DualPipe 的 2× 权重成本很少划算。

Key insight. Overlap 的收益取决于通信与计算的比例。comm : comp = 1 : 1 (DS-V3 跨节点) 时 overlap 能省近 50% wall-clock ; comm : comp = 1 : 5 (Mixtral 8×7B 单机) 时 overlap 只值几个 %。先 profile 再上 overlap , 别为了 overlap 而 overlap。

Wall 3 · Compute Efficiency Wall : kernel 碎片化

第 7 章讲的 grouped GEMM / permute fusion 都是这堵墙的武器。分布式场景又加了两个新问题 :

1. **Dropless MoE 的 D2H sync** : `torch.bincount(target_rank)` 是 GPU 上跑但要返回 CPU 用来做 `all_to_all_single` 的 `split_sizes`——一次 D2H sync 每层每 step 都发生 , 破坏 CUDA graph capture。
2. 动态 **shape** : 每卡收到多少 token (M_{local}) 每步不同 , kernel launch 时的 shape 都要重新解析、autotune cache miss。

Sync-Free MoE + Device-initiated Grouped GEMM

核心 : 把 shape 信息留在 GPU 上 , 让 grouped GEMM kernel 从 device memory 读 group sizes , 不做 **D2H sync**。配合预分配 upper-bound buffer (按 capacity 上限) , 整层可以进 CUDA graph。

```
# 常规做法 (有 D2H sync)
send_counts_gpu = torch.bincount(target_rank, minlength=EP)
send_counts_cpu = send_counts_gpu.tolist()          # <-- D2H sync!
dist.all_to_all_single(recv, send,
                        output_split_sizes=recv_counts_cpu,
                        input_split_sizes=send_counts_cpu, ...)
```

```
# Sync-free (Megatron-Core / HybridEP)
# send/recv 用固定 upper-bound buffer, counts 用 device tensor 传给 fused kernel
recv, counts_gpu = hybridep.dispatch(x, target_rank, capacity_per_expert=C)
h = device_grouped_gemm(recv, W_up, counts_gpu)     # kernel 内 gather counts
```

需要与 `--moe-expert-capacity-factor` + `--moe-pad-expert-input-to-capacity` 一起用 (换回 capacity + drop 的开销)

CUDA Graph (Partial vs Full)

Dropless MoE 只有 attention + router + moe_preprocess 是静态的 , 可以部分 graph :

```
--cuda-graph-impl transformer_engine
--cuda-graph-modules attn moe_router moe_preprocess
```

DS-V3 GB200 上报告 +10% 端到端 , 代价额外 7 GB 内存。

Full graph 只有在 capacity + pad 之后 (shape 静态) 才能做 , Sync-Free + HybridEP 是解锁 full graph 的关键组合。

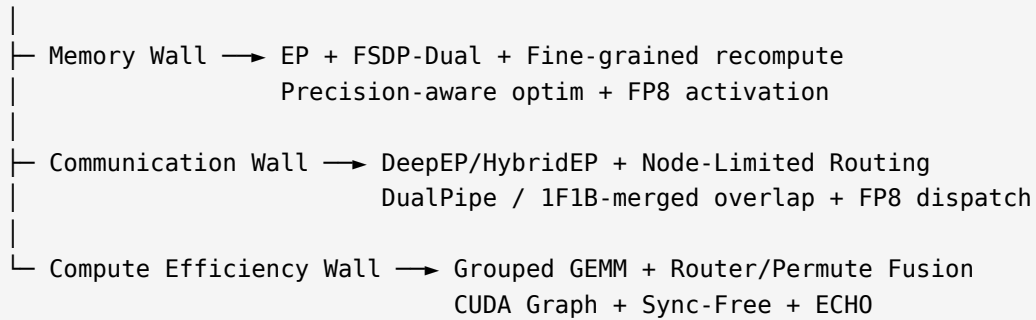
ECHO : Elastic Cloning for Hot Experts

Megatron-Core Tech Report 提出的负载均衡新方法。观察 : 即使有 aux-loss-free , 某些 batch 里仍会有 1-2 个“过热” expert , 导致同 EP 组里其他 rank 干等。ECHO 动态把热 expert **clone** 到空闲 **rank** , 用 bin-packing 最小化 clone 数 ; backward 时把梯度 reduce 回 home rank。

对 fine-grained MoE 尤其有用——256 experts 场景下 heat map 不均都是常态。

三墙武器统一映射

Parameter-Compute Mismatch ($E \gg K$)



三墙互相暴露：解决内存 → 引入 EP → 暴露通信 → overlap
→ 需更大 batch → FP8 → kernel 碎片化 → CUDA Graph

NCCL 调优

MoE all-to-all 通信性能极大依赖 NCCL 参数。生产建议：

```
export NCCL_ALGO=Tree           # 或 Ring, 根据拓扑测试
export NCCL_PROTO=Simple        # 大 message 稳定
export NCCL_MIN_NCHANNELS=8    # 增加并发 channel
export NCCL_NTHREADS=512       # 每 channel 线程数
export NCCL_BUFFSIZE=8388608  # 8MB buffer

# IB tuning
export NCCL_IB_HCA=mlx5_0,mlx5_1 # 双网卡
export NCCL_IB_GID_INDEX=3
export NCCL_IB_TIMEOUT=22

# Debug
export NCCL_DEBUG=INFO
```

具体值必须实测——不同集群拓扑差别巨大。NVIDIA NCCL Tests 可以基准。

主要框架

先看 kernel / a2a 层的框架谱系：

框架	EP	TP+EP	Grouped GEMM	特色
DeepSpeed-MoE	✓	部分	✓	早期成熟，与 ZeRO 集成
Megatron-LM MoE	✓	✓完整	✓	NVIDIA 官方，工业标准
Tutel	✓	✓	✓	微软，快速 a2a、动态 capacity
Megablocks	✓	部分	替代为 sparse	无 drop，DBRX 用
DeepEP	✓	✓	靠 grouped GEMM 库	DeepSeek 官方 a2a 库，V2 只吃 4-6 SM
HybridEP	✓	✓	✓	NVIDIA GB200 NVL72 / MNNVL

表 16 MoE 框架谱系。DeepEP / HybridEP 是 **a2a-only** 库，需要嵌入 Megatron-Core / 自研训练框架里用。

新项目的现实选择：

1. **H100** 单机 (< 64 卡) : Megatron-Core MoE + `alltoall` dispatcher，够用
2. **H100/H800** 跨节点 (> 128 卡) : Megatron-Core + `flex` + `deepep` backend
3. **GB200 NVL72** : Megatron-Core + `flex` + `hybridep`
4. 完全自研：参考 DeepSeek-V3 论文 + DeepEP 源码

生产系统对比：**Megatron-Core / MegaScale-MoE / DeepSeek-V3**

三个当前最完整的公开系统。选一个当模板抄比自己从头搭快得多：

维度	Megatron-Core MoE	MegaScale-MoE (ByteDance)	DeepSeek-V3
并行	任意 TP/EP/PP/DP + Parallel Folding	SP (attention) + EP (expert) ; MoE 层限 node 内	PP16 × EP64 × ZeRO-1 , 无 TP
a2a	DeepEP / HybridEP 可插	自研 tile-level fusion + swizzling	DeepEP + warp specialization
Pipeline	1F1B FWD-BWD merged (--overlap-moe- expert-parallel-comm)	MegaScale 继承的 DP/ PP overlap	DualPipe (bidirectional)
精度	BF16 / FP8 blockwise / MXFP8	BF16 + FP8 optional	FP8 E4M3 训练 (Fprop/Dgrad/Wgrad)
路由	aux-loss / seq_aux / global_aux / aux-loss-free	aux loss + token drop	aux-loss-free + seq-wise 小 α + Node-Limited Routing
Grouped GEMM	TE GroupedLinear / CUTLASS grouped	自研 GroupedGEMM + fuse gather/scatter	DeepGEMM (FP8)
规模验证	Mixtral 8×22B 49.3% MFU , Qwen2-57B 39.0% MFU (1024 H100)	352B / 1440 H800 / 1.41M tok/s, 1.88× vs Megatron-LM	671B / 2048 H800 / 180K GPU-hrs per T tokens
开源	✓ (NVIDIA/Megatron-LM)	✗ 论文 only	部分 : DeepEP / DualPipe / DeepGEMM 开源 , 训练主脚本未开

表 17 三个生产 MoE 训练系统对比。Megatron-Core 优势在特性齐全 + 官方支持 ; MegaScale-MoE 优势在 intra-op tile fusion ; DeepSeek-V3 优势在极致 overlap + FP8 + 大规模验证。

各系统的关键收益 (公开数字)

MegaScale-MoE (arXiv:2505.11432 , EuroSys '26) 核心贡献:

1. 用 **SP + EP** 替代 Megatron 的 TP + TP : SP+EP vs TP+TP **MFU +14.9-32.9%** (6 个 MoE 模型)
2. **Intra-op fusion** : Attention 里 A2A+GEMM、GEMM+A2A ; Expert 里 AG+scatter+GroupedGEMM、GroupedGEMM+gather+RS ——把 tile 级依赖打散 , 通信+算子时间缩短 **1.2-4.7×**
3. **Adaptive a2a ↔ AG+RS** : top-K > 6 时切换到 AG+RS 更省
4. **Selective Activation Rematerialization** : 显存 -45 57% , 吞吐几乎不变

DeepSeek-V3 训练 **infra** (arXiv:2412.19437 §3) 核心 :

1. **DualPipe** : bubble 系数从 $(PP-1)(F+B)$ 降到 $(PP/2-1)(F&B+B-3W)$
2. **DeepEP** : IB + NVLink 并行调度 + warp specialization , V2 只吃 4-6 SM

3. **FP8 E4M3** 全链路 :三路 GEMM (Fprop/Dgrad/Wgrad) 全用 FP8 ,val loss error < **0.25%** vs BF16
4. **Node-Limited Routing** ($M = 4$) : 通信量恒定 , 独立于总 expert 数
5. **aux-loss-free balancing** : $\gamma = 0.001$, 前 14.3T tokens 用 , 后 500B tokens 关掉

Megatron-Core MoE (arXiv:2603.07685) 核心 :

1. **Parallel Folding** : 打破 $EP \leq DP$ 约束 , Attention 与 MoE 用独立 process group
2. **Fine-grained recompute / offload** : 模块级 checkpoint , MLA up-proj / SwiGLU / LN 单独 recompute
3. **EP overlap** : `--overlap-moe-expert-parallel-comm` + `--delay-wgrad-compute` , 通信占比 30-40% $\rightarrow$ < 5%
4. **Sync-Free MoE + ECHO** : 让 dropless MoE 也能进 CUDA graph ; ECHO 处理热 expert bin-packing

完整训练配置样例

给两份可直接抄的配置——一份 Mixtral 8 $\times$ 7B(标准 MoE) , 一份 DeepSeek-V3 类 (fine-grained + FP8)。都用 Megatron-Core flag。

A. Mixtral 8 $\times$ 7B on 64 $\times$ H100

```
# 64 GPU, TP=1 EP=8 PP=4 DP=2, MBS=1 GBS=256 seq=4096
torchrun --nproc-per-node=8 --nnodes=8 \
  --node-rank=$NODE_RANK --master-addr=$MASTER pretrain_gpt.py \
  \
  # --- 模型 ---
  --num-layers 32 --hidden-size 4096 --ffn-hidden-size 14336 \
  --num-attention-heads 32 --seq-length 4096 --max-position-embeddings 32768 \
  --swiglu --normalization RMSNorm --position-embedding-type rope \
  \
  # --- MoE ---
  --num-experts 8 --moe-router-topk 2 \
  --moe-router-load-balancing-type aux_loss --moe-aux-loss-coeff 1e-2 \
  --moe-grouped-gemm --moe-permute-fusion --moe-router-fusion \
  --moe-token-dispatcher-type alltoall \
  \
  # --- 并行 ---
  --tensor-model-parallel-size 1 \
  --pipeline-model-parallel-size 4 \
  --expert-model-parallel-size 8 \
  --num-layers-per-virtual-pipeline-stage 8 \
  --sequence-parallel \
  --use-distributed-optimizer \
  \
  # --- Overlap ---
  --overlap-grad-reduce --overlap-param-gather \
  --overlap-moe-expert-parallel-comm --delay-wgrad-compute \
  \
  # --- 训练 ---
  --micro-batch-size 1 --global-batch-size 256 \
```



```
--bf16 --lr 3e-4 --min-lr 3e-5 --lr-decay-style cosine \  
--train-iters 100000
```

参考 throughput : Megatron-Core v0.9 官方 benchmark 报告约 **468 TFLOPS/GPU** (BF16, 64 GPU, `alltoall` dispatcher)。dense Mistral 7B 同 setup 是 **492 TFLOPS/GPU**——MoE 差 5% , 主要来自 a2a 未 100% overlap。

B. DeepSeek-V3 类 fine-grained MoE (FP8, 256 GPU+)

```
# 关键 flags, 省略常规参数  
  
# --- MoE: fine-grained + shared + aux-loss-free ---  
--num-experts 256 --moe-shared-expert-intermediate-size 2048 \  
--moe-router-topk 8 --moe-router-score-function sigmoid \  
--moe-router-enable-expert-bias --moe-router-bias-update-rate 1e-3 \  
--moe-router-load-balancing-type seq_aux_loss --moe-aux-loss-coeff 1e-4 \  
--moe-router-num-groups 8 --moe-router-topk-limited-devices 4 \  
--moe-router-dtype fp32 \  
  
# --- Dispatcher: cross-node ---  
--moe-token-dispatcher-type flex --moe-flex-dispatcher-backend deeppep \  
--moe-shared-expert-overlap \  
  
# --- 并行: PP + EP (无 TP) ---  
--expert-tensor-parallel-size 1 \  
--expert-model-parallel-size 64 \  
--pipeline-model-parallel-size 16 \  
--num-layers-per-virtual-pipeline-stage 4 \  
  
# --- FP8 ---  
--fp8-format e4m3 --fp8-recipe blockwise --fp8-param-gather \  
--moe-router-padding-for-fp8 \  
  
# --- Memory ---  
--recompute-granularity selective \  
--recompute-modules mla_up_proj layernorm moe_act moe_mlp core_attn \  
--fine-grained-activation-offloading \  
--offload-modules expert_fc1 moe_act \  
--use-precision-aware-optimizer --exp-avg-dtype bf16 --exp-avg-sq-dtype bf16 \  
  
# --- Overlap ---  
--overlap-moe-expert-parallel-comm --delay-wgrad-compute \  
  
# --- CUDA graph ---  
--cuda-graph-impl transformer_engine \  
--cuda-graph-modules attn moe_router moe_preprocess \  
  
# --- 稳定 ---  
--manual-gc --manual-gc-interval 10
```

环境变量 :

```
export CUDA_DEVICE_MAX_CONNECTIONS=8          # 多 stream overlap
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
export NCCL_NVLS_ENABLE=0
export NCCL_MIN_NCHANNELS=8
```

DeepSeek-V3 论文报告的成本：**180K H800-GPU-hours per T tokens** $\approx$ 3.7 天 @ 2048 GPUs。全 14.8T tokens pretrain **2.664M GPU-hrs**，按 \$2/GPU-hr 估算 **\$5.3M**。

常见坑

(1) All-to-all 的 send/recv counts 不匹配

每个 rank 发出的 counts 必须等于对应 rank 收到的 counts。这需要先做一次 **all-to-all** 交换 counts，再做数据 all-to-all。生产实现 (Tutel all_to_all) 内部封装了这一步。

(2) Backward 的 all-to-all

torch.distributed.all_to_all_single 的 autograd 反向自动生成——但要手动指定 **async_op** 或用 Megatron 的 MoELayer 封装，否则默认阻塞。

(3) Checkpoint 保存/加载的 expert sharding 一致性

EP=8 训练的 checkpoint 换到 EP=4 加载时，需要重新 **shard**。用 torch.distributed.checkpoint 或 Megatron dist ckpt 格式，保存元数据。

(4) Router 在 TP 组内 replicate

Gate 是小 tensor，全 rank 都存一份；但梯度合并要 AllReduce(TP)。不然不同 TP rank 上 router 参数 drift。

(5) M_{local} imbalance

跨 rank 的 tokens 数 (all-to-all 后每卡实际负载) 可能不均——同一步内某卡收 $2\times$ ，某卡 $0.5\times$ 。expert 内 grouped GEMM 是这卡本地的，不同卡计算时间不同 $\rightarrow$ 同步 barrier 拖慢整个 group。缓解：capacity 上限、Expert Choice、Megablocks。

面试考点

面试考点. Q1: EP 和 TP 都可以切 expert，为什么 EP 优先？

A: TP 切 expert 后每卡还是有 $\frac{E}{TP}$ 个“部分专家”——本质是 dense 化。EP 让每卡持有完整专家的一子集——利用 MoE 的稀疏性降低单卡权重，同时通信只对涉及的 token 做 (all-to-all)，比 TP 的 AllReduce 更符合 MoE 稀疏语义。

面试考点. Q2: All-to-all 的通信量为什么与 E 无关？

A: All-to-all 传的是每 token 的 **hidden vector**，总量 = 每卡 tokens $\times K \times H$ 。K 决定通信量 (1 个 token 走多少专家)， E 决定专家池大小、但每 token 走的还是 K 个——加多专家不增加通信。这也是 DeepSeek 用 256 experts 却通信开销不变的原因。

面试考点. Q3: 层次化 all-to-all 的收益怎么来的？

A: NVLink 400 GB/s vs IB 50 GB/s。同一 8 卡节点内的通信用 NVLink，跨节点用 IB。层次化把 $(8 \times N)$ -way all-to-all 拆成 8-way NVLink + N -way IB + 8-way NVLink，等价的 IB 带宽消耗从 $O(N^2KH)$ 降到 $O(NKH)$ 。对 100+ 节点场景差 10 $\times$ 。

面试考点. **Q4**: MoE 训练里 aux loss 的梯度怎么在分布式下汇总？

A: aux loss 每 rank 本地算(gate_probs 本地全)——但 f_e (per-expert token count) 需要跨 DP 组 AllReduce，不然是“per-DP-rank load”不是“global load”。生产实现里这个 AllReduce 常被遗忘，导致 aux loss 训不到位。

面试考点. **Q5**: EP 组内的 grouped GEMM，输入 shape 是什么？

A: 本地 expert 数 = $\frac{E}{EP}$ ，每个 expert 收到的 tokens 数 = $M_{\text{local},e}$ (all-to-all 之后)。所以 grouped GEMM 输入是 packed $(\sum_e M_{\text{local},e}, H)$ ，group_sizes 是 $\frac{E}{EP}$ 个 int。所有 EP rank 各跑一份，彼此独立。

面试考点. **Q6**: DeepSeek-V3 的 fine-grained overlap 为什么能显著加速？

A: 4 阶段 pipeline (dispatch a2a / attn / mlp / combine a2a) 让 GPU 一直有 work——通信不再是 blocking。传统实现里通信占 forward 15-30%，overlap 后接近 0%。代价是调度复杂度：需要 async collective + 精心排布的 CUDA stream。

面试考点. **Q7**: 单节点 8 卡上 EP=8 vs EP=4+TP=2 哪个好？

A: 看模型。 $E \geq 8$ 时 EP=8 通信更稀疏 (a2a 只在 EP 组内)。但 EP=8 要求 world_size ≥ 8 ，且 expert 数必须整除 EP；EP=4+TP=2 更灵活、每卡持 2 expert。生产 (Megatron) 允许 hybrid，实测选。

面试考点. **Q8**: MoE 的 pipeline parallel (PP) 有什么特殊？

A: PP 沿 layer 切与 MoE 内部结构正交。但 MoE 层的 all-to-all 会阻塞 pipeline bubble——上游 PP stage 的 all-to-all 未完成，下游 stage 无法开始。DeepSeek 用了“pipeline 内的 fine-grained overlap”缓解，但复杂度非常高。工业界一般 PP 层数少 (2-4)，让 MoE 主要走 EP。

模型层与低精度：Fine-grained、Shared、MLA、FP8

前两章讲怎么把训练跑快，这一章讲怎么把模型改得更适合训练与推理。都是“改模型结构”或“改数值精度”层面的优化——与前面的 kernel / 通信优化正交，可以叠加。

主线：

1. 专家结构：细粒度 (fine-grained) 与共享 (shared) 专家，2024 年之后的 SOTA MoE 都走这条
2. 低秩化：LatentMoE / MoLAE / MLA，把 hidden 维压掉一部分省通信和参数
3. 低精度训练：FP8 recipe，DeepSeek-V3 是当前唯一大规模验证过的
4. 生命周期：Upcycling (dense $\rightarrow$ MoE) Pruning / Merging (MoE $\rightarrow$ 小 MoE)

细粒度专家 + Shared Expert：DeepSeekMoE 范式

动机：从 Mixtral 到 DeepSeekMoE

Mixtral 8 $\times$ 7B (2023) 用 $E = 8$, top-2。DeepSeekMoE (2024)⁴ 观察到： E 少而每个 expert 大，实际上限制了“专家专精”——每个 expert 被迫承担多种技能，router 学不好精细划分。

两条改动：

1. **Fine-grained segmentation**：把每个 expert 的 FFN intermediate dim 缩到 $\frac{1}{m}$ ，然后 expert 数变 m 倍、top-K 变 m 倍，**iso-FLOP**
2. **Shared expert isolation**：留出 N_s 个 shared expert，每个 token 都必经过，吸收“公共知识”

组合数爆炸得很直观：Mixtral top-2 / 16 experts 只有 $\binom{16}{2} = 120$ 种组合；DeepSeek-V3 top-8 / 256 experts 有 $\binom{256}{8} \approx 4.09 \times 10^{14}$ ——路由空间大了 12 个数量级。

结构公式

$$y = \underbrace{\sum_{i=1}^{N_s} \text{FFN}_i^{\text{shared}(x)}}_{\text{always-on}} + \underbrace{\sum_{k=1}^{K_r} \tilde{w}_k \cdot \text{FFN}_{i_k}^{\text{routed}(x)}}_{\text{top-K routed}}$$

DeepSeek-V3 配置： $N_s = 1, N_r = 256, K_r = 8$ ，expert intermediate dim = 2048，sigmoid gating。

训练代码骨架

```
class DeepSeekMoELayer(nn.Module):
    def __init__(self, H, I_r, num_routed, num_shared, top_k):
        super().__init__()
        self.router = DeepSeekV3Router(H, num_routed, top_k) # 见第 6 章
        self.routed = nn.ModuleList([Expert(H, I_r) for _ in range(num_routed)])
        self.shared = nn.ModuleList([Expert(H, I_r * num_shared) for _ in
range(1)])
```

⁴Dai et al., DeepSeekMoE, arXiv:2401.06066.

```

# 注: shared expert intermediate 通常合并 (单个更大的 FFN),
# 因为它没有稀疏性可利用

def forward(self, x):
    # x: (N, H)
    # 1. shared: 每 token 都算
    y_shared = self.shared[0](x)  # (N, H)
    # 2. routed: 常规 top-K MoE
    weights, indices, logits = self.router(x)
    y_routed = dispatch_and_combine(x, weights, indices, self.routed)
    # 3. 相加
    return y_shared + y_routed, logits

```

收益与代价

收益：

- Specialization 显著↑，同 FLOP 下 downstream loss 更低（2B 规模 DeepSeekMoE 匹配 1.5× 大的 GShard）
- Router 更容易学（选择空间大但不需要 aux loss 强约束）
- Fine-grained 让 aux-loss-free bias tuning 更平滑（ b_i 微调影响小）

代价：

- Kernel 碎片化——256 个小 expert 的 grouped GEMM 效率比 8 个大 expert 差，通信 / launch overhead 占比↑。这正是 §8 讲的 compute wall——需要 Sync-Free + CUDA Graph + DeepEP V2 才能压回来
- Shared expert 是“额外 dense 计算”，每 token 都算——直接让 activation params 从 $K_r \cdot P_{\text{routed}}$ 增加 $1 \cdot P_{\text{shared}}$

Key insight. 从系统角度看，**fine-grained + shared** 不是纯粹的模型改进——它是“用 kernel 挑战换算法收益”。如果你没有 DeepEP / HybridEP 级别的 a2a 库，256 experts 的通信会拖垮训练。生产选择： $E \in [16, 64]$ 、 $K \in [2, 4]$ 是 kernel 挑战与算法收益的甜蜜点， $E \geq 128$ 需要专门的 infra 团队。

LatentMoE：把 hidden 压掉一部分

核心思路

LatentMoE⁵ 在 routed expert 路径外套两层共享投影：

$$x_{\text{lat}} = D_{\text{down}(x)} \in \mathbb{R}^{\ell}, \quad \ell = \frac{d}{\alpha}, \quad \text{常用 } \alpha = 4$$

$$y_{\text{lat}} = \sum_k \tilde{w}_k \cdot \text{FFN}_{i_k}^{\text{routed}(x_{\text{lat}})}$$

$$y = D_{\text{up}(y_{\text{lat}})}, \quad y \in \mathbb{R}^d$$

Router 和 shared expert 依然在 d 维上算。变化：

- Dispatch/combine 通信量： $\times \frac{1}{\alpha}$

⁵arXiv:2601.18089

- Expert 权重读取带宽： $\times \frac{1}{\alpha}$
- 参数分布：expert weights 从 $O(d^2)$ 变 $O(d \cdot \ell) = O\left(\frac{d^2}{\alpha}\right)$ ——但 $D_{\text{up}}, D_{\text{down}}$ 是全 d 的共享大矩阵

两种变体

1. **t-MoE_eff**：省下的带宽不复用，直接减少 GPU 使用（inference-friendly）
2. **t-MoE_acc**：把省下的带宽换算力——增加 $N_{r'} = \alpha N_r$ 或 $K_{r'} = \alpha K_r$ ，**iso-compute** 但精度更高。95BT pretrain (8B activated) 报告 MMLU Pro **+5.65 pp**

与 **DeepSeekMoE** 的关系

不是同一件事。DeepSeekMoE 是“拆细 + shared”；LatentMoE 是“投影到低维再做 routed”。可以叠加：LatentMoE + shared expert + fine-grained routed 是 Nemotron-3 的做法。

Multi-head Latent Attention (MLA)

严格说 MLA 不是 MoE 技术，是 attention 侧的 KV cache 压缩——但 DeepSeek-V2/V3 把它与 MoE 一起用，训练/推理系统必须一起处理，所以放这里。

机制

标准 MHA 每 token cache $2n_h \cdot d_h$ 个数值 (K + V)；MLA 把 KV 联合压缩到 latent $c_t \in \mathbb{R}^{d_c}$ ，只 cache c_t 和一个 decoupled RoPE key k_t^R ：

$$c_t^{\text{KV}} = W^{\text{DKV}} x_t, \quad c_t \in \mathbb{R}^{d_c}, \quad d_c = 512 \text{ (V3)}$$

$$K_t, V_t = W^{\text{UK}} c_t, W^{\text{UV}} c_t \quad \text{推理时不 materialize, 融合到 attn 里}$$

每 token cache 大小从 $2n_h d_h L$ (V3 是 $128 \times 128 \times 2 = 32\text{K}$) 减少到 $(d_c + d_h^R)L = 576$ ——**57×** 更小。DeepSeek-V2 相对 DeepSeek 67B (MHA) KV cache 减少 **93.3%**，推理吞吐提升 **5.76×**。

训练精度

MLA 的 down/up projection 参与训练，但保持 **BF16**（不进 FP8 通道）。DeepSeek-V3 §3.3.1 明确列出 attention 和 output head 不做 FP8。

FP8 训练：DeepSeek-V3 recipe

FP8 训练是 2024-2026 的 MoE 训练主战场。DeepSeek-V3 是第一个大规模 (671B, 14.8T tokens) 端到端 FP8 pretrain 成功的公开工作。

各 **tensor** 精度选择

DeepSeek-V3 §3.3 的完整精度表：

组件	精度	说明
Linear GEMM (Fprop/Dgrad/Wgrad)	FP8 E4M3	三路 GEMM 全 FP8 ,输出 BF16/FP32
Master weight	FP32	optimizer 持有
Weight gradient	FP32	batch 内累积
Optimizer state (Adam moment)	BF16	master weight 仍 FP32
Activation cache (Wgrad 用)	FP8	SwiGLU 输入也 FP8 + recompute
Attention 后 Linear 输入	E5M6 自定义	敏感路径, scale 2 的幂
MoE gate / router	BF16 / FP32	高精度保留
Embedding, output head, LN, Attention	BF16 / FP32	稳定训练
MoE dispatch activation	FP8	up-projection 前量化, 通信量 -50%
MoE combine (fwd + bwd)	BF16	关键路径保精度
MoE activation grad (down-proj 前)	FP8	减 backward 通信

表 18 DeepSeek-V3 FP8 训练精度分配。原则 :**GEMM 用 FP8 ,reduce/accumulate/router 用高精度。**

Fine-grained scaling

FP8 动态范围有限 (E4M3: $[-448, 448]$), outlier 一多就 clip。DeepSeek 用 **fine-grained tile/block scaling** :

- **Activation:** 1×128 tile-wise (per token per 128 channels)
- **Weight:** 128×128 block-wise
- **Scale:** 2 的整数幂 (硬件友好)
- **Format:** 全链路 E4M3 (不用混合 E4M3/E5M2), 靠 scaling 扩动态范围

TensorCore 累加精度

H100 TensorCore FP8 累加实际只有 14 bit, 当 K 维大 ($K=4096$) 时误差可达 **2%**。解决 : 每 $N_C = 128$ MMA 提升到 CUDA Core **FP32** 累加, 再继续。

框架支持

框架	机制	Flag
TransformerEngine	Float8BlockScaling (Hopper), inspired by DS-V3	fp8_recipe=blockwise
TransformerEngine	MXFP8BlockScaling 32-element scaling (Blackwell)	fp8_recipe=mxfp8
TorchAO	MXFP8 EP a2a dispatch/combine 专门 kernel	a2a_dispatch_mxfp8_fwd_hp_bwd 等
Megatron-Bridge	-fp8-recipe blockwise + DeepEP dispatcher	见 §8 config B
TensorRT-LLM	推理侧 EP a2a FP8	fp8_combine=True
FP8-Flow-MoE ⁶	端到端 FP8 dataflow (研究代码)	671B 上 +21% throughput, -16.5 GB peak

表 19 FP8 MoE 训练的生产框架。TransformerEngine 的 blockwise recipe 明确写“inspired by DeepSeek-V3”。

精度损失

DeepSeek-V3 报告 :FP8 vs BF16 pretrain 相对 val loss error **< 0.25%**(论文 Appendix B.1), 在训练随机性范围内。全 671B 未做端到端 ablation(成本原因), 只在 16B / DS-V2-Lite 规模验证。

Warning. FP8 训练不是 **free lunch**。数值故障常见 : Dgrad 用 block-wise quantization 曾导致 DeepSeek 16B 模型在 300B tokens 处发散 (Appendix B.2)。生产建议 : 先跑一个 BF16 baseline , 再用 FP8 做等比训练验证 loss 曲线 diverge < 0.5% , 再放大。

生命周期 : Upcycling 与 Pruning / Merging

Upcycling: Dense → MoE

Sparse Upcycling⁷ 提出 : 从预训练 dense checkpoint 出发 , 复制 FFN 权重成 E 份 expert , attention/norm 直接拷贝 , router 随机初始化。

收益 : T5/ViT 上 **50% dense** 预训练成本即可超过原 dense ; 也超过 from-scratch MoE。

Megatron flag :

⁶arXiv:2511.02302

⁷Komatsuzaki et al., Sparse Upcycling, arXiv:2212.05055.


```
--moe-use-upcycling
--moe-upcycling-granularity 1          # 每 dense FFN 拆几份
```

生产应用：Qwen1.5-MoE、GLM-4-MoE 都用了 upcycling。

Pruning / Merging: MoE → 小 MoE

推理侧压缩。有两类方法：

1. **Pruning** (去掉少用 expert) : REAP (arXiv:2510.13999) 证明 **pruning >> merging** 在生成任务上；50% 压缩 near-lossless
2. **Merging** (合并相似 expert) : MoE-SVD (ICML 2025)、Sub-MoE (arXiv:2506.23266)——25%/50% 削减保留 96%/86% 精度；小 MoE model routing 更简单

Expert 量化 (PTQ, 推理侧)

MoE 特有 PTQ 挑战：expert 激活极度不均衡 → calibration 数据分布 skewed。

- **Router** 必须高精度 (W8A8+) : logit 微扰 → 路由翻转 → 灾难性误差传播
- **Shared expert** 应分配更高 bit : 每 token 都激活，误差累积快
- **EAQuant** (arXiv:2506.13329) : expert-aware smoothing + router logit alignment ,W4A4 avg +1.15-1.37% vs DuQuant

面试考点

面试考点. **Q1**: 为什么 DeepSeek 从 Mixtral 的 $E = 8, K = 2$ 改成 $E = 256, K = 8$? 系统上代价是什么？

A: 算法上组合数 $10^{12} \times$,router 学到更精细专业化 ;系统代价 :a2a 通信量不变(只与 KH 有关 , 与 E 无关 , Node-Limited Routing 保证), 但 grouped GEMM 的 M_e 更小、更 skewed → kernel 效率下降。所以 DeepSeek 同时需要 DeepEP、Sync-Free、ECHO 才让 fine-grained 跑得动。

面试考点. **Q2**: Shared expert 与 residual (skip) connection 有什么区别？

A: Residual 是 $y = x + f(x)$, 无参数、恒等映射 ; shared expert 是 $y = \text{FFN}_s(x) + \text{MoE}(x)$, 有独立参数、每 token 必算。作用是承担“公共知识”(语言基本规律), 让 routed expert 专注专业化。DeepSeek-MoE 的实验显示去掉 shared expert 后多数下游 benchmark 显著下降。

面试考点. **Q3**: MLA 和 Grouped Query Attention (GQA) 的对比？

A: 都是 KV cache 压缩。GQA 是简单的“多个 query head 共享一份 K/V”(cache 减 $\frac{n_h}{n_g}$ 倍); MLA 是“KV 投影到低秩 latent ,推理时只 cache latent”(cache 减 $57\times$)。MLA 更激进、精度更高、需要 decoupled RoPE 处理位置 ,实现更复杂。V3/V2 用 MLA , Llama-3、Mistral、Gemma 用 GQA。

面试考点. **Q4**: DeepSeek-V3 的 FP8 训练里 ,为什么 MoE combine 保持 BF16 而 dispatch 用 FP8 ?

A: Dispatch 是 sender-side operation : token 从本卡出发, 能容忍量化误差 (下游还有 GEMM 会“稀释”这个误差)。Combine 是 receiver-side 的加权求和: 多份来自不同 expert 的结果相加, 量化误差会累积 (K 项相加)。且 combine 的输出直接送 residual + next-layer attention, 误差被放大。所以 combine 必须 BF16。

面试考点. **Q5:** Sparse Upcycling 相比 from-scratch MoE 训练的核心优势?

A: 复用 dense 已经学到的“通用知识”, MoE 只需要额外学“路由 + 专业化”。等价于给 MoE 一个非常好的 warm start。代价: expert 初始都是 dense FFN 的 copy $\rightarrow$ 严重相似 $\rightarrow$ 早期 collapse 风险高, 需要 warmup router 或加大 aux loss 系数一段时间。

面试考点. **Q6:** 如果我要压缩一个 $8 \times 22B$ MoE 到能在 $8 \times A100$ 上推理, 选 pruning 还是 merging?

A: 生成任务优先 **pruning** (REAP 论文结论: merging 会导致 functional subspace collapse, 生成任务掉分明显); 判别任务两者接近。实操: 先按 router 频次统计各 expert 使用率 $\rightarrow$ 剪掉最冷 25-50% $\rightarrow$ 短暂 finetune (10B tokens) 恢复精度。加上 W4A16 量化能进一步 $2 \times$ 内存节省。

数值稳定性、精度与常见陷阱

这一章是“实操 debug 手册”——把前几章散落的正确性问题、数值坑集中在一处，加上一些经验教训。写 MoE 代码的过程 = 踩坑的过程；先看到坑，再写代码。

分类

MoE 的坑大致分五类：

类别	举例
数值精度	softmax fp32、logit overflow、accumulator dtype
边界情况	空专家、空 batch、全 masked 行、 $K = E$
路由 semantics	renorm vs softmax(topk)、tie-break、bias 时序
Autograd	topk 反向、index_add 确定性、all-to-all backward
分布式	count 不匹配、TP router replication、ckpt reshard

数值精度类

Softmax 必须 fp32

第 3 章已详解，这里补一个 fp16 训练不 **upcast** 的具体后果：

假设 $E = 32$ ，router logits 在训练中期近似 $N(0, 5)$ 分布。fp16 下 $\exp(5) \approx 148$ ， $\exp(-5) \approx 0.0067$ ，比值 2×10^4 ——尚可。但训练早期若某个 logit 由于初始化偏大到 15：

- $\exp(15) \approx 3.3 \times 10^6$ — fp16 max 65504，直接 **inf**
- Sum = inf，除法后 all nan
- 一步 backward 后 gate weight 变 nan
- 整个模型死亡

用 fp32 accumulator： $\exp(15) = 3.3 \times 10^6$ 是合法 fp32 值，softmax 正常输出约 1 给该 expert（尖锐但有效）

诊断：训练中间若发现 gate weight 出现 nan 或极大值，第一 check：softmax 是不是 fp32？

Accumulator 与 GEMM dtype

Grouped GEMM 内部有两个 dtype：

- Input/output: bf16 (或 fp16)
- Accumulator: 应该 fp32

如果 kernel 用 bf16 accumulator（错误配置），大 K 维（如 $I = 14336$ ）的 dot product 会积累精度误差 10^{-2} 量级——梯度失真。

CUTLASS GroupedGemm 默认 fp32 accumulator。自己写 Triton kernel 要显式：

```
tl.dot(a, b, allow_tf32=True, out_dtype=tl.float32)
```

训练 vs 推理的精度切换

生产 pipeline 常见：

- 训练：bf16 参数 + fp32 optimizer + fp32 softmax
- 推理：bf16 参数 (或 fp8/int4) + fp16 softmax

推理换 fp16 softmax 的风险：如果训练时 gate logits 学到较大 magnitude (router 熵低)，推理 fp16 可能 overflow。ST-MoE 的 z-loss 就是为了训练时约束这一点。

诊断工具：训练后 `log max(abs(gate_logits))` per batch，应该 < 20 。

边界情况类

空专家 $M_e = 0$

某 batch 里某专家 0 token。行为要求：

范式 **A**: `if token_ids.numel() == 0: continue` — 必须。

范式 **B**: grouped GEMM 的 `group_sizes[e] = 0` 必须被 kernel 正确跳过。测试 fixture：手动构造 `expert_indices` 让某个 expert 从未出现，check `torch._grouped_mm` 输出正确。

Backward: 空专家的 $\partial_{\theta}^L W_e = 0$ (无贡献, 无梯度)，但 **optimizer step** 仍要执行——AdamW 会给 momentum 一个小 decay。不要跳过 optimizer step。

空 **batch** $N = 0$

不太常见 (batch size 通常大于 1)，但 GAN/RL 场景可能。处理：

```
if hidden_states.shape[0] == 0:
    return hidden_states, torch.empty(0, self.num_experts,
                                       device=hidden_states.device)
```

否则 `torch.topk` 对 (0, E) 的输入行为 undefined。

$K = E$ (全激活)

MoE 退化为 dense FFN 但计算重复。合法但性能差。用户可能出于 debug 目的这么做，代码不该报错。测试建议加：

```
def test_moe_k_equals_e():
    moe = MoE(H=8, I=16, E=4, K=4) # K = E
    y, _ = moe(x)
    # 应该数值等价于 sum of all expert outputs weighted by softmax
```

全 **masked** 行

Attention 场景，某 token 的所有 attention 被 mask 掉后 ($X = -\text{inf}$)，gate 输入是全 `-inf`：

$$\ell = W_g \cdot (-\infty) \rightarrow -\infty$$

$$p = \text{softmax}(-\infty, -\infty, \dots, -\infty) \rightarrow \text{nan}$$

必须在 gate 之前检查/替换全 mask 行：

```
mask = hidden_states.isfinite().all(dim=-1) # (N,)
safe_hidden = torch.where(mask.unsqueeze(-1), hidden_states, 0.0)
gate_logits = self.gate(safe_hidden)
# 对 masked token 输出 0 而非计算 MoE
```

生产实现要更 robust , 用 `attention_mask` 显式传入。

路由 semantics 类

Renorm 前后的 activation

前几章讲过两种 router 变体 , 还有一个隐含的坑 : `expert_weights` 传给后续计算的精度。

如果 renorm 用 fp32 但后面 mul 用 bf16 :

```
weights_fp32 = weights_fp32 / weights_fp32.sum(dim=-1, keepdim=True)
weights = weights_fp32.to(bf16)
# 后面: expert_output * weights
```

`weights` 的 bf16 表示可能让 `sum ≠ 1.0` 精确成立——一个 token 的 K 个权重和是 0.998 或 1.002。数学上小误差, 但训练稳定性: 如果整个模型对 “gate weight 精确 sum=1” 有暗含依赖 (很多 aux loss 计算), 会有几个 percent 的 loss 波动。

Fix: 保持 renorm 后的 weights 在 fp32 , 直到最后 output cast 时才降回。

Tie-break 行为

`torch.topk` 处理 tie 时选最先出现的——但 CUDA 上多线程/多 block 的 tie 处理是 **deterministic** 但 **platform-specific**。CPU vs GPU 结果可能不同 :

```
# CPU
torch.topk(torch.tensor([1.0, 1.0, 1.0, 0.5]), k=2)
# → values=[1, 1], indices=[0, 1]

# GPU 有时是
# → values=[1, 1], indices=[0, 2] (depending on kernel)
```

对训练不重要 (初始化会打破 tie), 但对 unit test 是问题——用 GPU-generated `expert_indices` 和 CPU reference 对比时要跳过 tie 情况。

DeepSeek Bias 更新时机

Bias tuning 每 step 更新 b_e ——放什么时候 ?

- Forward 前 ? 会污染 forward 用的 bias (本 step 用的和本 step 更新的不同)
- Forward 后 / optimizer.step 后 ? OK
- 每 100 step 一次 ? 可行但 lag 大

DeepSeek 官方实现是 “forward 后立刻用本 step 的负载统计更新”, 让下一 step 的 forward 用新 bias。

要注意 : 多 rank 情况下统计要 **AllReduce** 一次——本 rank 只知道本 rank 的负载。

Autograd 类

Topk 反向的意义

第 3 章讲过：topk 反向只把梯度给 top-K 位置。这不是 bug，但要理解：

```
w, i = torch.topk(x, k=2)  # x: (N, E), w: (N, 2), i: (N, 2)
loss = w.sum()
loss.backward()
# x.grad 只有 top-2 位置有值，其他 = 0
```

陷阱：如果你在 topk 输出上再做 gather，然后 backward，autograd 会给出正确梯度——只是“未选中位置”的梯度是 0。不要以为 gradient checker failed 了。

index_add_ 的确定性

第 4 章“一个隐藏的正确性坑”讲过：index_add_ 在 GPU 上默认非确定性。对训练可复现性有影响，但不影响正确性。

```
# 打开确定性
torch.use_deterministic_algorithms(True)
os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':4096:8'
```

代价 5-15% 慢。生产训练一般不开——loss curve 差异远小于训练噪声。

All-to-all 的 autograd

torch.distributed.all_to_all_single 支持 autograd——它的反向自动是 all_to_all with reversed counts。但要求：

- Send 和 recv counts 是 static (不依赖于 tensor 值)
- Async_op=False (aka blocking mode, 默认)

Async 版本 (async_op=True) 返回 work handle，autograd 不会自动等待——需要手动 .wait()。生产实现用 Megatron 的 MoELayer 会封装好。

分布式类

Send/recv counts 不匹配

Rank A 说要发 100 token 给 rank B，但 rank B 说要收 90 token——mismatch，all-to-all 死锁或数据 corruption。

Fix: 每个 all-to-all 之前先做一次 counts exchange：

```
send_counts = compute_locally(...) # (world_size,)
recv_counts = torch.zeros_like(send_counts)
dist.all_to_all_single(recv_counts, send_counts) # 交换 counts
# 现在 recv_counts 是 "别 rank 要给我发的量"
data = dist.all_to_all_single_uneven(data, recv_counts, send_counts)
```

Tutel 和 Megatron 内部封装了这个流程。

TP 组内 Router replication

Gate weight W_g : (E, H) 很小, 全 TP rank 都存一份。但 forward 里每个 TP rank 各算一次——结果应该相同 (deterministic if same input)。

问题: H 维在 TP 里被切了, attention output 出来是每 **TP rank** 各持一份 **replicated hidden** 吗?

- Megatron TP 里 output linear 是 row-parallel, 最后有 **AllReduce(TP)**——之后 hidden 在 TP 内 replicated。
- 但如果 sequence parallel (SP) 开启, hidden 在 TP 内是沿 **seq** 切的——不 replicated。

Router 需要 full hidden。所以在 SP 场景下 gate 前要 AllGather(TP) 恢复完整 hidden。

生产实现里这个 AllGather 是隐藏的: Megatron MoE 层内部自动插入, 用户不感知。

Checkpoint reshard

训练时 EP=8, 恢复时想改 EP=4 (比如换更少节点)——需要重新 shard expert weights:

```
原 EP=8: rank 0 持 W_up[0], rank 1 持 W_up[1], ..., rank 7 持 W_up[7]
新 EP=4: rank 0 持 W_up[0:2], rank 1 持 W_up[2:4], rank 2 持 W_up[4:6], rank 3 持 W_up[6:8]
```

推荐用 `torch.distributed.checkpoint` (2.x+) 或 Megatron 的 dist ckpt 格式——它们把 sharding 元数据编码进 checkpoint, 加载时自动 reshard。手写文件 IO 极易出错。

训练稳定性与调试

Loss 突然 NaN 的排查顺序

1. **Router** 输出: `torch.isfinite(gate_probs).all()` — 如果 nan, softmax 精度问题
2. **Expert** 输入: `torch.isfinite(packed_input).all()` — attention 出的 nan 传下来的
3. **Expert** 输出: 同上, 通常是权重梯度爆炸
4. **Aux loss**: 有时 aux loss 系数太大压制主 loss, check ratio
5. **All-to-all**: 通信中数据 corruption (rare, NCCL 稳定)

日志要 log 每层 `gate_logits.abs().max()` 和 `packed_input.abs().max()`——nan 出现前通常有几百 step 的“数值爬升”预警。

Reproducibility

MoE 完全 reproducible 训练 (bit-exact) 需要:

- `torch.use_deterministic_algorithms(True)`
- `CUBLAS_WORKSPACE_CONFIG` 环境变量
- 固定 all-to-all 顺序 (NCCL 有随机性)
- 固定 grouped GEMM tile scheduler

工业界一般不追求 bit-exact——只要 loss curve 在噪声范围内一致就 OK。

训练 vs 推理的路由差异

训练: 需要 aux loss (监督) + drop token (保证 GEMM shape) 推理: 不需要 aux loss; drop 会伤害精度, 用大 capacity 或 Megablocks

如果生产 pipeline 是“训练时 drop、推理时不 drop”, 会有 train/eval mismatch。缓解: 训练用 Megablocks 或大 capacity, 保证 drop 率 < 0.1%。

DeepSeek-V3 训练/推理都 no-drop，一致性好。

常见坑速查表

症状	可能原因	Fix
Loss 训到一半 NaN	Router softmax 非 fp32	显式 dtype=torch.float32
某专家不学习	Router collapse 早期	aux loss $\uparrow$, init std $\downarrow$
Loss 曲线抖动	aux loss 系数过大	降到 0.005-0.01
Gate logits 越来越大	无 z-loss	加 z-loss $\beta = 0.001$
Grouped GEMM nan	空专家 $M_e = 0$	跳过或用支持的 kernel
推理精度掉	训练 drop 太多	提高 capacity 或 Megablocks
多机训练卡住	a2a counts mismatch	先 exchange counts
Reshard 后 loss 突变	Ckpt 格式不对	用 dist ckpt API
多次跑 loss 不一致	index_add 非确定	use_deterministic_algorithms
Gate 梯度爆炸	batch 全走 1 个专家	提升 aux loss 或降 LR

面试考点

面试考点. Q1: Softmax 不用 fp32 会出什么问题？给一个具体数值例子。

A: fp16 下 $\exp(11.1) \approx 6.6 \times 10^4$ ，接近 fp16 上限 65504。如果 gate logit 到 15， $\exp(15) \approx 3.3 \times 10^6$ 直接溢出为 inf，softmax 输出 nan。fp32 支持到 $\exp(88) \approx 1.6 \times 10^{\{38\}}$ ，安全。

面试考点. Q2: torch.topk 处理相等值时的顺序保证？

A: CPU 上稳定 (按位置排)；GPU 上 deterministic 但 kernel-specific，不同 PyTorch 版本可能不同。要 reproducible 就在 CPU 上做 topk，或加小 noise 打破 tie。

面试考点. Q3: 训练时 drop token，推理时不 drop，会有什么问题？

A: Train/eval mismatch — 训练时 router 学到“选中但被 drop”的 token 无梯度信号，推理时该 token 走全部专家，行为分布不同。缓解：训练用大 capacity (≥ 2) 或 Megablocks 减少 drop；或 evaluate loss 时也开相同 drop 策略。

面试考点. Q4: 分布式 MoE 里，aux loss 的 f_e 需要跨 rank 同步吗？

A: 需要。 f_e 是“这个 batch (globally) 每 expert 的负载比例”——如果每 rank 各自算，得到的是“本 rank 的负载”，训练早期本 rank 恰好 collapse 到 expert 0，其他 rank collapse 到 expert 1，local aux loss 都 0，但 **global** 不均衡。必须 AllReduce(f) across DP-group。

面试考点. Q5: 为什么生产 MoE 训练不追求 bit-exact reproducibility？

A: index_add_、all_to_all 的非确定性会引入低位 bit 差异；bit-exact 需要牺牲 5-15% 性能。工业界只要 loss curve 在噪声范围内一致 (1σ 内) 就够。学术论文报告 3 seed avg + std 即可。

面试考点. **Q6**: Router 参数 (gate weight) 在 EP 组内是 replicated 还是 sharded?

A: Replicated (小)。每 EP rank 都算完整 router 输出, 然后基于 expert_indices 决定 all-to-all 目标。梯度也 replicated——但通过 DP AllReduce 保持一致 (跨 DP 组时)。

面试考点. **Q7**: 遇到 loss NaN, 检查顺序是什么?

A: (1) `gate_probs.isfinite()`; (2) `input to expert .isfinite()`; (3) `W_expert.grad.abs().max()` 是否爆炸;(4) aux loss 数值;(5) NCCL log 有无 corruption。90% 是 (1)——softmax 精度问题。

面试考点. **Q8**: EP=8 训练完的 checkpoint 换到 EP=4 加载, 实操怎么做?

A: 用 `torch.distributed.checkpoint.save/load` (PyTorch 2.x) 或 Megatron 的 `dist ckpt`。它们保存“每个 expert 的完整 tensor + placement 元数据”, 加载时按目标 EP 自动 reshard。手写 file IO 极易出错 (记住 expert 排列顺序、参数字典键名一致)。

附录 A：符号表 & 延伸阅读

符号表

符号	含义
B	batch size
S	sequence length
$N = BS$	flatten 后的 token 总数
H	hidden dim (Transformer 主维度)
I	FFN intermediate dim ($I \approx 4H$ 常见, SwiGLU 常 $\frac{8}{3}H$)
E	专家总数 num_experts
K	top-k, 每 token 激活的专家数
M_e	路由到专家 e 的 token 数 ($\sum_e M_e = NK$)
C	capacity, 每个专家在 batch 内允许接收的最大 token 数
c_{factor}	capacity factor, $C = \text{ceil}(N \frac{K}{E} \times c_{\text{factor}})$
f_e	expert e 被 top-1 (或 top-K) 选中的 batch 内比例
P_e	gate_probs 对 expert e 的 batch 均值
L_{aux}	auxiliary load-balancing loss, $= E \cdot (f \cdot P)$
L_z	router z-loss, $= \text{mean}(\text{lse}(\ell)^2)$
EP	expert parallel world size
TP	tensor parallel world size
DP	data parallel world size
PP	pipeline parallel world size
W_g	router (gate) weight, shape (E, H)
$W_{\text{up}}, W_{\text{gate}}, W_{\text{down}}$	SwiGLU expert 三个 Linear 的 weight
ℓ / gate_logits	router pre-softmax 输出 (N, E)
p / gate_probs	router 后 softmax 输出 (N, E)
exp_ids	expert_indices (N, K)
exp_wts	expert_weights (N, K)
packed_input	permute 后按 expert 排序打包的 input, (NK, H)
group_sizes	每 expert 的 token 数, $(E,)$
a2a	all-to-all

延伸阅读

经典论文

- Shazeer et al. 2017, “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”, arxiv:1701.06538 — 开山之作

- Lepikhin et al. 2020, “GShard: Scaling Giant Models with Conditional Computation”, arxiv:2006.16668 — 生产级 MoE, capacity+drop
- Fedus et al. 2021, “Switch Transformer”, arxiv:2101.03961 — $K = 1$ 极简路由
- Du et al. 2022, “GLaM: Efficient Scaling of Language Models with MoE”, arxiv:2112.06905 — 1.2T 参数 MoE
- Zoph et al. 2022, “ST-MoE: Designing Stable and Transferable Sparse Expert Models”, arxiv:2202.08906 — router z-loss, 稳定性
- Zhou et al. 2022, “Mixture-of-Experts with Expert Choice Routing”, arxiv:2202.09368 — 反向路由

生产模型

- Jiang et al. 2024, “Mixtral of Experts”, arxiv:2401.04088
- Databricks 2024, “DBRX: Introducing a New State-of-the-Art LLM”, blog — 使用 Megablocks
- DeepSeek-AI 2024, “DeepSeek-V3 Technical Report”, github/DeepSeek-V3 — 671B 参数, aux-loss-free, fine-grained overlap
- DeepSeek-AI 2024, “DeepSeekMoE: Towards Ultimate Expert Specialization”, arxiv:2401.06066 — 细粒度 + shared expert
- Qwen 2024, “Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters”, blog

Kernel / 系统

- Gale et al. 2022, “MegaBlocks: Efficient Sparse Training with MoE”, arxiv:2211.15841
- Hwang et al. 2022, “Tutel: Adaptive Mixture-of-Experts at Scale”, arxiv:2206.03382
- Rajbhandari et al. 2022, “DeepSpeed-MoE”, arxiv:2201.05596
- Cai et al. 2024, “Shortcut-connected Expert Parallelism for Accelerating MoE”, arxiv:2404.05019 — expert parallel overlap

代码仓库

- deepseek-ai/DeepEP — DeepSeek expert parallel 通信库
- NVIDIA/Megatron-LM — Megatron-LM MoE
- microsoft/tutel — Tutel
- databricks/megablocks — MegaBlocks
- vllm-project/vllm — 推理时 fused MoE kernel
- sgl-project/sglang — SGLang MoE 推理

博客

- HuggingFace, “Mixture of Experts Explained”, hf.co/blog/moe
- Yao Fu, “MoE Paradigm: Sparse Activations for Efficient LLMs”, notion
- Cameron Wolfe, “Mixture-of-Experts (MoE) LLMs”, substack

本仓库文件

- 最小 PyTorch 实现: `python/pytorch/test_moe.py`
- 运行 smoke test:

```
python python/pytorch/test_moe.py
```

- 本书源码: `books/moe/`
- 编译: `cd books/moe && typst compile book.typ`

附录 D : MoE 系统面试题 20 问

以下 20 题是 MoE 系统面试的高频问题，覆盖从基础概念到分布式实现。答案里注明了参考章节，可以回去精读。

基础概念 (Q1-Q5)

面试考点. **Q1**: 用一句话解释 MoE 相比 dense FFN 的核心思想。

A: 把一个大 FFN 拆成 E 个小专家，用 router 让每个 token 只走 K 个专家 ($K \ll E$)。参数量 $\times \frac{E}{K}$ ，激活计算量几乎不变。见第 2 章“一句话直觉”。

面试考点. **Q2**: Mixtral $8 \times 7B$ 里“ $8 \times 7B$ ”是什么意思？总参数量约 47B 是怎么来的？

A: 8 个专家，每个专家的 FFN 部分约 5.5B 参数。总参数 = $8 \times 5.5B$ (experts) + 5B (attention + embedding + norm) $\approx 47B$ 。“ $8 \times 7B$ ”是营销措辞而非精确算术。见第 2 章“里程碑速览”。

面试考点. **Q3**: 为什么现代 MoE 论文都强调“稀疏激活”？稀疏在哪一层？

A: 稀疏在 **expert** 选择上——每 token 只激活 $\frac{K}{E}$ 的专家。FLOPs 稀疏（大部分 expert 不计算），但显存不稀疏（所有专家权重都要装）。区别于“activation sparsity”（激活值多为 0）。见第 2 章“MoE 的核心思想”。

面试考点. **Q4**: MoE 相比 dense 训练需要更多什么？

A: 主要三样：(a) 显存，因为 E 倍权重；(b) 通信（分布式时 all-to-all）；(c) 代码复杂度（router + dispatch + load balancing）。计算量是节约的，但工程复杂度显著上升。见第 2 章“代价：三个新问题”。

面试考点. **Q5**: 什么情况下 MoE 不适合？

A: (1) 单机小模型 ($< 10B$) — dense 更简单，性能相当；(2) 严格显存受限 (edge deployment) — MoE 权重全存装不下；(3) BS 极小 (自回归 decode) — expert 内 GEMM 小到没效率；(4) 训练数据不足 — collapse 严重。

Router 与负载均衡 (Q6-Q10)

面试考点. **Q6**: 为什么 router 的 softmax 必须 fp32？给一个具体的 overflow 数字。

A: fp16 上限 65504， $\exp(11.1) \approx 66000$ 就溢出为 inf。训练中期 logit 值可到 ± 15 ， $\exp(15) \approx 3.3e6$ 必然 nan。fp32 支持 $\exp(88)$ 无 overflow。见第 3 章“为什么 softmax 要用 fp32”和第 9 章。

面试考点. **Q7**: `torch.topk` 会返回相同 expert id 两次吗？

A: 不会。topk(gate_probs, k=K) 从 E 个不同的位置选 K 个索引, 索引本身天然 unique。除非 tie (值完全相等) ——tie 处理是 stable ordering, 返回的还是不同 index。见第 3 章面试考点 Q2。

面试考点. **Q8**: Expert collapse 是什么? 怎么诊断? 怎么修?

A: Router 学到“永远选那几个 expert”, 其他 expert 收不到梯度, 参数浪费。诊断: $\frac{\text{std}(f_e)}{\text{mean}(f_e)} > 0.5$ 持续。修: (1) 检查 init (std=0.02); (2) 提升 aux loss $\alpha = 0.01 \rightarrow 0.03$; (3) 加 z-loss; (4) 换 DeepSeek 的 aux-loss-free bias tuning。见第 6 章“问题重述”和“Aux Loss”。

面试考点. **Q9**: Aux loss 的公式为什么是 $E \cdot (f \cdot P)$? 各项什么意思?

A: f_e = expert e 被 top-1 选中的比例 (不可微), P_e = gate_probs 均值 (可微)。均匀分布时 $f = P = \frac{1}{E}$, $L_{\text{aux}} = E \cdot E \cdot \left(\frac{1}{E}\right)^2 = 1$ (最小)。乘 E 是为了让最小值 = 1 (scale invariant)。见第 6 章“GShard/Switch Aux Loss”。

面试考点. **Q10**: DeepSeek-V3 为什么弃用 aux loss? bias tuning 怎么工作?

A: aux loss 给 router 冲突梯度 (主任务 vs 均衡)。DeepSeek 用可学习 bias b_e , 只影响 topk 选择 ($\hat{\ell} = \ell + b$) 不影响 gate 权重梯度。 b 用简单反馈规则调整: 过载专家 bias $\downarrow$, 欠载 bias $\uparrow$ 。router 梯度纯净。见第 6 章“DeepSeek-V3 的 Aux-Loss-Free 均衡”。

实现与性能 (Q11-Q15)

面试考点. **Q11**: 逐 expert for-loop (范式 A) vs Grouped GEMM (范式 B) 差在哪?

A: (1) A 有 E 次 kernel launch, B 只有 1 次; (2) A 里每个 GEMM 的 M 维小 (< 256), TensorCore 吃不满; B 内部全局 tile scheduling, SM 利用率高; (3) B 需要 permute+unpermute overhead。端到端 B 比 A 快 1.5-2 $\times$ 。见第 4 章“范式 A/B”和第 7 章。

面试考点. **Q12**: `expert_weights[token_ids, k_ids]` 的行为? 为什么不是 `expert_weights[token_ids][:, k_ids]`?

A: 前者是 2D fancy indexing, 两个 index 同步 broadcast, 输出 shape $(M,)$: 每个 (t, k) pair 取一个元素。后者是先取行再对每行的全部列取 k_ids , 输出 shape (M, M) 。是常见 bug。见第 5 章“Dispatch 循环”。

面试考点. **Q13**: `index_add_` 在 GPU 上有确定性问题吗? MoE 里这个问题在哪些地方触发?

A: 有——多个原子加的顺序取决于 CUDA 调度。在范式 A 的单次 expert 循环内 `token_ids` unique (不触发), 但跨 expert 循环、同一 token 被 K 次 add — 顺序影响低位 bit。要 reproducible 训练需 `torch.use_deterministic_algorithms(True)`。见第 4 章“一个隐藏的正确性坑”和第 9 章。

面试考点. **Q14**: Grouped GEMM 里 permute 阶段有多重要? 为什么要 fuse 进 GEMM prologue?

A: 独立 permute 要写读一次 (NK, H) tensor 到 HBM, 约几百 MB traffic。融进 GEMM prologue 直接从 hidden gather 到 shared memory, 省掉一次 HBM 往返, 端到端 +10-15%。见 §6, v3。

面试考点. **Q15**: 单机 MoE 的显存开销主要来自哪里?

A: (1) 权重: E 倍 FFN 权重是最大头; (2) **optimizer state**: AdamW 12 bytes/param $\times$ 权重量; (3) 激活: packed_hidden $(N K, I)$ 是最大 activation, selective ckpt 可省 30%; (4) grad 累积。见 §6。

分布式 (Q16-Q20)

面试考点. **Q16**: 为什么 MoE 需要 Expert Parallel (EP), 用 TP/DP 不够吗?

A: DP 每卡都要装完整 MoE — $E \times$ 权重装不下。TP 切 expert 内 GEMM, 切完后每卡还是有 $\frac{E}{TP}$ 个“部分专家”——本质是 dense 化, 没有利用稀疏。EP 让每卡持有完整 **expert** 的一子集, 权重 $\times \frac{1}{EP}$, 通信只在被路由的 token 上做 (a2a), 符合 MoE 稀疏语义。见第 8 章“为什么需要 Expert Parallel”。

面试考点. **Q17**: All-to-all 的通信量公式? 为什么与 E 无关?

A: 每 rank 通信量约 $N_{\text{local}}KH \times \text{bytes}$ 。传的是 token 的 hidden vector, 每 token 走 K 个专家、 K 份数据发出去、 K 份收回来。 E 只决定专家池大小, 不改变每 token 的通信。 $N_{\text{local}} = 4K, K = 2, H = 4K, \text{bf16} \rightarrow 128 \text{ MB per a2a}$ 。见第 8 章“通信量分析”。

面试考点. **Q18**: 层次化 all-to-all 是什么? 收益从哪来?

A: 把 $(N \text{ nodes} \times 8 \text{ GPUs})$ -way a2a 拆成 intra-node (NVLink, 400 GB/s) + inter-node (IB, 50 GB/s) + intra-node。IB 带宽消耗从 $O(N^2KH)$ 降到 $O(NKH)$ 。100+ 节点场景 wall-clock 差 10 $\times$ 。DeepSpeed-MoE、Tutel 都实现。见第 8 章“层次化 All-to-All”。

面试考点. **Q19**: DeepSeek-V3 的 fine-grained overlap 怎么工作? 为什么能加速?

A: 把每 MoE 层拆成 4 阶段 pipeline: dispatch a2a $\rightarrow$ attention $\rightarrow$ mlp $\rightarrow$ combine a2a。多个 micro-batch 交替, 每 stage 一直有 work, 通信持续 overlap 计算。传统实现通信占 forward 15-30%, overlap 后近 0%。代价: async collective + 精心排布的 CUDA stream。见第 8 章“Overlap”。

面试考点. **Q20**: 分布式 MoE 训练中 aux loss 的 f_e 需要 AllReduce 吗?

A: 需要 (跨 DP 组)。 f_e 是“全局 batch 内每 expert 的负载比例”——每 rank 各算得到的只是“本 rank 的负载”。训练早期可能本 rank collapse 到 expert 0、别 rank collapse 到 expert 1, local aux loss 都 0, 但 **global** 不均衡。必须 AllReduce(f)。这是分布式 MoE 训练最常被遗漏的一步。见第 9 章“Send/recv counts 不匹配”和相关小节。

加分题：动手实操

如果面试官愿意手撸代码，可能会问：

B1: 现场写一个最小 MoE forward (禁用 grouped GEMM)。

- 参考本书 §4 逐段实现。关键点：`view(-1, H)`、`F.softmax(dtype=fp32)`、`torch.topk`、`renorm`、`for expert_idx in range(E)` 循环、`torch.where`、`expert_weights[token_ids, k_ids]`、`index_add_`。

B2: 现场推导 aux loss 的最小值证明。

- 均匀分布 $f = P = \frac{1}{E}$ 时 $L = E \cdot \sum_e \left(\frac{1}{E}\right) \left(\frac{1}{E}\right) = E \cdot E \cdot \frac{1}{E^2} = 1$ 。要证是最小值：拉格朗日 or Cauchy-Schwarz。

B3: 现场画 EP=4, K=2, N_local=4 的 all-to-all 前后 tensor 布局。

- 画一个 4×4 的 (rank, expert) grid，每 rank 4 个 token 各走 2 个 expert，画出 send/recv counts 和 packed_input shape。

B4: 现场写一段 grouped GEMM 的 permute (不用 argsort)。

- Counting sort: `bincount` → `cumsum` → 得每 expert 的 start offset → scan `expert_indices` 填 `permute_idx`。生产实现用这个避免 argsort。